

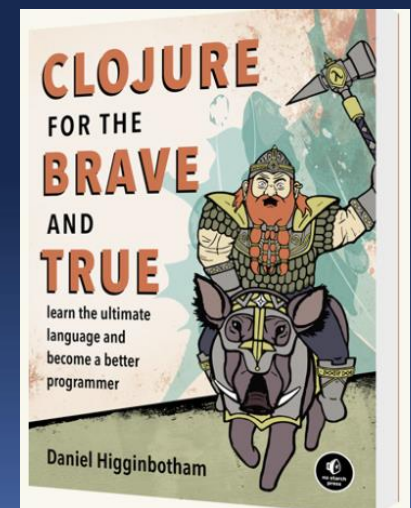
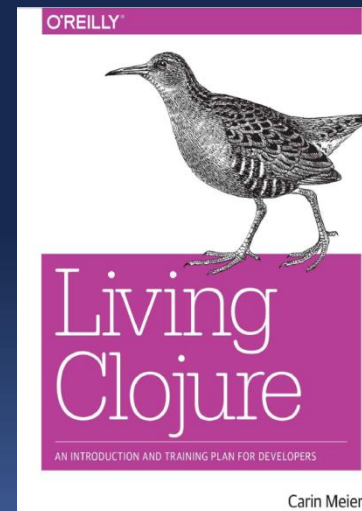
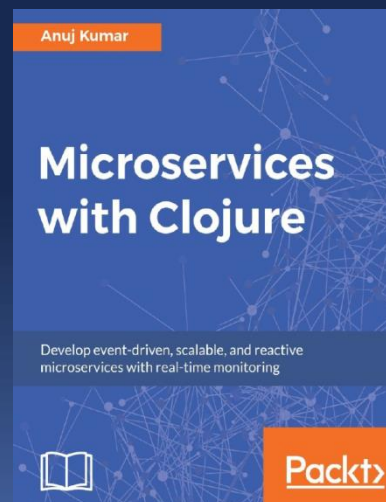
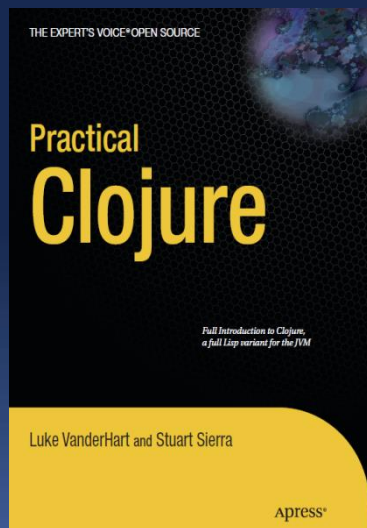
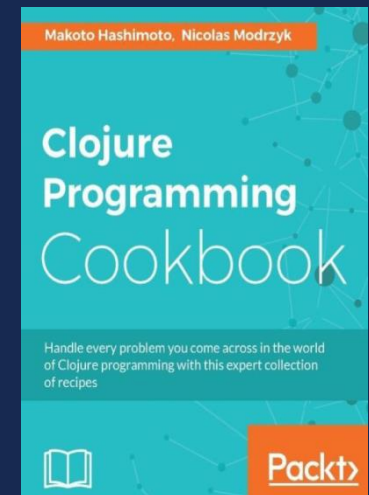
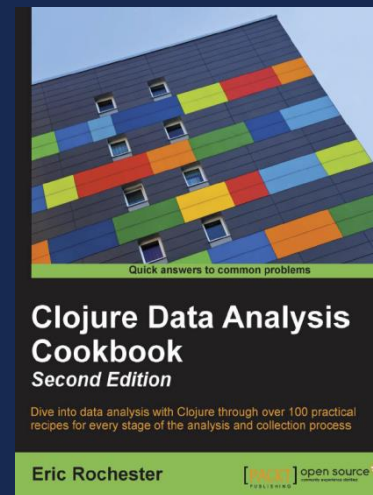
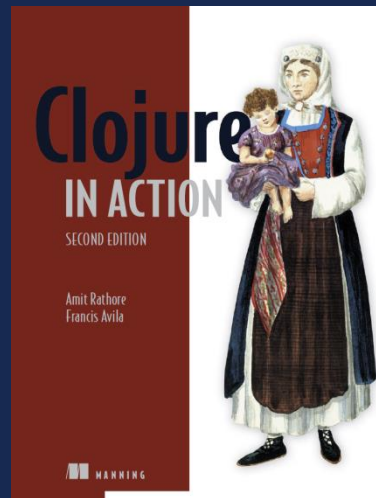
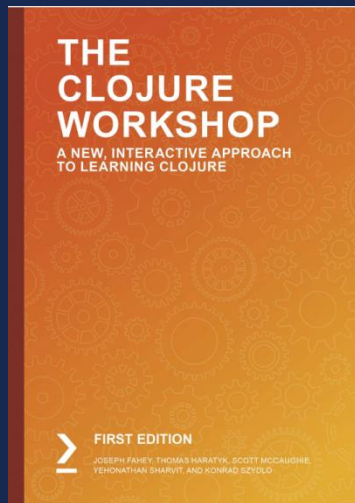
Programação Funcional

Unidade 4 – Visão geral da Linguagem Clojure



Prof. Aparecido V. de Freitas
Doutor em Engenharia
da Computação pela EPUVSP
aparecido.freitas@maua.br
aparecidovfreitas@gmail.com

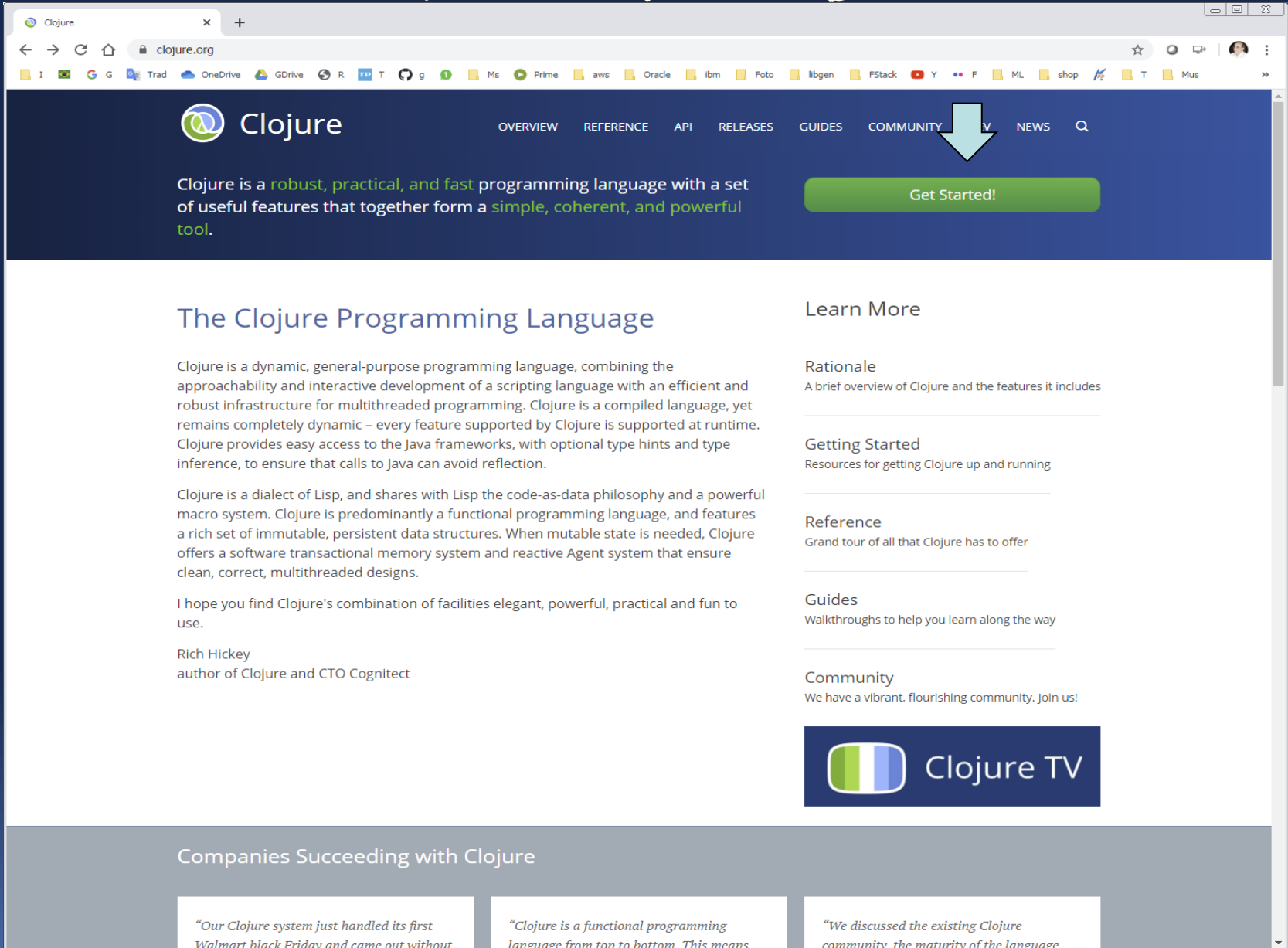
Bibliografia



Antes de iniciar

- ✓ Antes de iniciar a instalação do **Clojure**, esteja certo de que **JDK** está instalado;
- ✓ **Clojure** é implementado em **Java** e roda na **JVM** (Java Virtual Machine);
- ✓ **Clojure** é desenhado para ser uma **hosted** language, enquanto que uma outra implementação, chamada **ClojureScript**, roda em um runtime **JavaScript**, por exemplo, um **web browser** ou **Node.js**;
- ✓ **Clojure** não requer uma versão particular da Máquina Virtual Java, mas, recomenda-se que a versão **Java 8** seja instalada;
- ✓ Instruções de instalação em <https://clojure.org/>
- ✓ É possível interagir códigos **Clojure** com libraries (**packages**) escritos em **Java** (API'S escritas em Java);

https://clojure.org/



The screenshot shows the Clojure.org website. At the top, there's a navigation bar with links: OVERVIEW, REFERENCE, API, RELEASES, GUIDES, COMMUNITY, NEWS, and a search icon. A green arrow points to the 'COMMUNITY' link. Below the navigation bar is a large blue banner with the Clojure logo and the text: "Clojure is a robust, practical, and fast programming language with a set of useful features that together form a simple, coherent, and powerful tool." A green button labeled "Get Started!" is on the right. Below the banner, the main content area is divided into two columns. The left column is titled "The Clojure Programming Language" and contains a paragraph about Clojure's dynamic, general-purpose nature, its approachability, and its robust infrastructure. It also mentions that Clojure is a dialect of Lisp and shares the code-as-data philosophy. The right column is titled "Learn More" and contains links to "Rationale", "Getting Started", "Reference", "Guides", and "Community". At the bottom, there's a section titled "Companies Succeeding with Clojure" with three quotes from users.

The Clojure Programming Language

Clojure is a dynamic, general-purpose programming language, combining the approachability and interactive development of a scripting language with an efficient and robust infrastructure for multithreaded programming. Clojure is a compiled language, yet remains completely dynamic – every feature supported by Clojure is supported at runtime. Clojure provides easy access to the Java frameworks, with optional type hints and type inference, to ensure that calls to Java can avoid reflection.

Clojure is a dialect of Lisp, and shares with Lisp the code-as-data philosophy and a powerful macro system. Clojure is predominantly a functional programming language, and features a rich set of immutable, persistent data structures. When mutable state is needed, Clojure offers a software transactional memory system and reactive Agent system that ensure clean, correct, multithreaded designs.

I hope you find Clojure's combination of facilities elegant, powerful, practical and fun to use.

Rich Hickey
author of Clojure and CTO Cognitect

Learn More

Rationale
A brief overview of Clojure and the features it includes

Getting Started
Resources for getting Clojure up and running

Reference
Grand tour of all that Clojure has to offer

Guides
Walkthroughs to help you learn along the way

Community
We have a vibrant, flourishing community. Join us!

Clojure TV

Companies Succeeding with Clojure

"Our Clojure system just handled its first Walmart black Friday and came out without"

"Clojure is a functional programming language from top to bottom. This means"

"We discussed the existing Clojure community, the maturity of the language"

Instruções de Instalação

Local build

Download and build Clojure from source (requires Git, Java, and Maven):

```
git clone https://github.com/clojure/clojure.git
cd clojure
mvn -Plocal -Dmaven.test.skip=true package
```

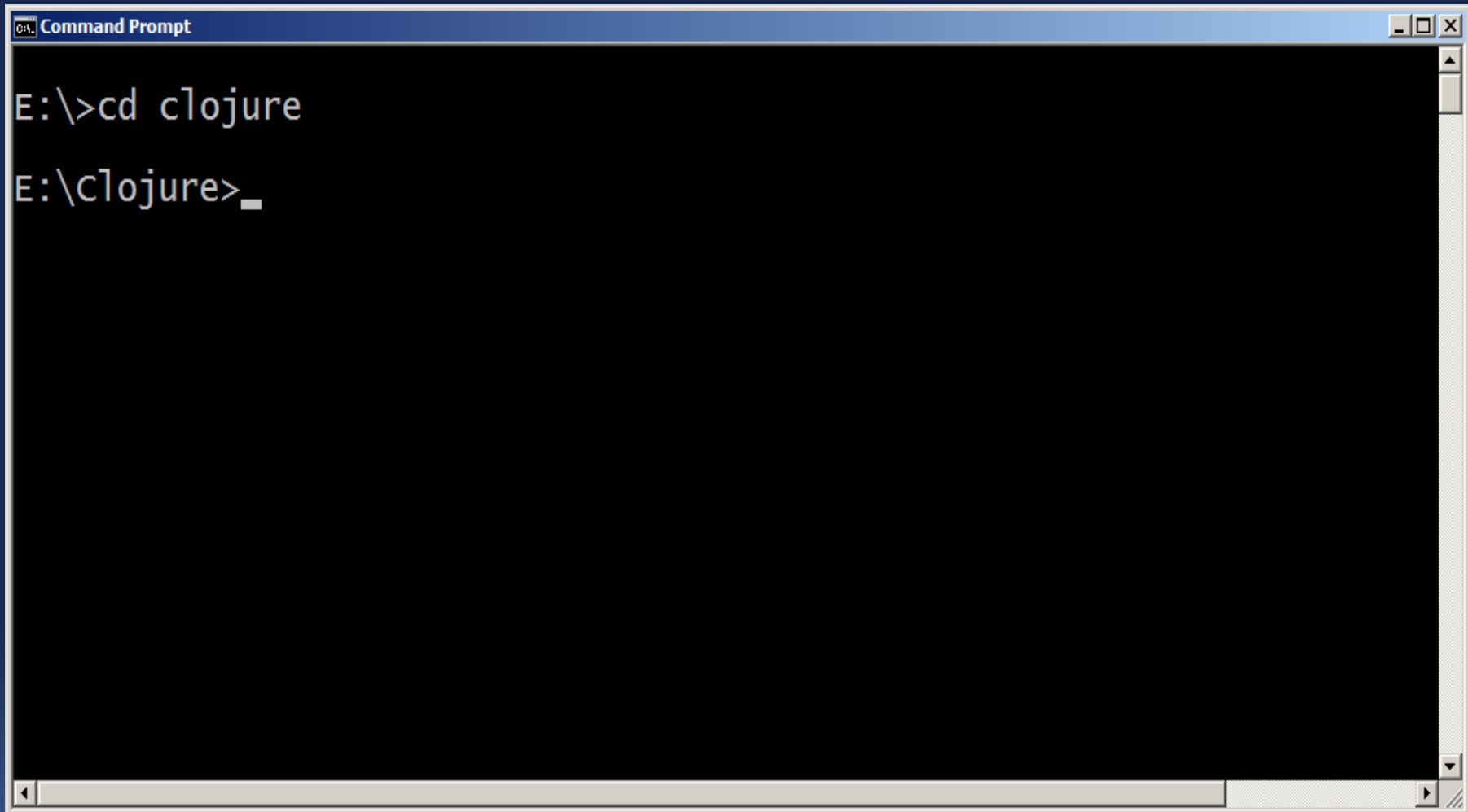
Then start the REPL with the local jar:

```
java -jar clojure.jar
```

Try Clojure online

repl.it provides a browser-based Clojure repl for interactive exploration.

Criar pasta E:\clojure

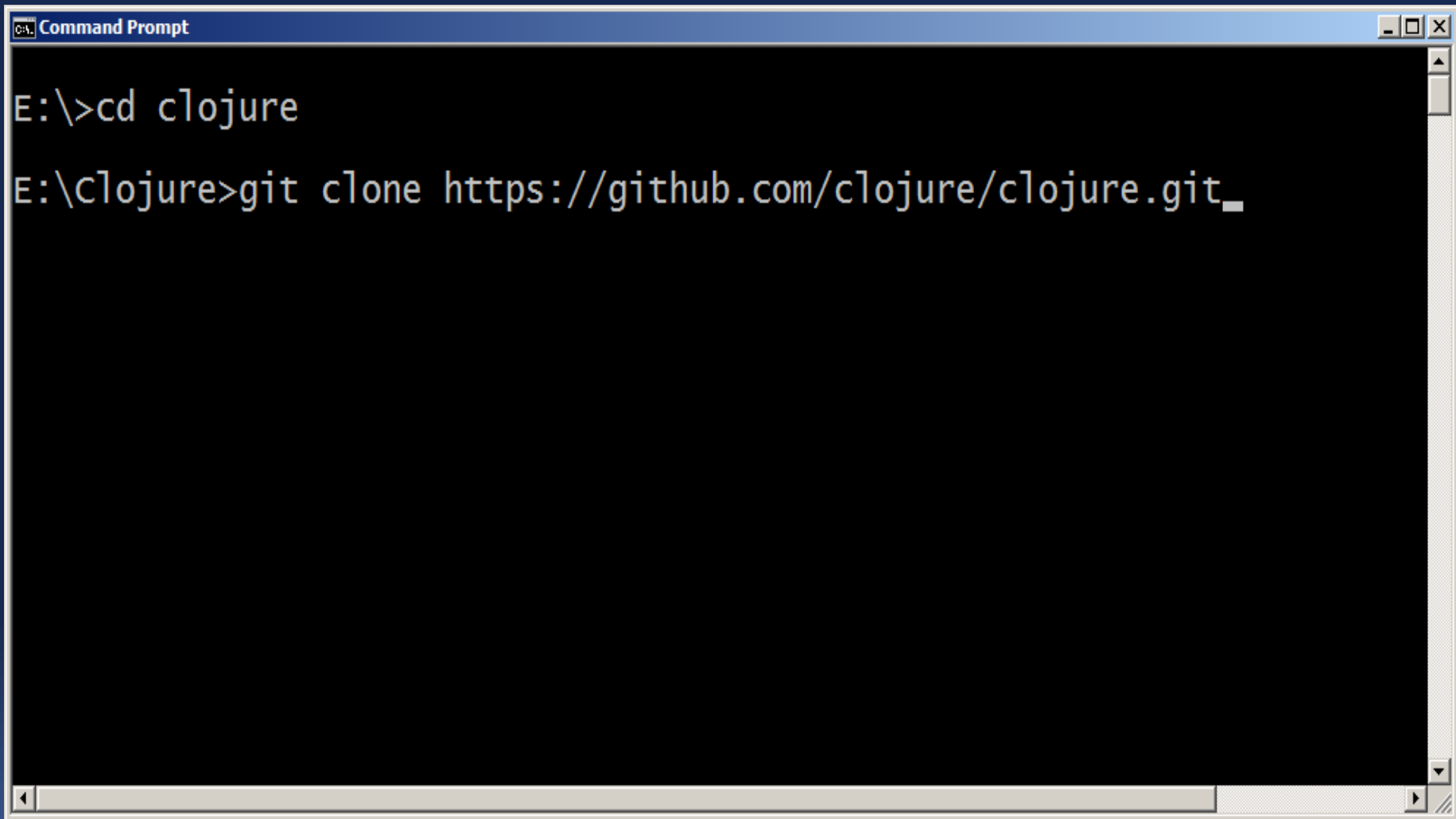


```
Command Prompt

E:\>cd clojure

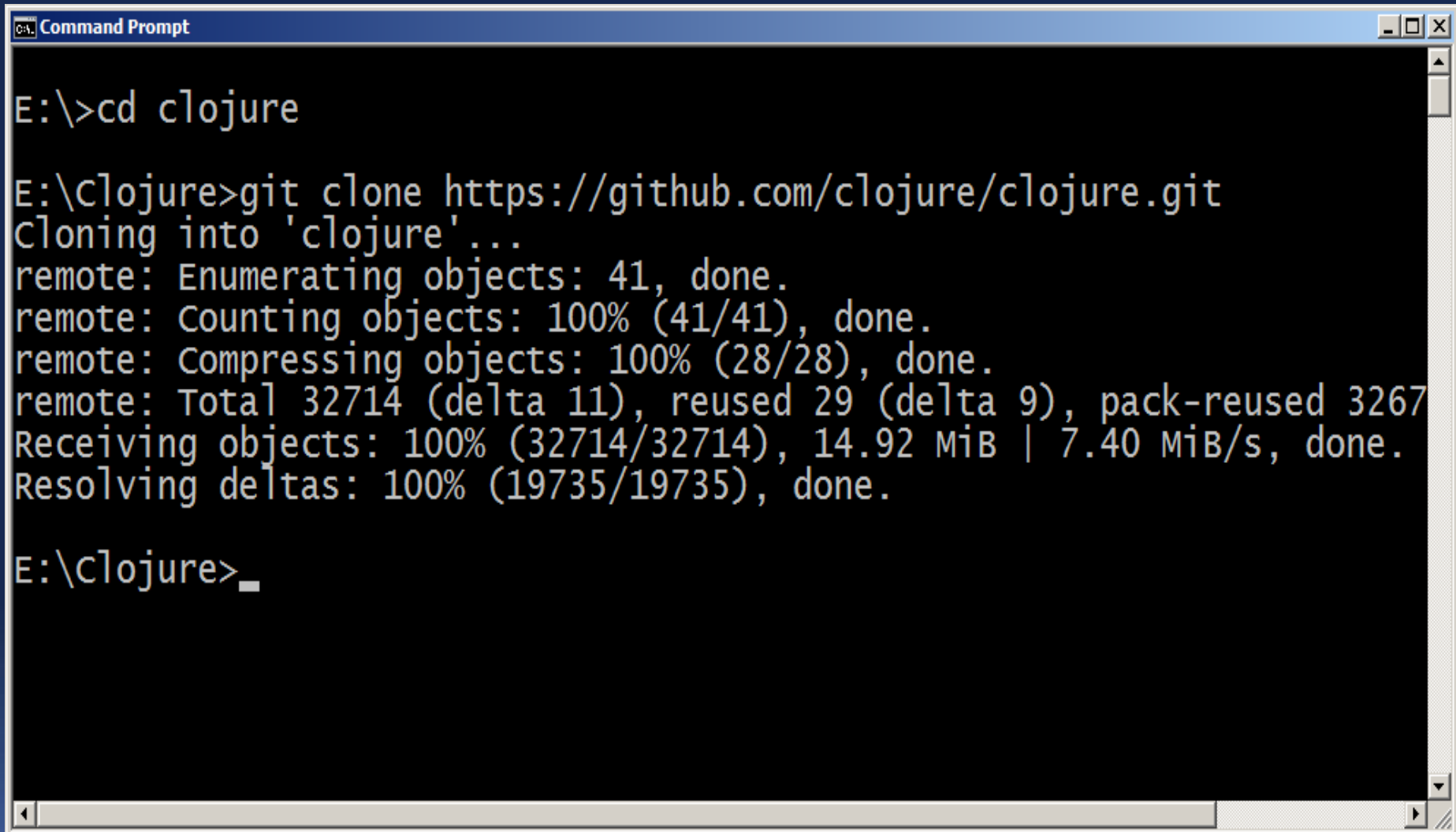
E:\clojure>_
```

Baixando clojure



```
Command Prompt
E:\>cd clojure
E:\Clojure>git clone https://github.com/clojure/clojure.git_
```

Baixando clojure



```
Command Prompt

E:\>cd clojure

E:\Clojure>git clone https://github.com/clojure/clojure.git
Cloning into 'clojure'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 32714 (delta 11), reused 29 (delta 9), pack-reused 3267
Receiving objects: 100% (32714/32714), 14.92 MiB | 7.40 MiB/s, done.
Resolving deltas: 100% (19735/19735), done.

E:\Clojure>
```

Pasta Clojure

new folder

Name ^	Date modified	Type	Size
 .git	20-May-20 2:00 PM	File folder	
 .idea	20-May-20 2:00 PM	File folder	
 doc	20-May-20 2:00 PM	File folder	
 src	20-May-20 2:00 PM	File folder	
 test	20-May-20 2:00 PM	File folder	
	20-May-20 2:00 PM	Text Document	1 KB
 antsetup	20-May-20 2:00 PM	Shell Script	1 KB
 build	20-May-20 2:00 PM	XML Document	9 KB
 changes	20-May-20 2:00 PM	Markdown Source File	115 KB
 clojure.iml	20-May-20 2:00 PM	IML File	2 KB
 CONTRIBUTING	20-May-20 2:00 PM	Markdown Source File	1 KB
 epl-v10	20-May-20 2:00 PM	Chrome HTML Docu...	13 KB
 pom	20-May-20 2:00 PM	XML Document	11 KB
 readme	20-May-20 2:00 PM	Text Document	14 KB

Building Clojure com Maven

mvn -Plocal -Dmaven.test.skip=true package

```
Command Prompt
Downloaded from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/1.2/jdependency-1.2.jar (22 kB at 20 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-analysis/6.0_BETA/asm-analysis-6.0_BETA.jar (21 kB at 16 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom/1.1.3/jdom-1.1.3.jar (151 kB at 117 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/aether/aether-util/0.9.0.M2/aether-util-0.9.0.M2.jar (134 kB at 100 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-util/6.0_BETA/asm-util-6.0_BETA.jar (47 kB at 35 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar (2.3 MB at 391 kB/s)
[INFO] Including org.clojure/spec.alpha:jar:0.2.187 in the shaded jar.
[INFO] Including org.clojure/core.specs.alpha:jar:0.2.44 in the shaded jar.
[INFO] Including org.clojure/test.check:jar:0.9.0 in the shaded jar.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:28 min
[INFO] Finished at: 2020-05-20T14:12:28-03:00
[INFO] -----
E:\clojure>
```

Iniciando REPL

java -jar clojure.jar

Command Prompt - java -jar clojure.jar

```
E:\clojure>java -jar clojure.jar  
Clojure 1.10.2-master-SNAPSHOT  
user=>
```

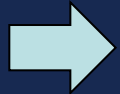
Testando REPL

java -jar clojure.jar



```
Command Prompt - java -jar clojure.jar

E:\clojure>java -jar  clojure.jar
Clojure 1.10.2-master-SNAPSHOT
user=> (+ 2 3 )
5
user=>
```

 REPL
Clojure 1.10.2-master-SNAPSHOT
user=> (System/exit 0)

Instalação Clojure no Windows

✓ PASSO 1 — INSTALAR O JAVA (OBRIGATÓRIO)

👉 Recomendado: Java 21 (LTS) [Clojure Ask](#)

◆ Verificar se já tem Java

⟨⟩ PowerShell

```
java -version
```

Se aparecer versão → ok

Se não → instale

◆ Instalar Java (recomendado)

Baixe:

👉 <https://adoptium.net> ↗

Escolha:

- Temurin 21
- Windows x64
- Installer (.msi)

◆ Após instalar

Configure variável de ambiente:

</> PowerShell

```
JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-21
```

E adicione ao PATH:

```
%JAVA_HOME%\bin
```



PASSO 2 — INSTALAR CLOJURE CLI (OFICIAL)

👉 O próprio site recomenda usar instalador no Windows

Baixe:

👉 <https://github.com/cassielc/clj-msi/releases> ↗

Baixe o arquivo:

```
clojure-<versao>.msi
```

Execute normalmente (Next → Next → Finish)

✓ Ele instala:

- `clj`
- `clojure`
- configura PATH automaticamente

Testando instalação

1. Ideia geral

Quando você instala o Clojure CLI no Windows, normalmente ficam disponíveis dois comandos:

- `clj`
- `clojure`

Os dois fazem parte da mesma ferramenta oficial, mas são usados de formas um pouco diferentes:

- `clj` : mais voltado ao uso **interativo**, abrindo o **REPL**
- `clojure` : mais usado para **executar arquivos**, **rodar expressões** e trabalhar com opções de linha de comando

Testando instalação

2. Primeiro teste

Abra o PowerShell e digite:

```
</> PowerShell  
  
clojure -Sdescribe
```

Se aparecer uma descrição da instalação, o ambiente está funcionando.

Você também pode testar:

```
</> PowerShell  
  
clj
```

Se abrir o prompt do Clojure, como:

```
</> clojure  
  
user=>
```

está tudo certo.

3. Usando o `clj`

3.1 Abrindo o REPL

Digite:

```
</> PowerShell
```

```
clj
```

Você verá algo assim:

```
</> clojure
```

```
user=>
```

Esse ambiente é o **REPL**, onde você digita expressões e recebe o resultado imediatamente.

```
PS D:\> cljs
Clojure 1.12.4
user=>
user=> (+ 2 3 )
5
user=> |
```

```
PS D:\> clj
Clojure 1.12.4
user=> (* 2 8)
16
user=>
```

```
user=>  
user=> (str "Olá, " "Mauá ! ")  
"Olá, Mauá ! "  
user=> |
```

3.2 Testando listas

```
</> clojure
```

```
(list 1 2 3)
```

Resultado:

```
</> clojure
```

```
(1 2 3)
```

```
</> clojure
```

```
(first '(10 20 30))
```

Resultado:

```
</> clojure
```

```
10
```

Saindo do REPL

`</>` PowerShell

`Ctrl + C`

ou feche a janela.

4. Usando o `clojure`

O comando `clojure` é muito útil para executar arquivos e expressões.

4.1 Executando um arquivo `.clj`

Crie um arquivo chamado `hello.clj` com este conteúdo:

```
</> clojure
```

```
(println "Olá, mundo em Clojure!")
```

No PowerShell, execute:

```
</> PowerShell
```

```
clojure hello.clj
```

Saída esperada:

```
Olá, mundo em Clojure!
```

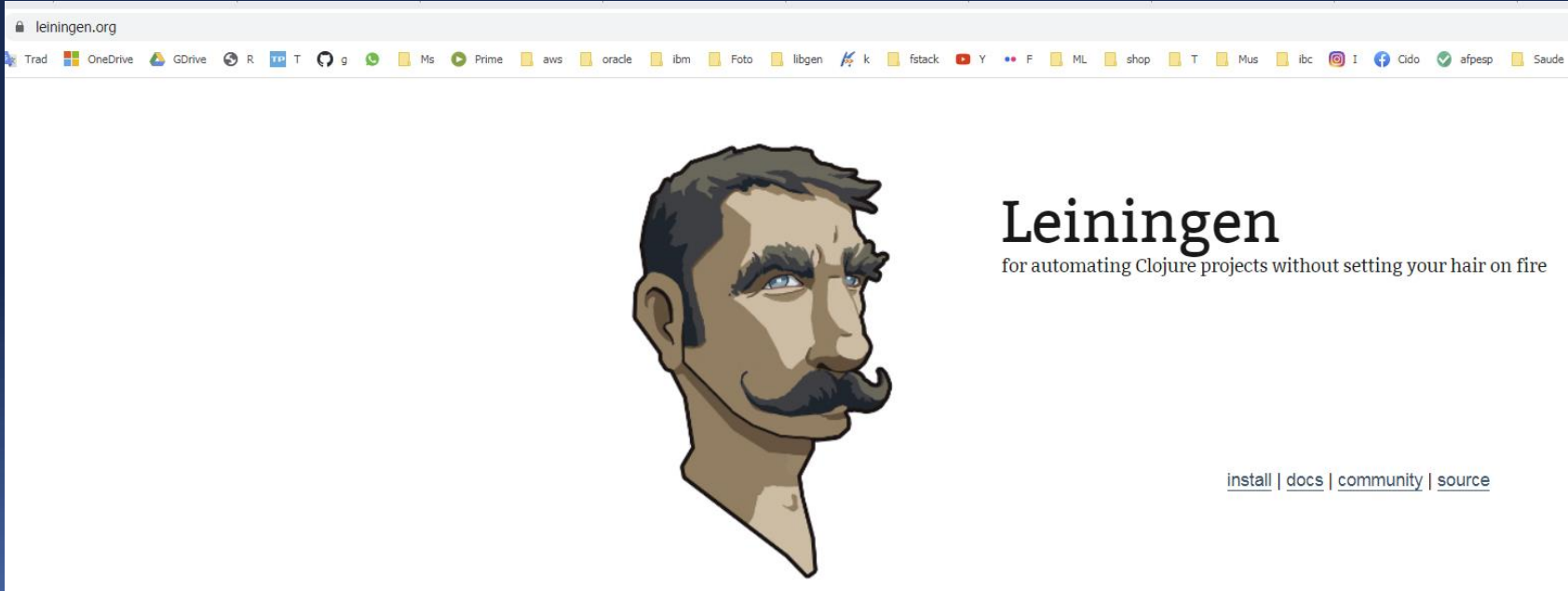


Leiningen

Leiningen – www.leiningen.org

👉 O Leiningen (`lein`) é uma ferramenta para:

- ✓ criar projetos
- ✓ gerenciar dependências
- ✓ compilar e executar código Clojure
- ✓ rodar testes



Leiningen



ANALOGIA (PERFEITA)

Linguagem

Ferramenta

Java

Maven / Gradle

Node

npm

Python

pip + poetry

Clojure

Leiningen

Leiningen



PARA QUE SERVE?



Ele facilita tudo isso:

- criar projeto automaticamente
- baixar bibliotecas
- rodar o programa
- abrir REPL dentro do projeto



COMO INSTALAR (Windows)

1. Baixar



<https://leiningen.org> ↗

Baixe o arquivo:

```
lein.bat
```

2. Colocar no PATH

Exemplo:

```
C:\lein\lein.bat
```

Adicionar ao PATH:

```
C:\lein
```

3. Testar

⟨/⟩ PowerShell

```
lein
```

👉 Na primeira execução ele baixa tudo automaticamente.



PRIMEIRO PROJETO

1. Criar projeto

⌘ PowerShell

```
lein new app meu-projeto
```



Isso cria uma estrutura pronta

2. Entrar na pasta

PowerShell

```
cd meu-projeto
```

3. Estrutura criada

Você verá algo assim:

```
meu-projeto/  
├─ project.clj  
├─ src/  
└─ test/
```

👉 O arquivo principal é:

```
project.clj
```

O QUE É `project.clj`?

 É o “coração” do projeto

Exemplo:

```
</> clojure
```

```
(defproject meu-projeto "0.1.0-SNAPSHOT"  
  :dependencies [[org.clojure/clojure "1.12.0"]])
```



ABRIR REPL NO PROJETO

⟨⟩ PowerShell

```
lein repl
```

👉 Agora você está dentro do projeto

Exemplo:

⟨⟩ clojure

```
(+ 2 3)
```



ADICIONAR BIBLIOTECA

Abra `project.clj`:

```
</> clojure
```

```
:dependencies [  
  [org.clojure/clojure "1.12.0"]  
  [clj-time "0.15.2"]  
]
```

Depois:

```
</> PowerShell
```

```
lein deps
```



RODAR TESTES

⟨/⟩ PowerShell

```
lein test
```



COMANDOS PRINCIPAIS

Comando	Função
<code>lein new app nome</code>	cria projeto
<code>lein run</code>	executa
<code>lein repl</code>	abre REPL
<code>lein test</code>	roda testes
<code>lein deps</code>	baixa dependências

LEININGEN É USADO HOJE?

 SIM, MAS...

✓ Ainda é usado

✓ Muitos projetos antigos usam

! Porém:

 Não é mais o padrão oficial



SITUAÇÃO NO MERCADO

Situação

Uso

Projetos antigos

🔥 muito usado

Projetos novos

⚠️ menos comum

Mercado real

✓ ainda aparece



FERRAMENTA MODERNA

Hoje o padrão é:

👉 Clojure CLI (`c1j` + `deps.edn`)



DEVO APRENDER LEININGEN?



SIM (IMPORTANTE)

👉 Você deve conhecer porque:

- muitos projetos usam
- aparece em entrevistas
- ainda está no mercado



Clojure CLI (`clj` + `deps.edn`)



VISÃO GERAL

Com o Clojure moderno você usa:

- `clj` → REPL
- `clojure` → execução
- `deps.edn` → dependências e configuração



1. ESTRUTURA DE UM PROJETO

Um projeto típico:

```
meu-projeto/  
├── deps.edn  
├── src/  
│   └── core.clj  
└── test/
```



2. CRIANDO UM PROJETO DO ZERO

◆ Passo 1: criar pasta

</> PowerShell

```
mkdir meu-projeto
```

```
cd meu-projeto
```

◆ Passo 2: criar `deps.edn`

⟨/⟩ PowerShell

```
notepad deps.edn
```

Conteúdo:

⟨/⟩ clojure

```
{:deps {org.clojure/clojure {:mvn/version "1.12.0"}}
```

👉 Esse arquivo diz:

- qual versão do Clojure usar
- quais bibliotecas baixar

3. CRIANDO O CÓDIGO

◆ Criar pasta src

</> PowerShell

```
mkdir src
```

◆ Criar arquivo principal

⟨/⟩ PowerShell

```
notepad src\core.clj
```

Conteúdo:

⟨/⟩ clojure

```
(ns core)
```

```
(println "Olá Clojure com CLI!")
```



4. EXECUTANDO O PROJETO

⟨/⟩ PowerShell

```
clojure src/core.clj
```

Saída:

```
Olá Clojure com CLI!
```



5. USANDO O REPL COM O PROJETO

```
</> PowerShell
```

```
clj
```

Agora dentro do REPL:

```
</> clojure
```

```
(require 'core)
```



IMPORTANTE: NAMESPACE



O arquivo:

```
src/core.clj
```

deve ter:

```
</> clojure
```

```
(ns core)
```



Nome do arquivo = namespace

Exemplo Hello World – clj + deps.edn

1. O que vamos criar

Vamos montar um projeto simples com esta estrutura:

```
hello-clojure/  
├── deps.edn  
└── src/  
    ├── hello/  
    │   └── core.clj
```

Esse projeto terá um programa que imprime:

```
Hello World
```

2. Entrar na pasta onde você quer criar o projeto

No PowerShell, vá para uma pasta de trabalho. Exemplo:

</> PowerShell

```
cd D:\  
mkdir clojure-projetos  
cd clojure-projetos
```

Agora crie a pasta do projeto:

</> PowerShell

```
mkdir hello-clojure  
cd hello-clojure
```

3. Criar o arquivo `deps.edn`

Esse arquivo diz ao Clojure quais dependências e configurações o projeto usa.

No PowerShell:

```
</> PowerShell
```

```
notepad deps.edn
```

Cole este conteúdo:

```
</> clojure
```

```
{:paths ["src"]  
 :deps {org.clojure/clojure {:mvn/version "1.12.0"}}}
```

Salve e feche.

```
{:paths ["src"]  
 :deps {org.clojure/clojure {:mvn/version "1.12.0"}}}
```

O que isso significa

```
:paths ["src"]
```

Diz que o código-fonte do projeto está dentro da pasta `src`.

```
:deps
```

Lista as dependências do projeto.

```
org.clojure/clojure
```

É a linguagem Clojure em si.

```
{:mvn/version "1.12.0"}
```

Informa a versão usada.

4. Criar a pasta do código-fonte

Agora vamos criar as pastas:

</> PowerShell

```
mkdir src
```

```
mkdir src\hello
```

5. Criar o arquivo principal do programa

No PowerShell:

⟨/⟩ PowerShell

```
notepad src\hello\core.clj
```

Cole este conteúdo:

⟨/⟩ clojure

```
(ns hello.core)
```

```
(println "Hello World")
```

Salve e feche.

```
(ns hello.core)  
  
(println "Hello World")
```

6. Entendendo o código

Linha 1

```
</> clojure  
  
(ns hello.core)
```

Essa linha define o **namespace** do arquivo.

Em Clojure, o namespace funciona como o nome lógico do módulo.

```
(ns hello.core)  
  
(println "Hello World")
```

Linha 2

⌕ clojure

```
(println "Hello World")
```

Esse comando imprime texto no terminal.

7. Executar o arquivo diretamente

Ainda dentro da pasta do projeto, execute:

⟨/⟩ PowerShell

```
clojure src\hello\core.clj
```

Saída esperada:

```
Hello World
```



<https://tryclojure.org/>



Got 5 minutes?

If you want everything to be familiar, you'll never learn anything new. - Rich Hickey

Let's have some fun! Here on the right you have a **REPL**, a command line with a *read-eval-print* loop. Everything you write will be evaluated. Try to type some expressions as `(+ 1 2)` or click on code to auto insert. You can type `(help)` for more commands.

Type `(start)` when you're ready!

Type your Clojure symbolic expressions here

=> |

<https://tryclojure.org/>

Type your Clojure symbolic expressions here

```
=> (println "Hello")
```

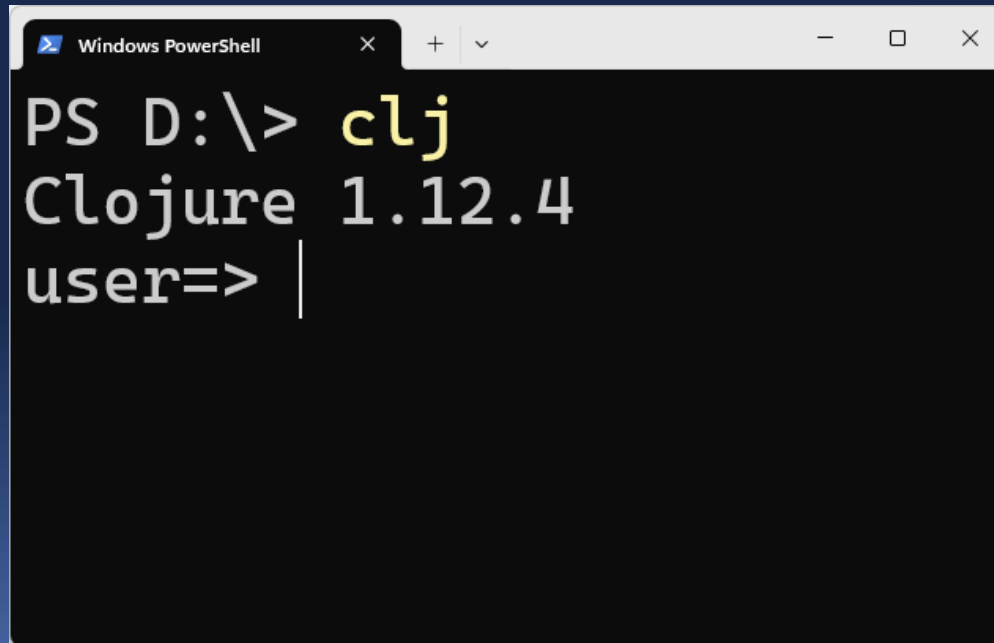
```
Hello
```

```
nil
```

```
=> |
```

Usando REPL

- ✓ REPL significa **READ EVAL PRINT LOOP** e representa um bom recurso para iniciar o estudo de **Clojure**;
- ✓ É uma **interface de comandos** que permite a avaliação direta de código Clojure;
- ✓ No prompt do REPL, a primeira linha representa a **versão** do Clojure, o qual em nosso caso é a **1.12.4**;



```
Windows PowerShell
PS D:\> clj
Clojure 1.12.4
user=> |
```

O QUE É REPL?

REPL significa:

Read → ler

Eval → executar

Print → mostrar resultado

Loop → repetir

 Em português:

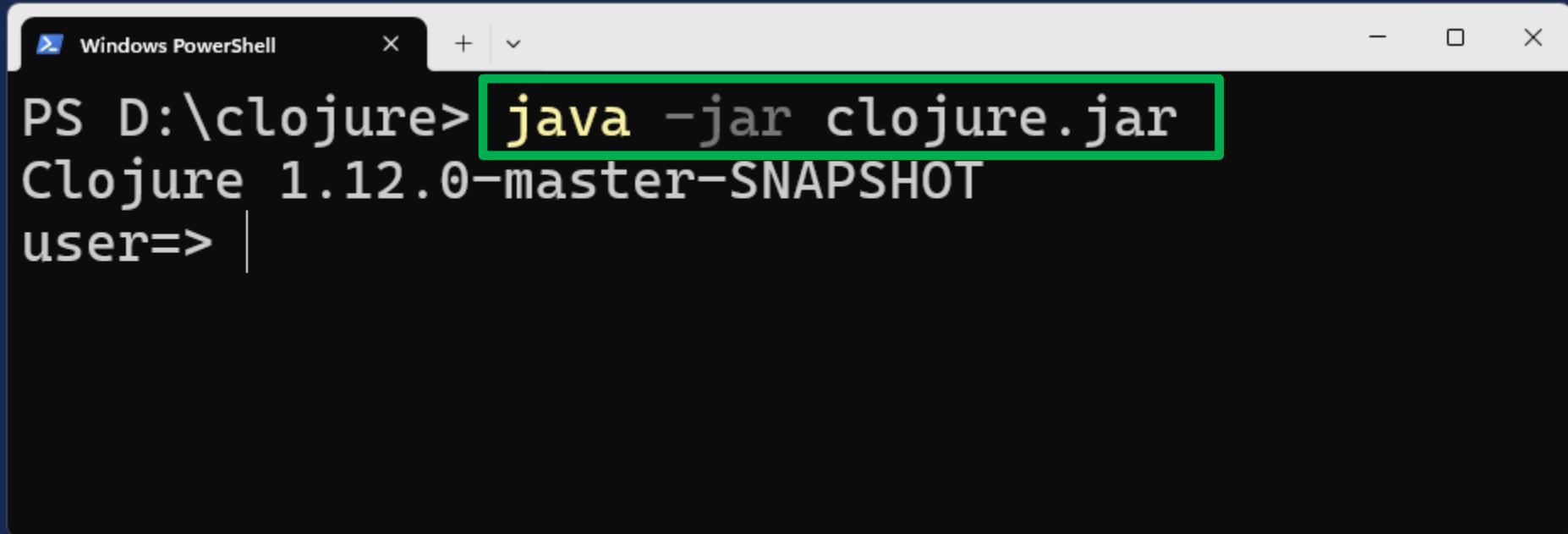
“um ambiente onde você digita código e vê o resultado na hora”

```
Windows PowerShell
PS D:\clojure> dir

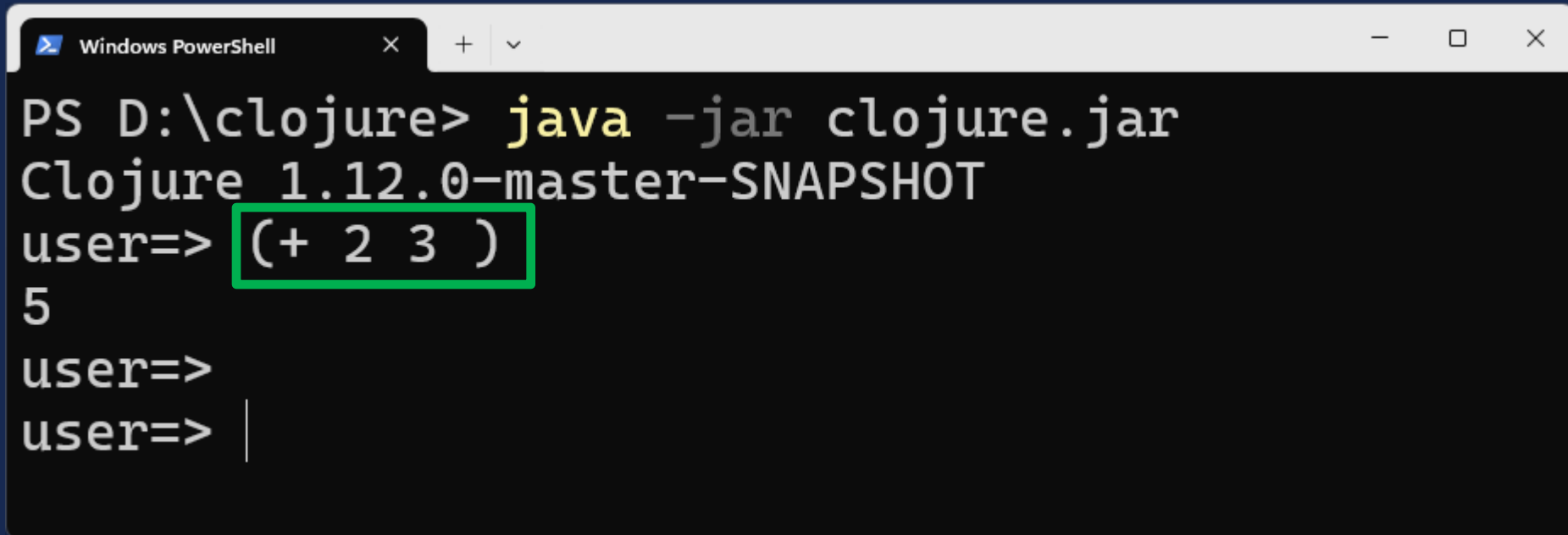
Diretório: D:\clojure

Mode                LastWriteTime         Length Name
----                -
-a-----         15/03/2026   21:36      4851348 clojure.jar

PS D:\clojure> |
```



```
Windows PowerShell
PS D:\clojure> java -jar clojure.jar
Clojure 1.12.0-master-SNAPSHOT
user=> |
```



```
Windows PowerShell
PS D:\clojure> java -jar clojure.jar
Clojure 1.12.0-master-SNAPSHOT
user=> (+ 2 3 )
5
user=>
user=> |
```

Outro exemplo:

```
</> clojure
```

```
(* 4 5)
```

Resultado:

```
</> clojure
```

```
20
```

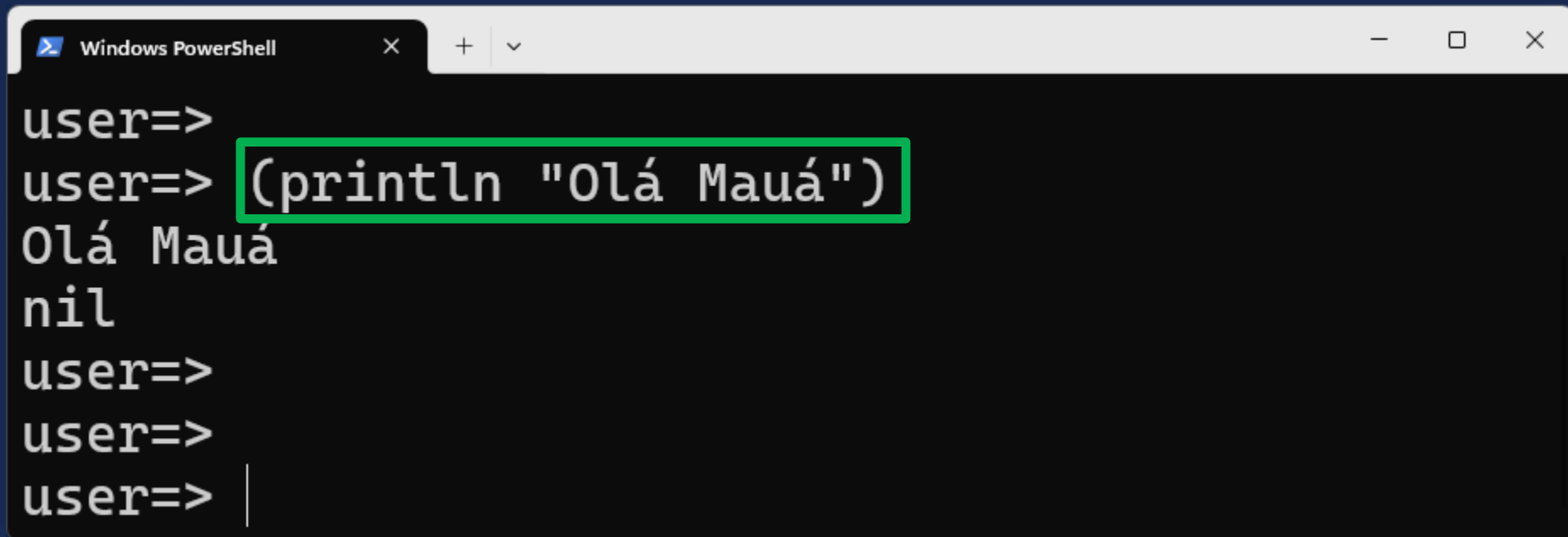


IDEIA PRINCIPAL



No REPL você:

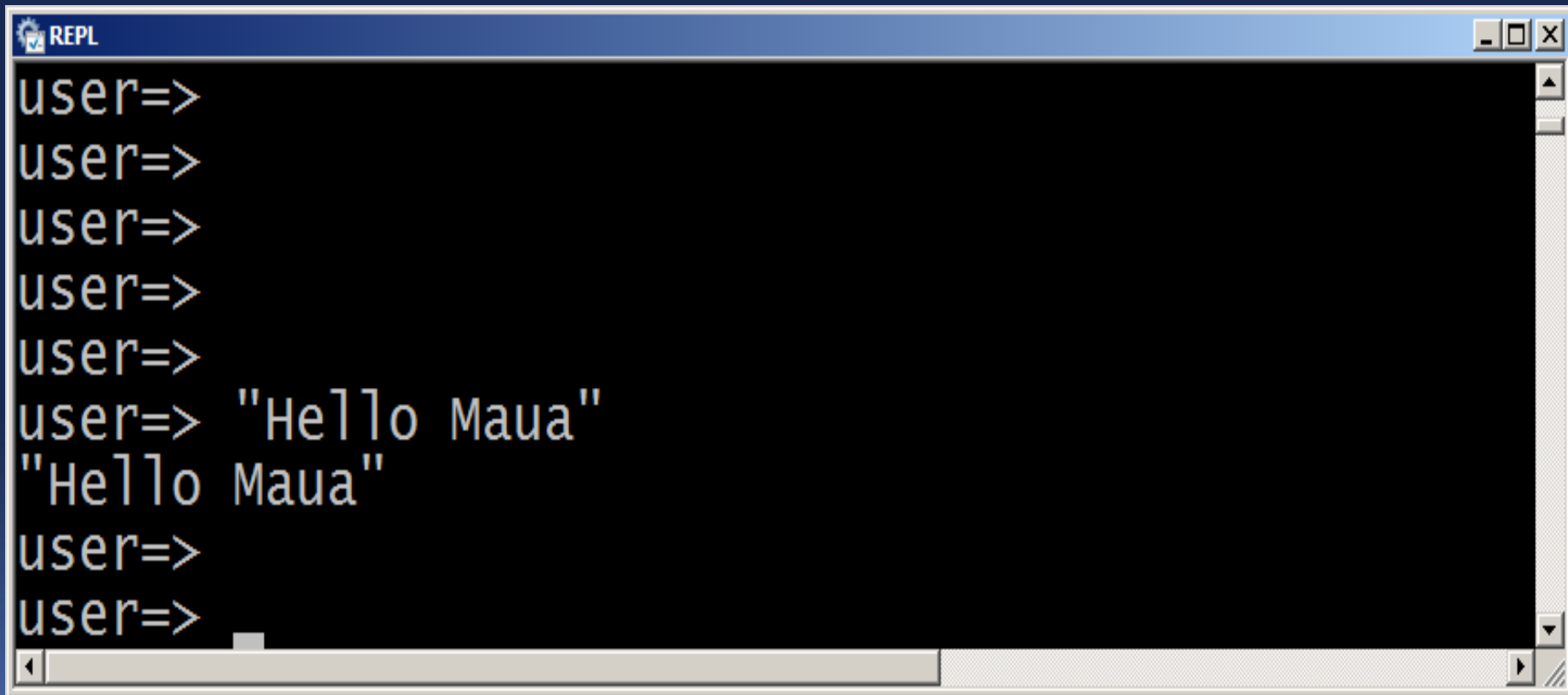
- escreve código
- executa imediatamente
- vê o resultado na hora



```
Windows PowerShell
user=>
user=> (println "Olá Mauá")
Olá Mauá
nil
user=>
user=>
user=>
```

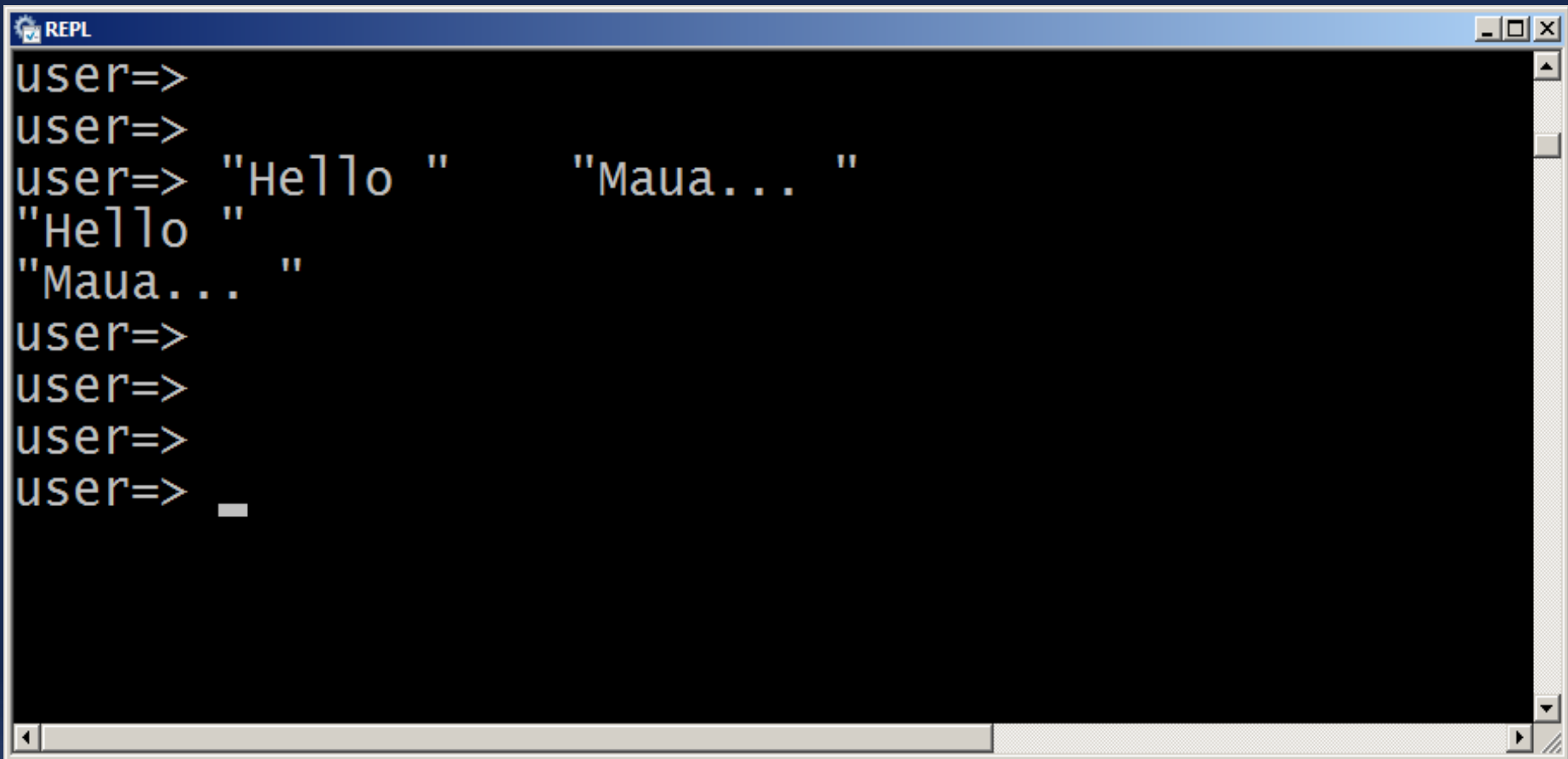
Avaliando expressões

- Em **Clojure**, **literais string** são criados com aspas duplas, “ ”;
- Um **literal** é uma notação para representar valores **fixos** no código fonte.



```
REPL
user=>
user=>
user=>
user=>
user=>
user=> "Hello Maua"
"Hello Maua"
user=>
user=>
```

Avaliando múltiplos strings



```
REPL
user=>
user=>
user=> "Hello "      "Maua... "
"Hello "
"Maua... "
user=>
user=>
user=>
user=> _
```

- ✓ Nesse exemplo, duas expressões string foram avaliadas sequencialmente, e cada qual foi retornada em duas linhas separadas.

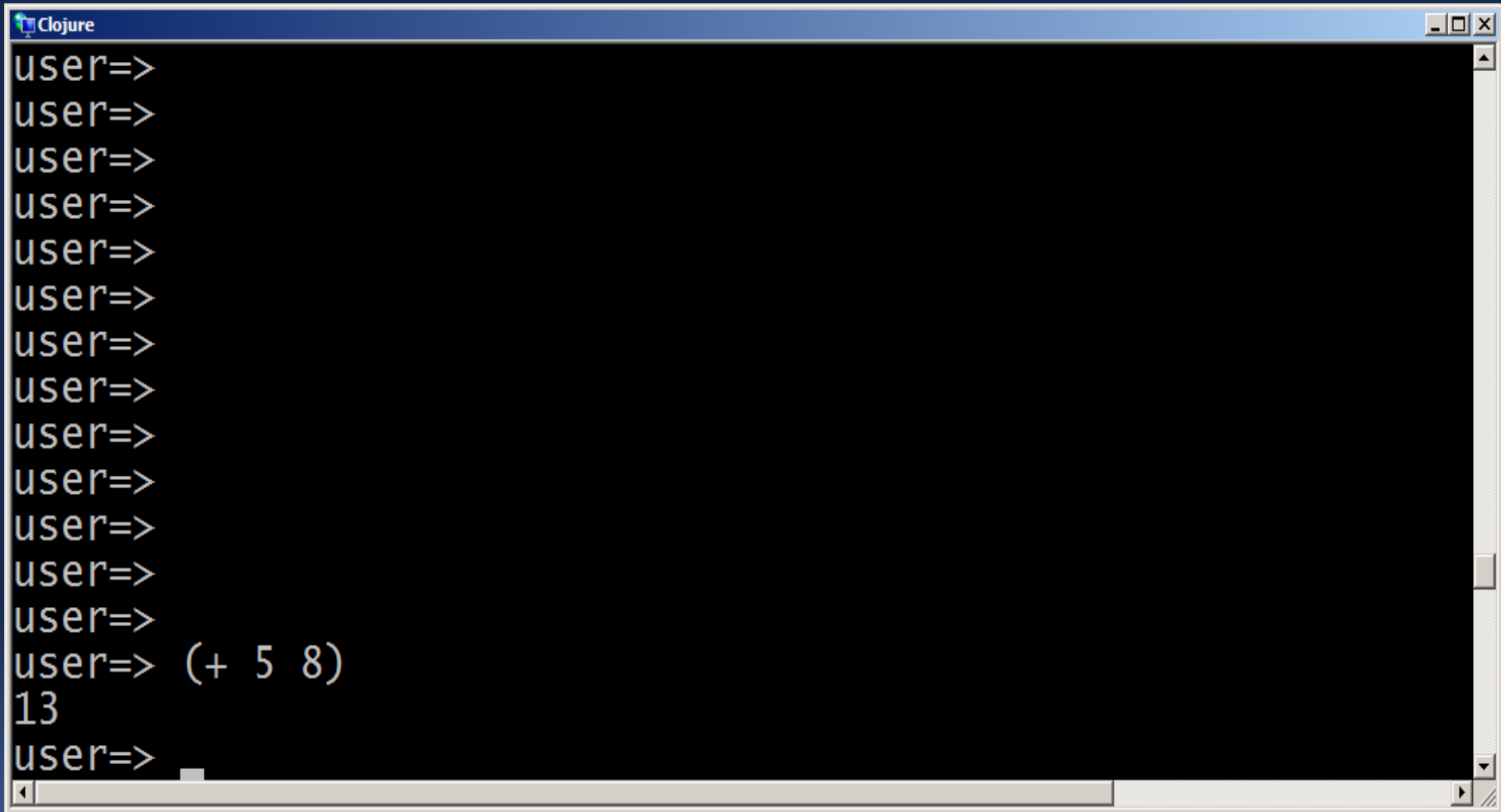
Avaliando expressões aritméticas

```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> 5 + 8
5
#object[clojure.core$_PLUS_ 0x294a6b8e "clojure.core$_PLUS_@2
8
user=>
```



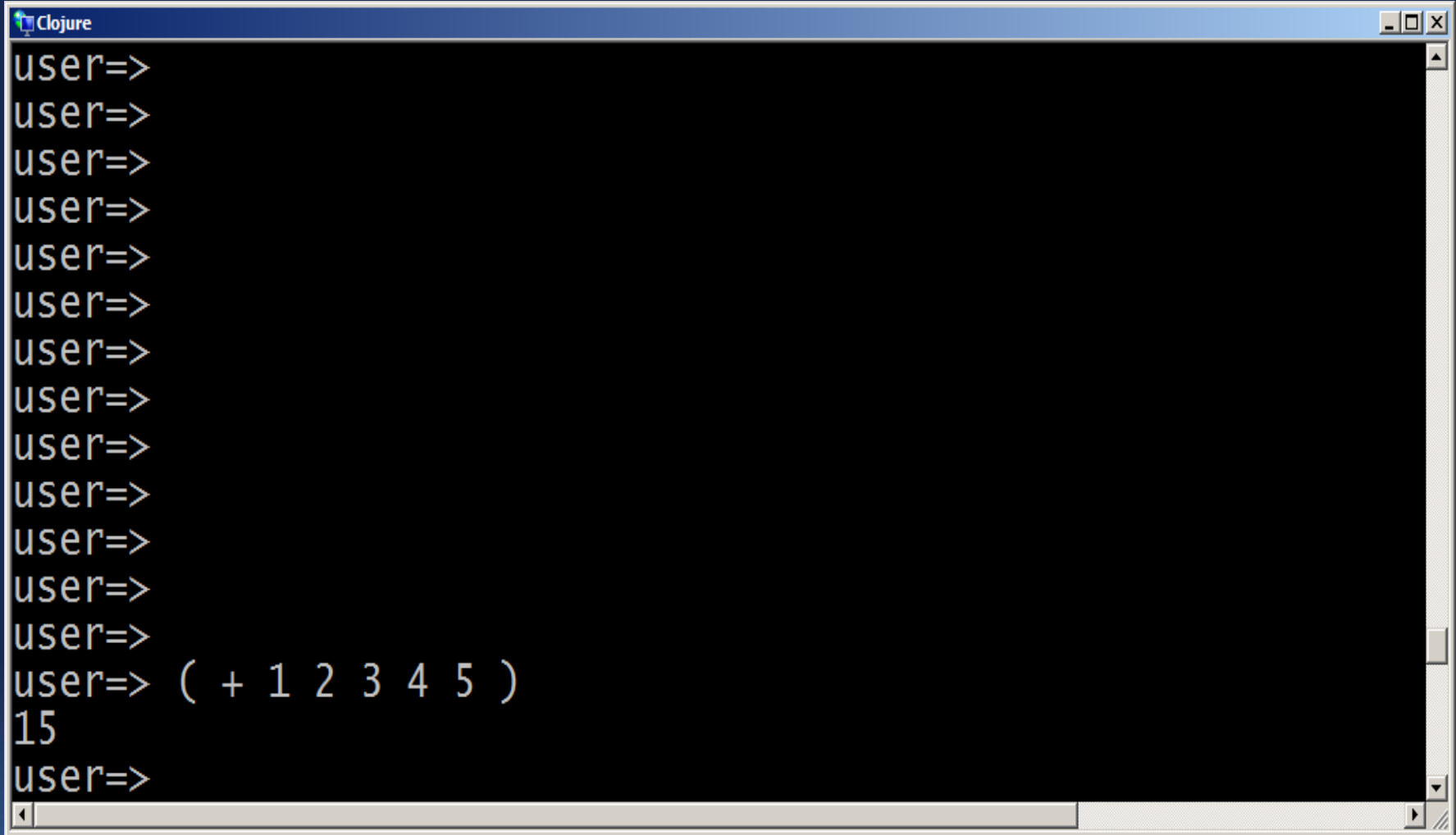
Clojure retornou **erro**, pois o primeiro argumento deve ser uma **função**. No caso, o símbolo **+** está ligado (**bounded**) à uma função e deve ser o primeiro elemento da lista a ser avaliada, seguido pelos operandos (**argumentos**).

Avaliando expressões aritméticas



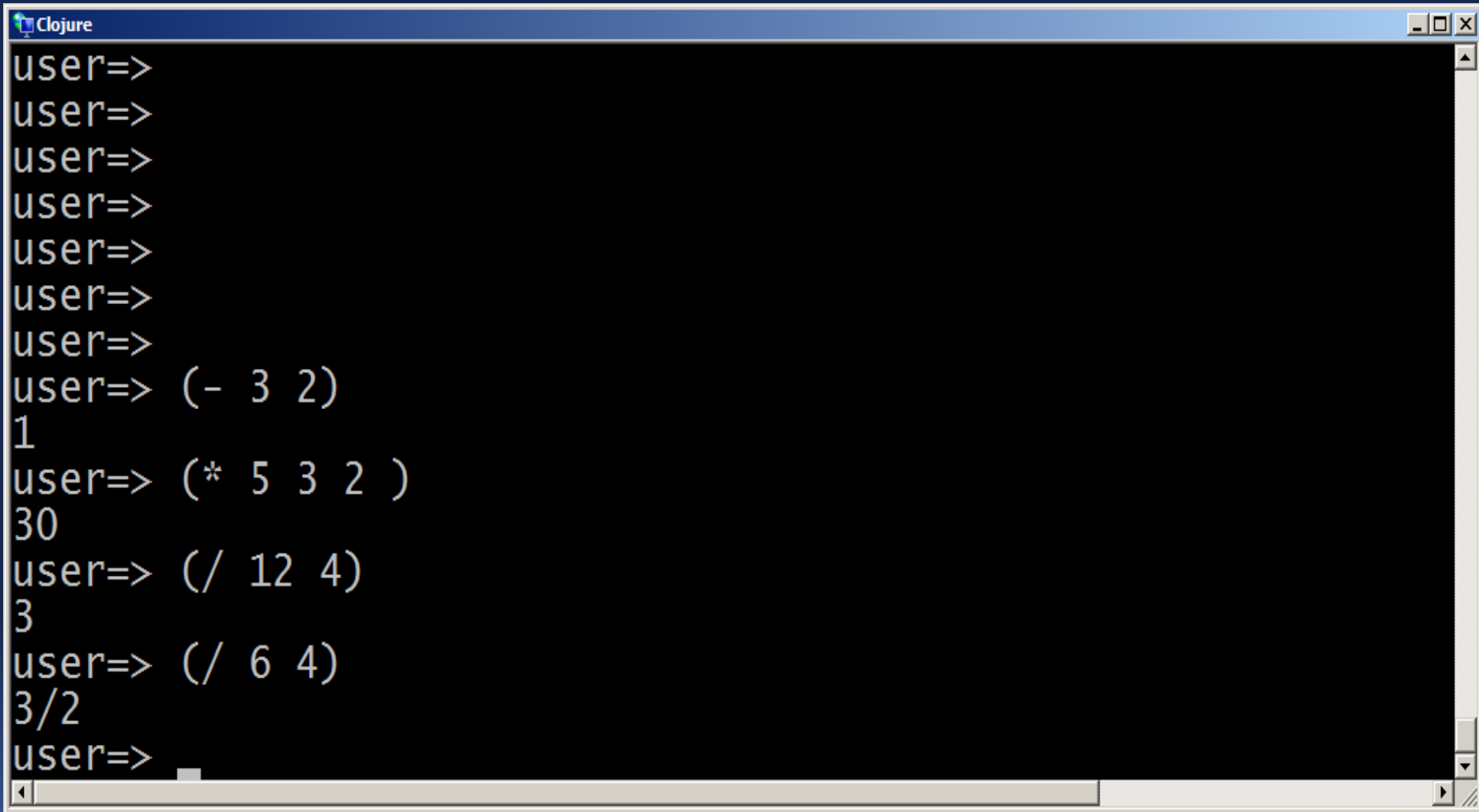
```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (+ 5 8)
13
user=>
```

Avaliando expressões aritméticas



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> ( + 1 2 3 4 5 )
15
user=>
```

Avaliando expressões aritméticas



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (- 3 2)
1
user=> (* 5 3 2 )
30
user=> (/ 12 4)
3
user=> (/ 6 4)
3/2
user=>
```

Função println



```
REPL
user=>
user=>
user=>
user=>
user=>
user=> (println "Hello Maua")
Hello Maua
nil
user=>
user=>
```

- ✓ O texto impresso pela função **println** foi um **SIDE EFFECT**;
- ✓ A função na verdade **retornou nil**.

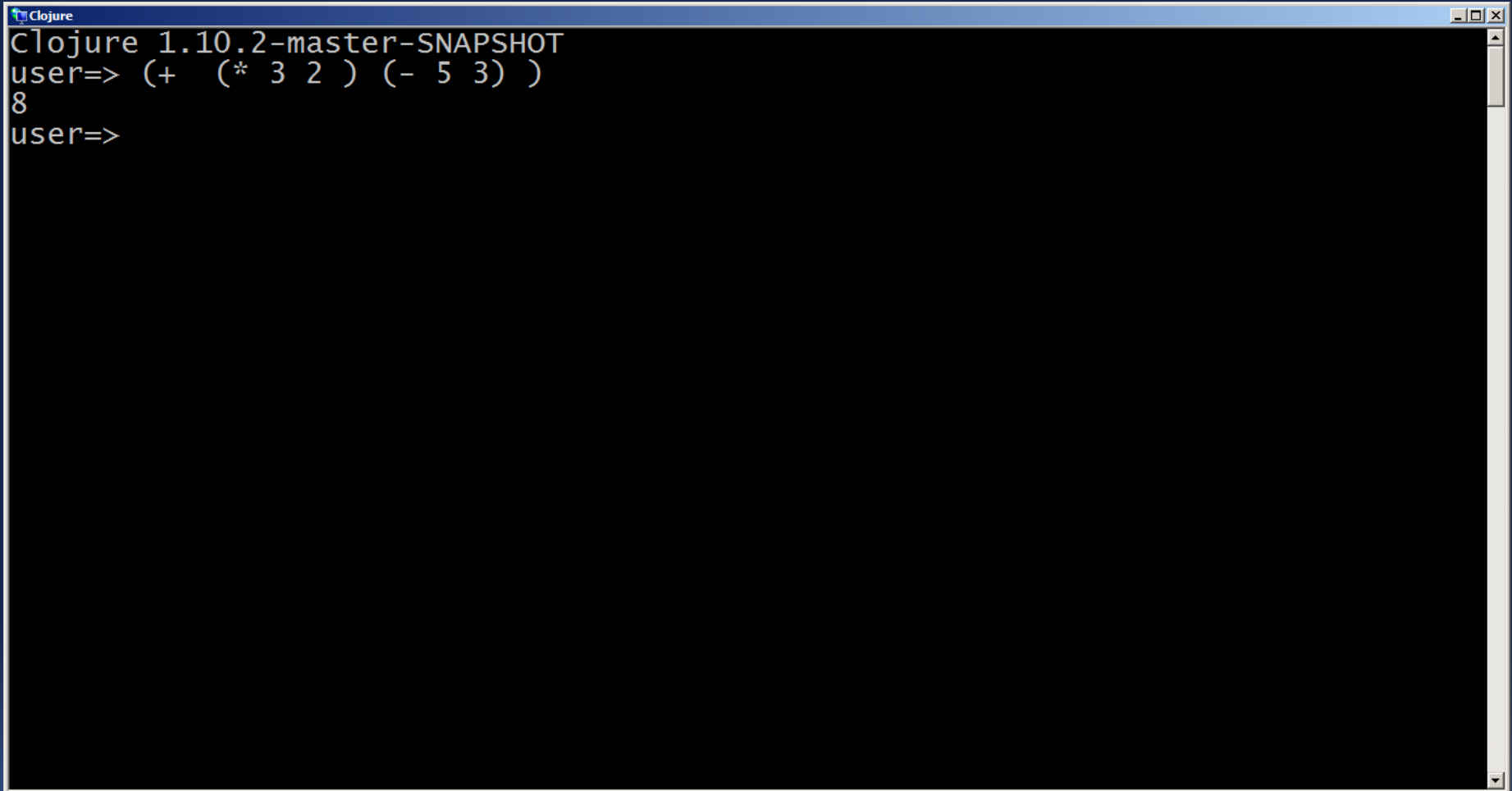
nil

- ✓ Equivale em **Clojure** a um valor “**null**” ou “**nada**”;
- ✓ Ou seja, trata-se da **ausência** de **significado**;
- ✓ As funções **print** e **println** são usadas para imprimir objetos para a saída padrão e retornam **nil** uma vez que a impressão foi efetivada;



```
REPL
user=>
user=>
user=>
user=>
user=>
user=> (print "Hello Maua")
Hello Maua nil
user=>
```

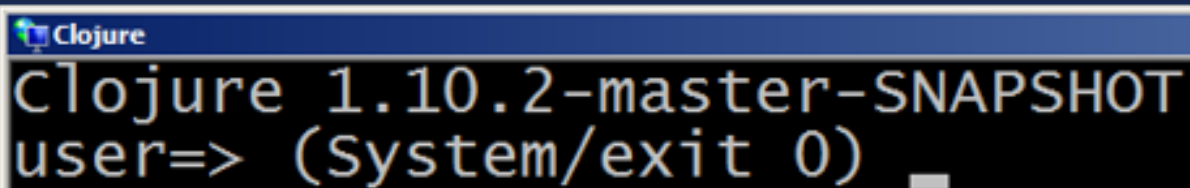
Funções nested



```
Clojure 1.10.2-master-SNAPSHOT
user=> (+ (* 3 2) (- 5 3))
8
user=>
```

Exit REPL

(System/exit 0)



A screenshot of a Clojure REPL window. The title bar is blue and contains the text 'Clojure' next to a small icon. The main area is black with white text. It shows 'Clojure 1.10.2-master-SNAPSHOT' on the first line and 'user=> (System/exit 0) _' on the second line, with a white cursor at the end of the second line.

```
Clojure 1.10.2-master-SNAPSHOT
user=> (System/exit 0) _
```

Operando com a última expressão avaliada => *1

```
Clojure
Clojure 1.10.2-master-SNAPSHOT
user=> (inc 10)
11
user=> (inc *1)
12
user=> (inc *1)
13
user=> *1
13
user=>
```

Operando com a variável que contém o resultado da última

exceção => ***e**

```
user=>
user=>
user=> (/ 1 0)
Execution error (ArithmeticException) at user/eval7 (REPL:1).
Divide by zero
user=>
user=>
user=>
user=>
user=> *e
#error {
  :cause "Divide by zero"
  :via
  [{:type java.lang.ArithmeticException
    :message "Divide by zero"
    :at [clojure.lang.Numbers divide "Numbers.java" 188]}]
```

Concatenando Strings – str

- ✓ A função **str** sem parâmetros retorna o String nulo ("")
- ✓ Com um argumento **x** retorna **x.toString()**;
- ✓ Com **mais de um argumento** **x . . . y** retorna a **concatenação** desses argumentos !

```
Clojure
user=>
user=>
user=> (str "Hello" " USCS...." " Tudo bem ? ")
"Hello USCS.... Tudo bem ? "
user=>
user=>
user=>
user=>
user=> (str "Hello")
"Hello"
user=>
user=>
user=> (str)
""
user=>
```

Função doc

- ✓ A função **doc** permite que se pesquise a **documentação** através de **REPL**;
- ✓ Dado que se conhece o nome da função que se quer usar, pode-se ler a documentação desta função por meio da função **doc**.
- ✓ Exemplo: Obtenção da documentação da função **str**:

```
user=> (doc str)
-----
clojure.core/str
([] [x] [x & ys])
  With no args, returns the empty string. With one arg x, returns
  x.toString(). (str nil) returns the empty string. With more than
  one arg, returns the concatenation of the str values of the args.
nil
user=>
```

Função doc

- ✓ Exibe o nome completamente qualificado da função (incluindo o **namespace**) na primeira linha, os possíveis **parâmetros** (ou “aridades”) na próxima linha e finalmente a descrição da função que se está pesquisando;
- ✓ Exemplo: **(doc mod)**

```
user=>  
user=> (doc mod)  
-----  
clojure.core/mod  
([num div])  
  Modulus of num and div. Truncates toward negative infinity.  
nil  
user=>
```

clojure.core

- ✓ Tudo o que está definido em **clojure.core** estará disponível ao seu **namespace** corrente por **default**, dispensando-se assim de se explicitamente requerê-la;
- ✓ Por exemplo: **(str "Hello")** ao invés de **(clojure.core/str "Hello")**

```
user=>  
user=> (clojure.core/str "Hello")  
"Hello"  
user=>
```

(find-doc)

- ✓ Quando **não** se sabe o nome da função, mas se tem uma ideia de sua descrição ou nome que pode conter, pode-se usar a função **find-doc**;

```
user=>
user=> (find-doc "float")
-----
clojure.pprint/float-parts
([f])
  Take care of leading and trailing zeros in decomposed floats
-----
clojure.pprint/float-parts-base
([f])
  Produce string parts for the mantissa (normalized 1-9) and exponent
-----
clojure.core/aset-float
([array idx val] [array idx idx2 & idxv])
  Sets the value at the index/indices. Works on arrays of float. Returns val.
-----
clojure.core/float
([x])
  Coerce to float
```

lower-case e upper-case

- ✓ Observamos na saída que **lower-case** e **upper-case** estão definidas no namespace **clojure.string**.



```
user=>  
user=>  
user=> (clojure.string/upper-case "HELllo")  
"HELlLLO"  
user=>  
user=>  
user=> (clojure.string/lower-case "HELllo")  
"hellllo"  
user=>
```



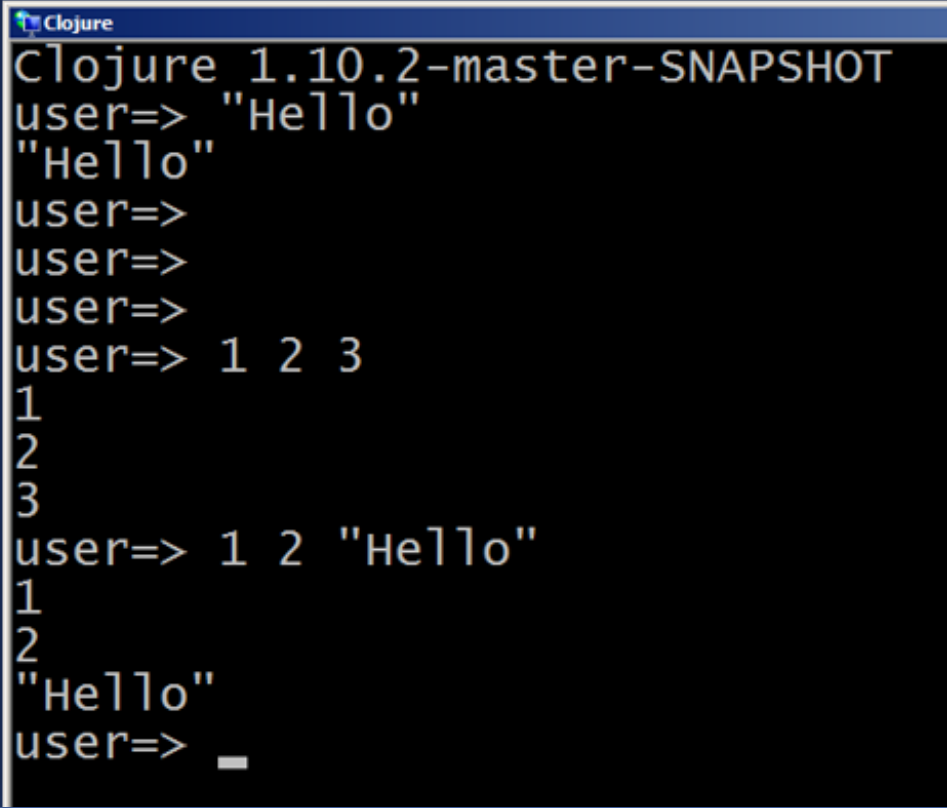
Tarefa 1



- ✓ Abra o **REPL**;
- ✓ Imprima a mensagem: “**Eu não tenho medo de parênteses**”
- ✓ Some 1, 2 e 3 e multiplique o resultado por 10 menos 3;
- ✓ Imprima a mensagem “**Tarefa Concluída...**”;
- ✓ Encerre o **REPL**.

Avaliação de código em Clojure

- ✓ Em **Clojure**, literais são expressões válidas e ao serem avaliadas retornam os próprios literais.



The screenshot shows a Clojure REPL window titled "Clojure" with the version "1.10.2-master-SNAPSHOT". The prompt "user=>" is followed by several inputs and their corresponding outputs. Three large blue arrows point to the first three input-output pairs. The first pair shows the input "Hello" and the output "Hello". The second pair shows the input "user=>" and the output "user=>". The third pair shows the input "user=>" and the output "user=>". The fourth pair shows the input "1 2 3" and the output "1 2 3". The fifth pair shows the input "1 2" and the output "1 2". The sixth pair shows the input "1 2 \"Hello\"" and the output "1 2 \"Hello\"". The seventh pair shows the input "user=>" and the output "user=>".

```
Clojure 1.10.2-master-SNAPSHOT
user=> "Hello"
"Hello"
user=>
user=>
user=>
user=> 1 2 3
1
2
3
user=> 1 2 "Hello"
1
2
"Hello"
user=> _
```

Comentários em Clojure

✓ Qualquer linha iniciada com ;

```
user=>  
user=>  
user=> ; Isto é um comentário em Clojure!!!  
user=>
```

Funções em Clojure

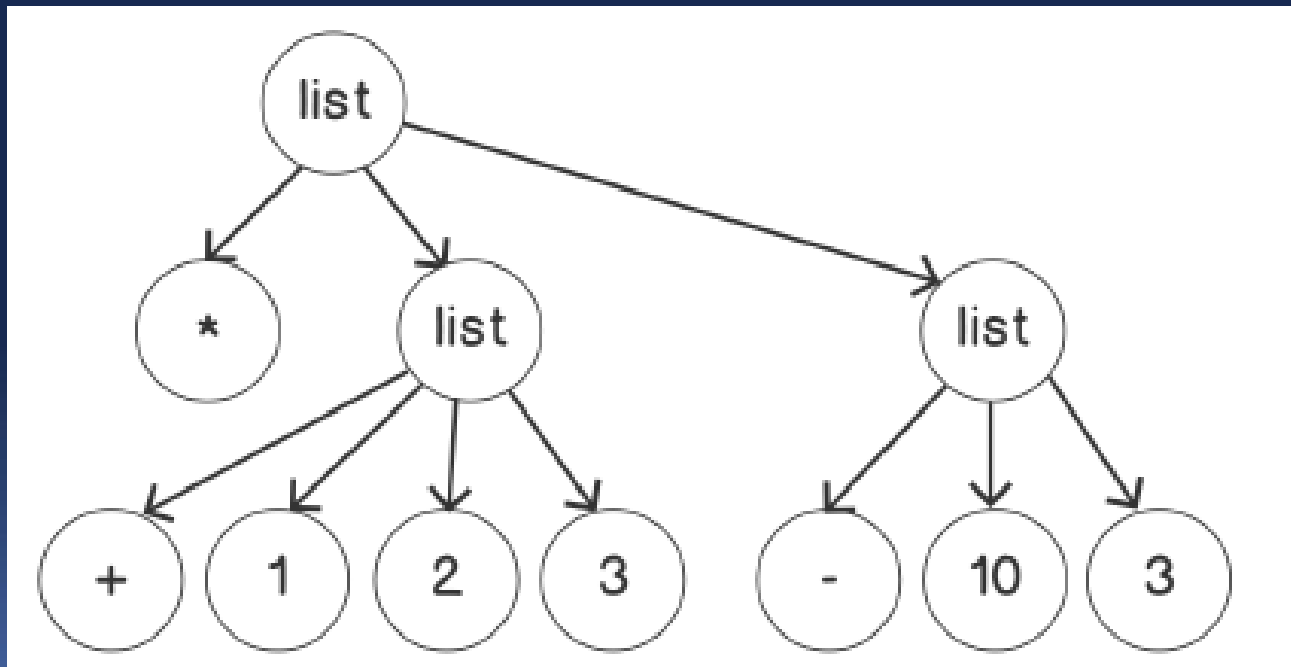
✓ São invocadas com a seguinte estrutura:

```
; (operator operand-1 operand-2 operand-3 ...)  
; for example:  
user=> (* 2 3 4)  
24
```

Expressões Simbólicas (s-expression)

✓ Podem ser visualizadas em uma árvore

```
(* (+ 1 2 3) (- 10 3))
```



Listas

- ✓ São usadas para representar funções;
- ✓ Ao usarmos listas para representar **dados**, Clojure retorna erro:

```
user=>  
user=> ( 1 2 3 )  
Execution error (ClassCastException) at user/eval1 (REPL:1).  
java.lang.Long cannot be cast to clojure.lang.IFn  
user=>
```

Listas como dados

- ✓ Para que listas não sejam interpretadas como funções, mas simplesmente como dados, devemos circundá-las por apóstrofe (`'`);

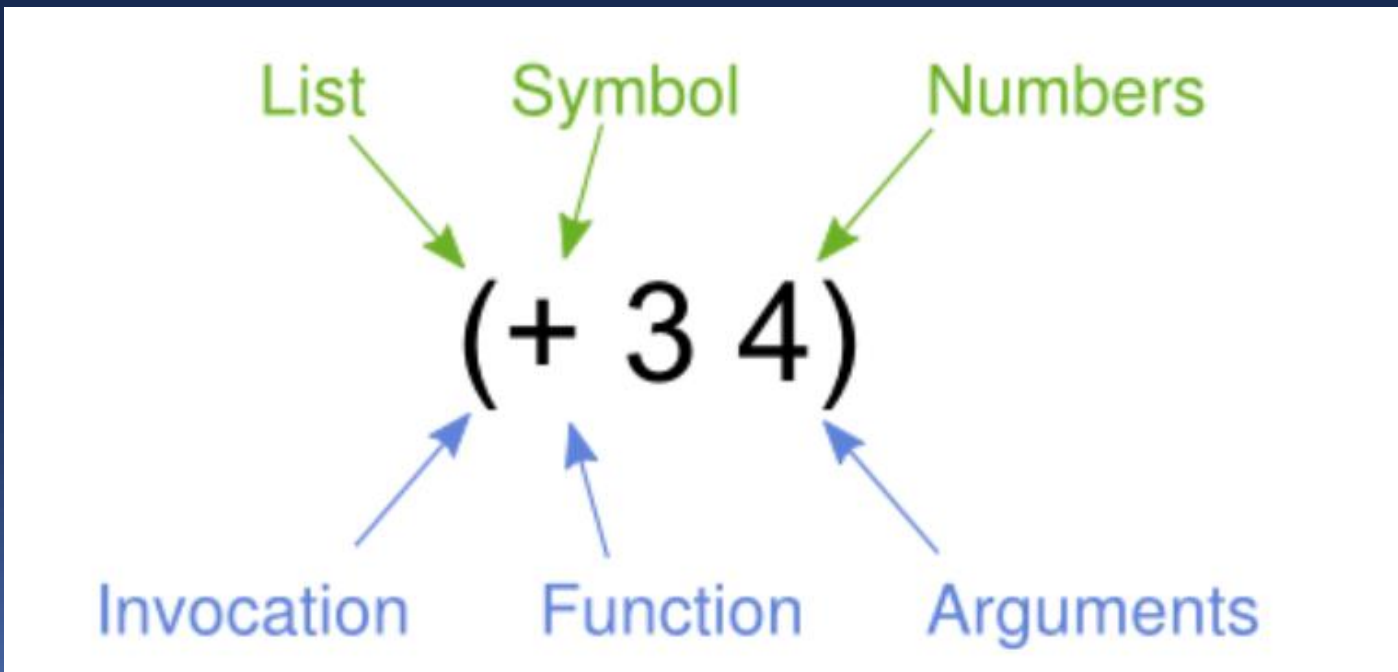
```
user=>
user=> '( 1 2 3 )
(1 2 3)
user=>
user=>
user=>
user=>
user=>
user=> '(1 (2 3 4 ) ("hello" 1 2 ("Hello" "World") ) )
(1 (2 3 4) ("hello" 1 2 ("Hello" "World")))
user=>
```

Código Clojure

- ✓ Clojure é dialeto **Lisp**;
- ✓ Código **Clojure** é constituído por estruturas de dados;
- ✓ Nossos programas podem gerar essas estruturas de dados;
- ✓ Assim, num jargão **Lisp**, programas em tempo de execução podem gerar código;
- ✓ Esse conceito é conhecido **por Meta-Programação**.

Special Forms

- ✓ Vimos até agora códigos nos quais **Clojure** emprega regras simples de avaliação incorporadas em listas;
- ✓ **Mas**, existem alguns comportamentos que **não** são tratados como nas avaliações usuais;



Special Forms

- ✓ Por exemplo, argumentos passados para uma função sempre são avaliados, mas e se **não** quisermos que todos os argumentos sejam avaliados ?
- ✓ Essas regras especiais de avaliação ocorrem nos “**Special Forms**”.
- ✓ Por exemplo, a “**special form**” **if** pode **não** avaliar um de seus argumentos, dependendo do resultado da avaliação do primeiro argumento;

Fluxo de Controle

- ✓ Em Java, expressões retornam valores, enquanto comandos (statements) não.

⟨⟩ Java

// "if" é um comando porque não retorna um valor:

```
String s;  
if (x > 10) {  
    s = "maior";  
} else {  
    s = "menor ou igual";  
}  
obj.someMethod(s);
```

// O operador ternário é uma expressão; ele retorna um valor:

```
obj.someMethod(x > 10 ? "maior" : "menor ou igual");
```

Fluxo de Controle

- ✓ De acordo com isso, os operadores de controle de fluxo também são expressões!
- ✓ Os operadores de controle de fluxo são componíveis, ou seja, podem ser utilizados em qualquer contexto. Isso resulta em menos código duplicado, bem como em uma menor necessidade de variáveis intermediárias.



Fluxo de Controle

- ✓ Em Clojure, no entanto, tudo é uma expressão !
- ✓ Tudo retorna um valor, e um bloco com múltiplas expressões retorna o último valor;
- ✓ Expressões que executam apenas efeitos colaterais retornam nil.



if

```
(if (> 2 5) "maior")
```

👉 Resultado:

```
nil
```

👉 mostra que if retorna nil quando não entra no bloco

if

- ✓ if é a expressão condicional mais importante — ela é composta por uma condição, um bloco “then” (então) e um bloco “else” (senão);
- ✓ O if avalia apenas o ramo selecionado pela condição.

```
(if true "sim" "não")
```

👉 Resultado:

```
"sim"
```

```
(if false "sim" "não")
```

👉 Resultado:

```
"não"
```

if

```
(if (< 2 1) "verdadeiro" "falso")
```

👉 Resultado:

```
"falso"
```

```
(if (even? 4) "par" "ímpar")
```

👉 Resultado:

```
"par"
```

if

```
(str "Resultado: " (if (> 10 5) "ok" "erro"))
```

👉 Resultado:

```
"Resultado: ok"
```

```
(if (= 3 3) "igual" "diferente")
```

👉 Resultado:

```
"igual"
```

if

```
(if true 10 20)
```

👉 Resultado:

```
</> clojure
```

```
10
```

📌 Aqui o `if` é o próprio valor

```
(+ 5 (if (> 3 2) 10 0))
```

👉 Resultado:

```
</> clojure
```

```
15
```

📌 O `if` retorna 10 → entra na soma

if

```
(str "Resultado: " (if (> 2 5) "erro" "ok"))
```

👉 Resultado:

```
</> clojure
```

```
"Resultado: ok"
```

```
(if (even? 6) 100 200)
```

👉 Resultado:

```
</> clojure
```

```
100
```

📌 O `if` funciona como um “selecionador de valor”

if

```
(if (> 5 3)  
  (if (even? 4) "ok" "erro")  
  "fora")
```

👉 Resultado:

```
</> clojure
```

```
"ok"
```

if

```
(max (if (> 10 5) 10 0) 7)
```

👉 Resultado:

```
</> clojure
```

```
10
```

📌 O `if` retorna um valor que vira argumento

if

```
(* 2 (if (> 4 3) 5 1))
```

👉 Resultado:

```
</> clojure
```

```
10
```

✓ Mostra claramente:

👉 `if` retorna valor → entra na multiplicação

if

```
(+ 5 (if (> 2 10) 3))
```

👉 Resultado:

✗ erro (porque retorna `nil`)

📌 Excelente para ensinar:

👉 nem sempre dá pra usar `if` sem `else`

Special form do

 O que é o `do`?

-  O `do` permite executar várias expressões em sequência
-  Ele retorna o valor da ÚLTIMA expressão

Special Forms – **do**

- ✓ Usado para executar uma série de expressões e retornar o valor da última expressão.

```
user=>  
user=> (do (println "2") (println "3"))  
2  
3  
nil  
user=>
```

Special form do

```
(do  
  1  
  2  
  3)
```

👉 Resultado:

```
</> clojure
```

```
3
```

📌 Só retorna o último valor

Special form do

```
(do  
  (+ 1 2)  
  (* 2 3)  
  (- 10 4))
```

👉 Resultado:

```
</> clojure
```

6

📌 Apenas o último resultado importa

Special form do

```
(if true
  (do
    (println "Entrou no IF")
    "resultado final")
  "não entrou")
```

👉 Saída:

```
Entrou no IF
```

👉 Resultado retornado:

```
</> clojure

"resultado final"
```

📌 Muito importante:

👉 `do` permite mais de uma instrução dentro do `if`

Special form do

```
(if true  
  (println "linha 1")  
  (println "linha 2"))
```

👉 Aqui só pode ter uma expressão por ramo

📌 Para múltiplas → precisa do `do`

Special form do

```
(do  
  (println "Calculando...")  
  (+ 10 20))
```

👉 Saída:

Calculando...

👉 Resultado:

⌘ clojure

30

Special form do

```
(if (> 5 3)
  (do
    (println "5 é maior que 3")
    "ok")
  "erro")
```

👉 Resultado:

```
</> clojure
```

```
"ok"
```

Special form do

```
(* 2
  (do
    (println "Multiplicando...")
    5))
```

👉 Saída:

Multiplicando...

👉 Resultado:

</> clojure

10

📌 Mostra:

👉 `do` executa várias coisas

👉 mas retorna só o último valor

Macro – **when**

- ✓ Usado quando somente estamos interessados no caso em que a condição lógica for verdadeira;
- ✓ É semelhante a um `if`, mas não contém ramificação `else` e pode ser tratado como um “do” implícito.

```
user=>  
user=> (when true (println "1") (println "2") (println "3") )  
1  
2  
3  
nil  
user=>
```

Macro – **when**

A macro `when` é usada quando você quer:

- 👉 Executar algo somente se a condição for verdadeira
- 👉 Sem precisar de `else`

Forma geral

`</>` `clojure`

```
(when condicao  
  expressao1  
  expressao2  
  ...)
```

- 👉 Se a condição for verdadeira → executa tudo
- 👉 Se for falsa → retorna `nil`

Macro – **when**

```
(when true  
  (println "Executou!"))
```

✓ Saída:

Executou!

</> clojure

```
(when false  
  (println "Não executa"))
```

✓ Saída:

(nada acontece)

Macro – **when**

```
(when (> 10 5)  
  (println "10 é maior que 5"))
```

✓ Saída:

```
10 é maior que 5
```

Macro – **when**

```
(when true  
  (println "Linha 1")  
  (println "Linha 2")  
  (println "Linha 3"))
```

✓ Saída:

```
Linha 1  
Linha 2  
Linha 3
```

👉 Aqui está o segredo:

O `when` usa internamente um `do`

Macro – **when**



Equivalente com `if`

```
</> clojure
```

```
(if true  
  (do  
    (println "Linha 1")  
    (println "Linha 2"))))
```



Ou seja:



`when` = `if` + `do`

Macro – **when**

```
(when true  
  "Valor retornado")
```

✓ Resultado:

```
"Valor retornado"
```

```
(when false  
  "Não aparece")
```

✓ Resultado:

```
nil
```

Macro – **when**

```
(when (> 18 17)  
  (println "Maior de idade"))
```

👉 Muito usado para:

- validações simples
- efeitos colaterais (println, salvar, etc.)

Macro – **when**

 Quando **NÃO** usar `when`

Se precisar de `else`, **NÃO** use `when` 

```
(when (> 5 10)
  (println "Maior"))
```

 Aqui não tem alternativa!

 Use `if`:

`</>` clojure

```
(if (> 5 10)
  (println "Maior")
  (println "Menor"))
```

Macro – **when**

Situação	Use
Só tem "se verdadeiro"	 <code>when</code>
Tem "senão"	 <code>if</code>
Várias expressões	 <code>when</code>
Quer retorno com else	 <code>if</code>

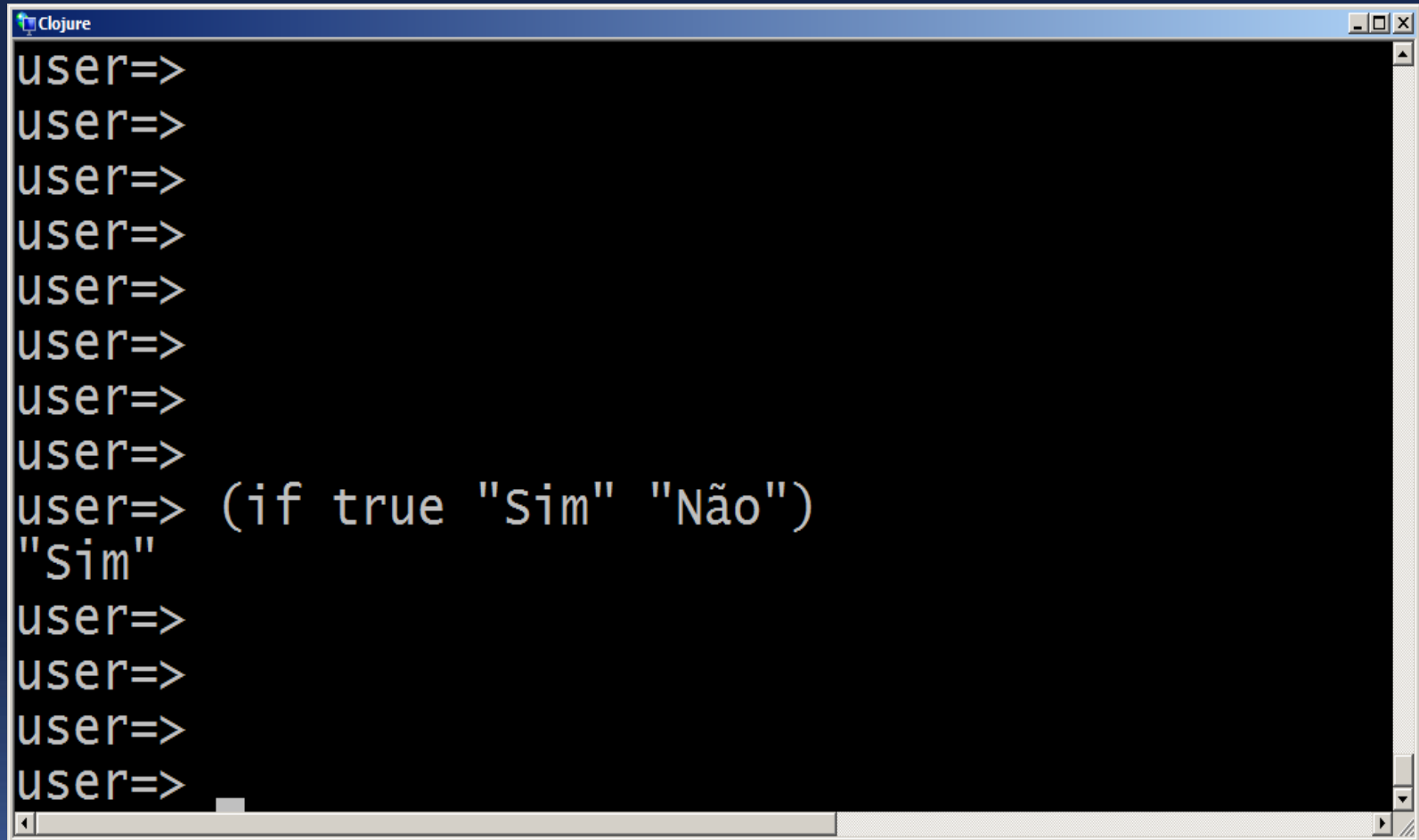


"Use `when` quando você só quer fazer algo se a condição for verdadeira."

Tarefa 2

- ✓ Abra o **REPL**;
- ✓ Escreva a função `if` que retorna "Sim" se a condição for verdadeira e "Não" caso contrário.
- ✓ Encerre o **REPL**.

Tarefa 2



A screenshot of a Clojure REPL window. The window has a title bar that says "Clojure". The background is black with white text. The text shows a series of prompts "user=>" followed by a newline. The eighth prompt is followed by the code "(if true \"Sim\" \"Não\")". The ninth prompt is followed by the output "\"Sim\"". The window has standard OS window controls (minimize, maximize, close) in the top right corner and a scrollbar on the right side.

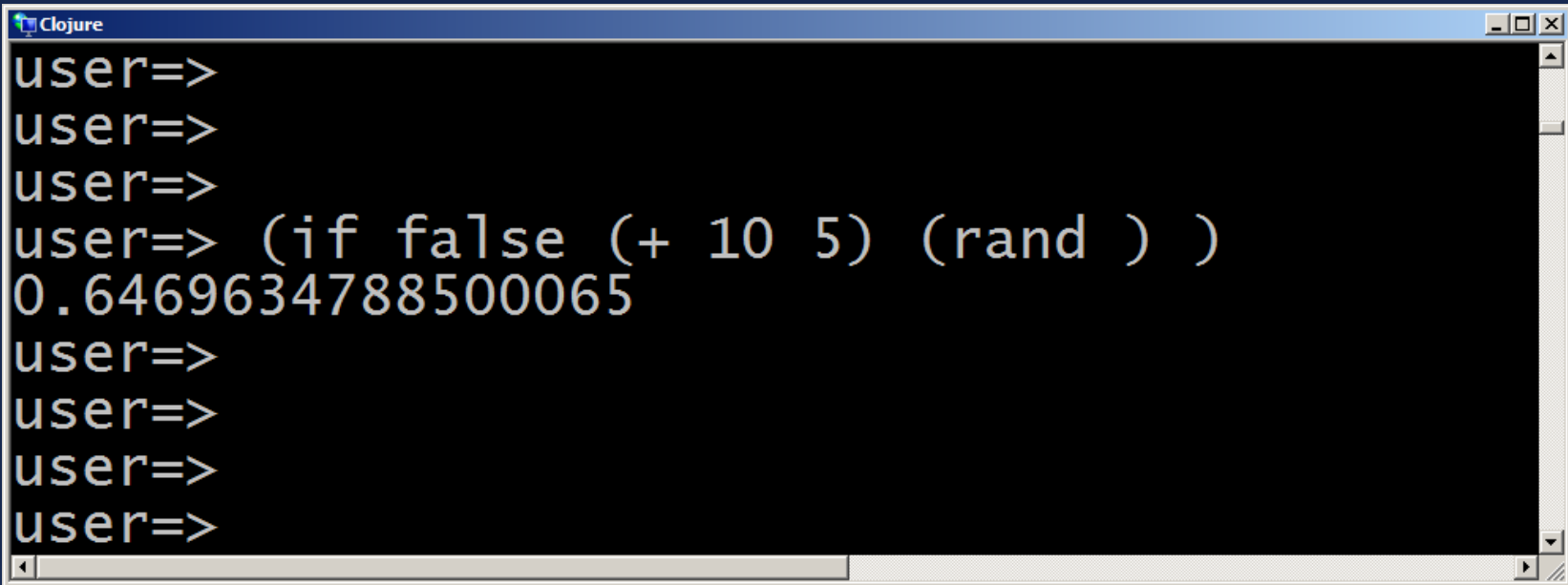
```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (if true "Sim" "Não")
"Sim"
user=>
user=>
user=>
user=>
```

Tarefa 3

- ✓ Abra o **REPL**;
- ✓ Escreva a função `if` que retorna a soma de 10 com 5 se a condição for falsa. Caso contrário retorna um número randômico.
- ✓ Encerre o **REPL**.

Dica. A função `rand` retorna um número **rand**.

Tarefa 3

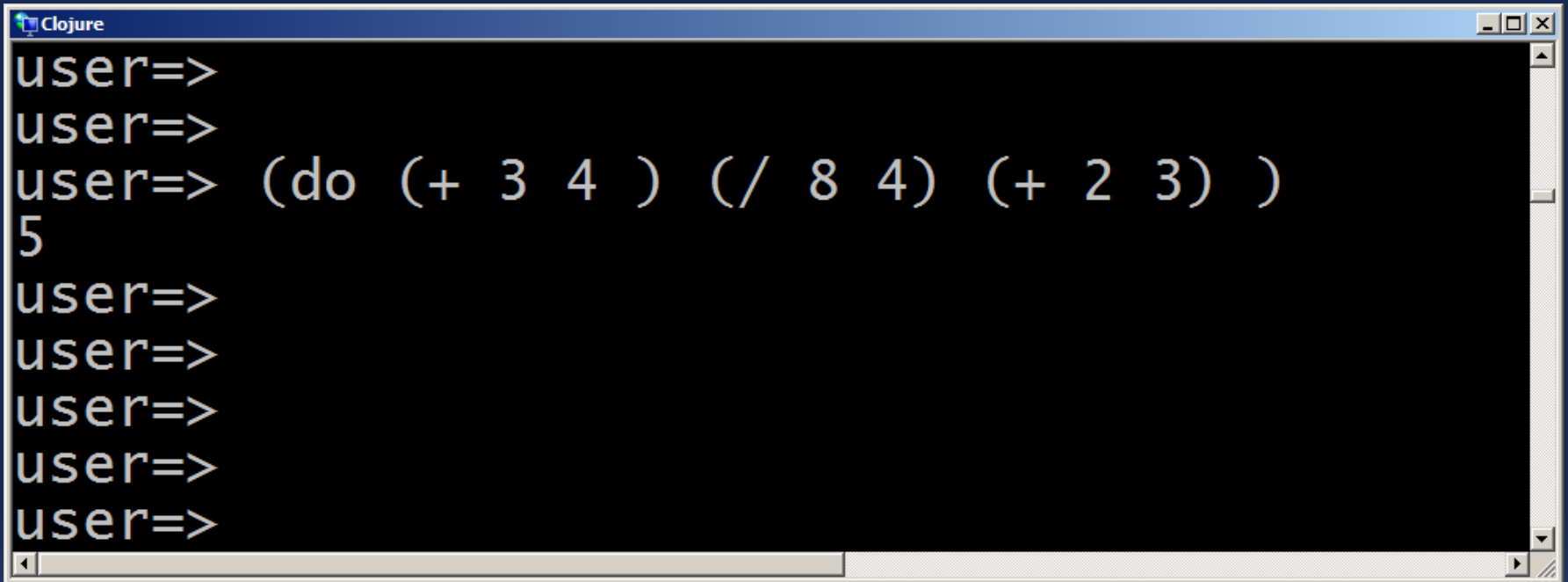
A screenshot of a Clojure REPL window. The window has a title bar with the text 'Clojure' and standard window controls (minimize, maximize, close). The main area is a black terminal with white text. It shows a series of prompts 'user=>' and the execution of a Clojure expression. The expression is '(if false (+ 10 5) (rand))'. The result of the execution is '0.6469634788500065'. There are four more 'user=>' prompts below the result, but they are not followed by any input or output.

```
Clojure
user=>
user=>
user=>
user=> (if false (+ 10 5) (rand) )
0.6469634788500065
user=>
user=>
user=>
user=>
```

Tarefa 4

- ✓ Abra o **REPL**;
- ✓ Escreva a função "**do**" que execute sequencialmente as seguintes avaliações:
 - ✓ Multiplica 3 por 4;
 - ✓ Divide 8 por 4;
 - ✓ Soma 2 com 3.
- ✓ Encerre o **REPL**.

Tarefa 4

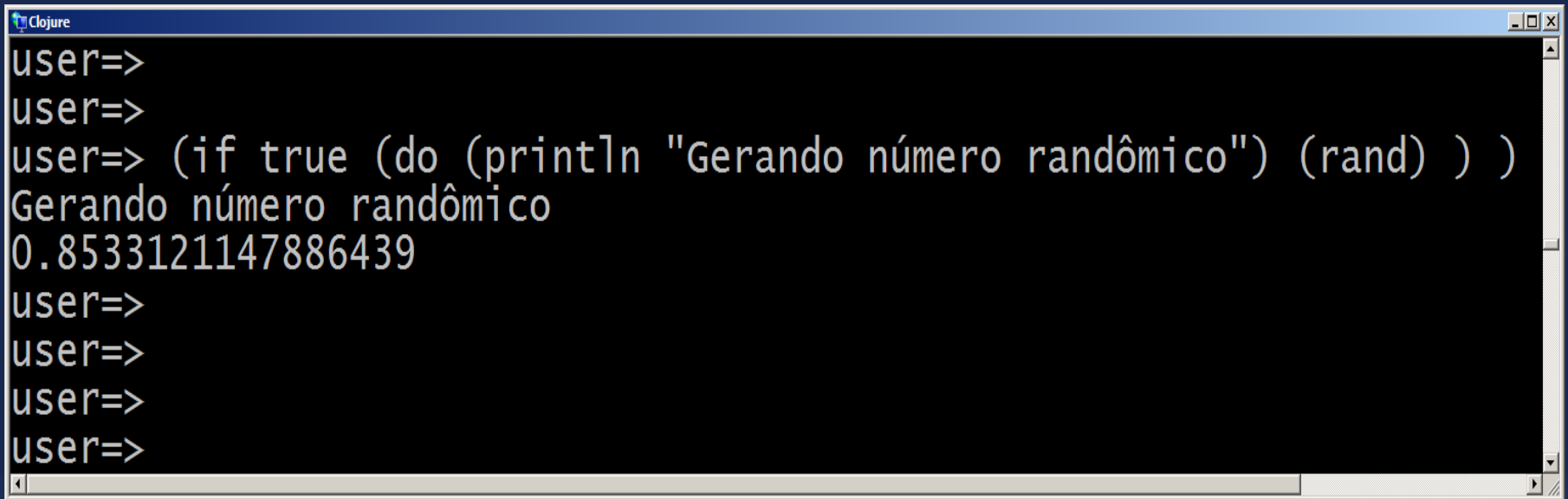
A screenshot of a Clojure REPL window. The window has a title bar with the text "Clojure" and standard window controls (minimize, maximize, close). The main area is a black terminal with white text. It shows a series of prompts "user=>" and corresponding inputs and outputs. The first two prompts are empty. The third prompt is followed by the expression "(do (+ 3 4) (/ 8 4) (+ 2 3))" and the output "5". The next four prompts are empty.

```
Clojure
user=>
user=>
user=> (do (+ 3 4) (/ 8 4) (+ 2 3) )
5
user=>
user=>
user=>
user=>
user=>
```

Tarefa 5

- ✓ Abra o **REPL**;
- ✓ Escreva a função "**if**" que imprime a mensagem "Gerando um número randômico" e exibe o número gerado, caso a condição seja verdadeira;
- ✓ Encerre o **REPL**.

Tarefa 5

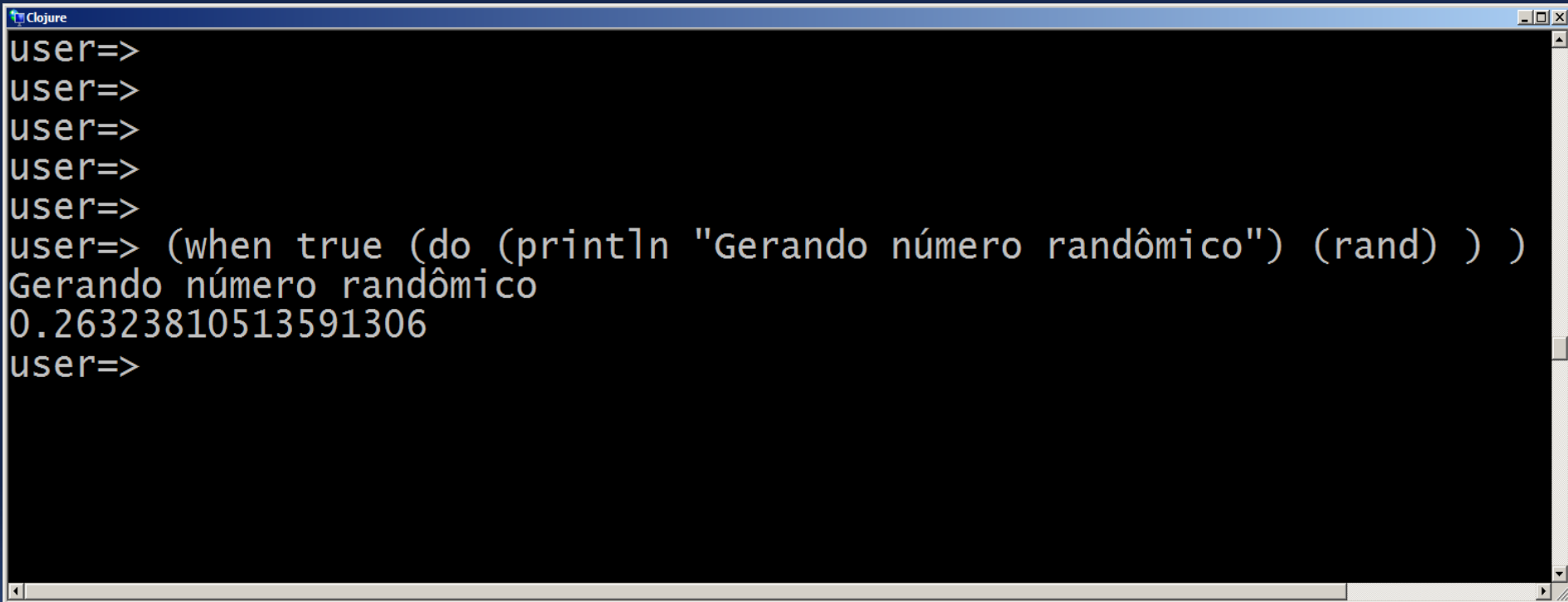
A screenshot of a Clojure REPL window. The window has a title bar that says "Clojure". The background is black with white text. The text shows a series of user prompts "user=>" and the execution of a Clojure expression. The expression is "(if true (do (println \"Gerando número randômico\") (rand)))". The output of the expression is "Gerando número randômico" followed by a new line and the value "0.8533121147886439". There are four more "user=>" prompts below the output, but they are not followed by any text.

```
Clojure
user=>
user=>
user=> (if true (do (println "Gerando número randômico") (rand) ) )
Gerando número randômico
0.8533121147886439
user=>
user=>
user=>
user=>
```

Tarefa 6

- ✓ Abra o **REPL**;
- ✓ Escreva a função "**when**" que imprime a mensagem "Gerando um número randômico" e exibe o número gerado, caso a condição seja verdadeira;
- ✓ Encerre o **REPL**.

Tarefa 6

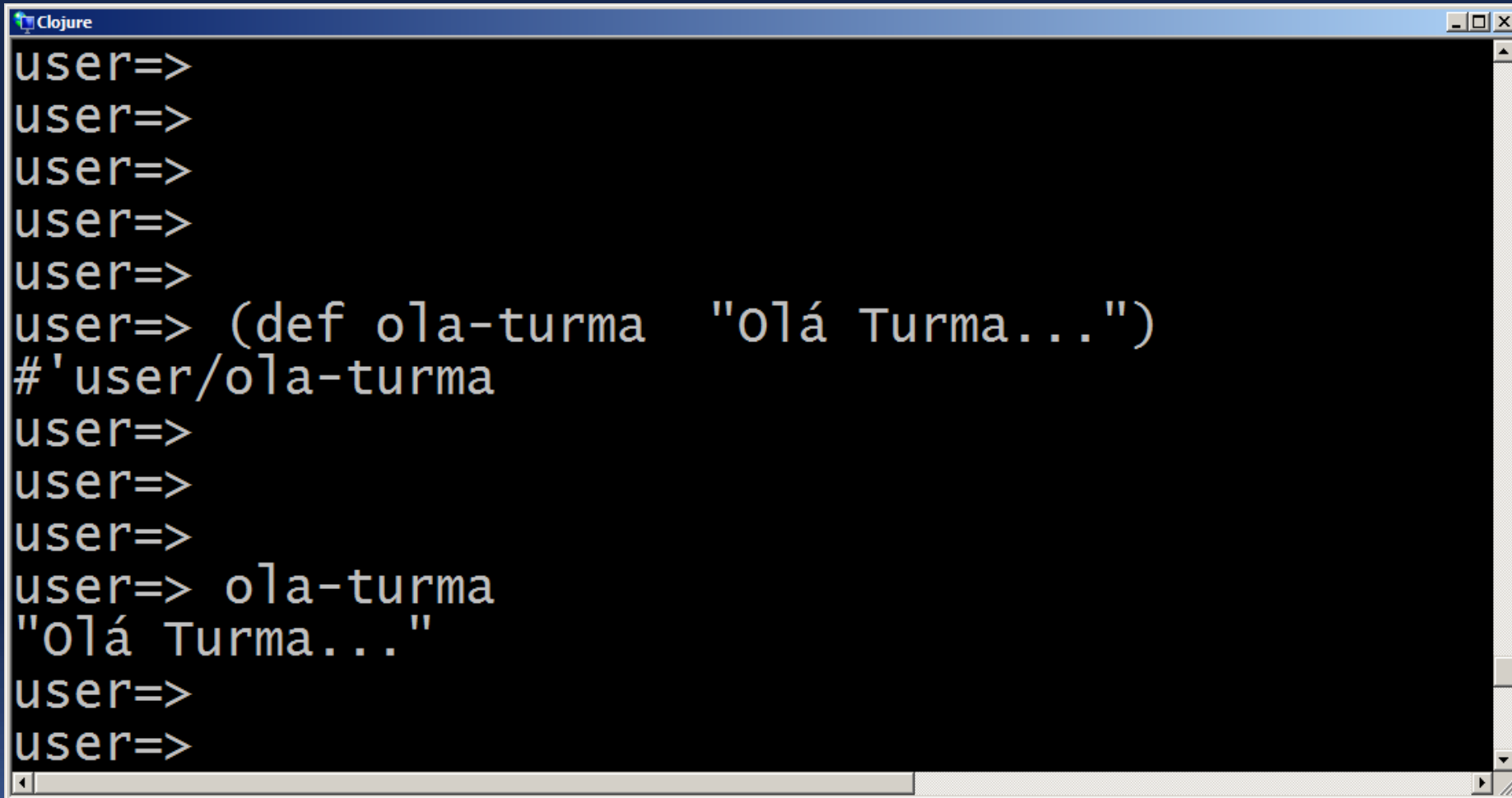
A screenshot of a Clojure REPL window. The window has a title bar that says "Clojure". The background is black with white text. The text shows a series of prompts "user=>" and a function call. The function call is "(when true (do (println \"Gerando número randômico\") (rand)))". The output of the function call is "Gerando número randômico" followed by the random number "0.26323810513591306".

```
Clojure
user=>
user=>
user=>
user=>
user=>
user=> (when true (do (println "Gerando número randômico") (rand) ) )
Gerando número randômico
0.26323810513591306
user=>
```

Bindings

- ✓ Em **Clojure**, **Binding** significa **ligação** de símbolos à valores;
- ✓ O termo **Binding** é preferível ao invés de atribuição, pois essas ligações quase sempre são feitas uma única vez;
- ✓ Diferentemente de atribuição em variáveis que, na grande maioria das linguagens, ocorrem com frequência justificando assim o uso no nome de variáveis;
- ✓ Em **Clojure**, o nome **símbolo** é preferível ao nome **variável**;
- ✓ **Bindings** podem ser feitos em **Clojure** de forma local (função **let**) ou de forma global (função **def**).

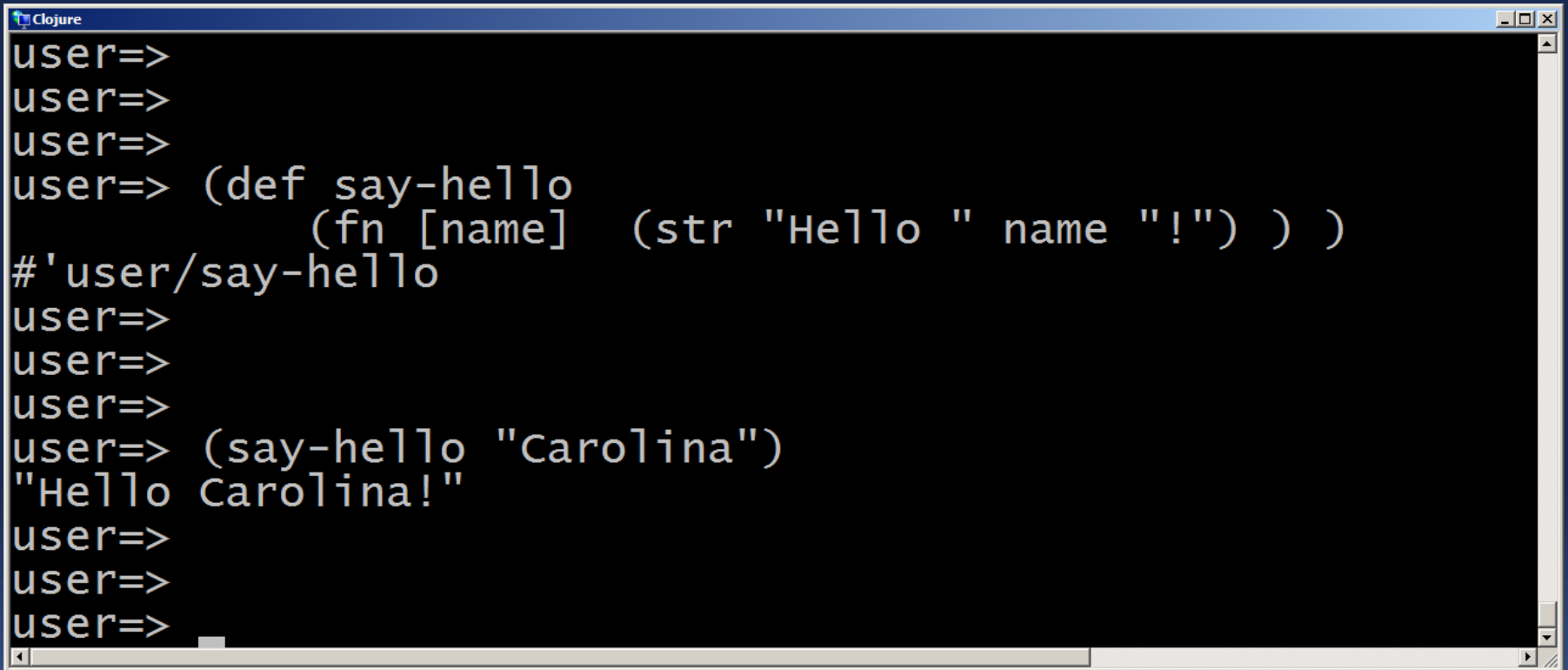
def



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=> (def ola-turma "Olá Turma...")
#'user/ola-turma
user=>
user=>
user=>
user=> ola-turma
"Olá Turma..."
user=>
user=>
```

def

✓ Pode ser usada para se associar um **nome** à uma função;



```
Clojure
user=>
user=>
user=>
user=> (def say-hello
        (fn [name] (str "Hello " name "!")) ) )
#'user/say-hello
user=>
user=>
user=>
user=> (say-hello "Carolina")
"Hello Carolina!"
user=>
user=>
user=>
```

def

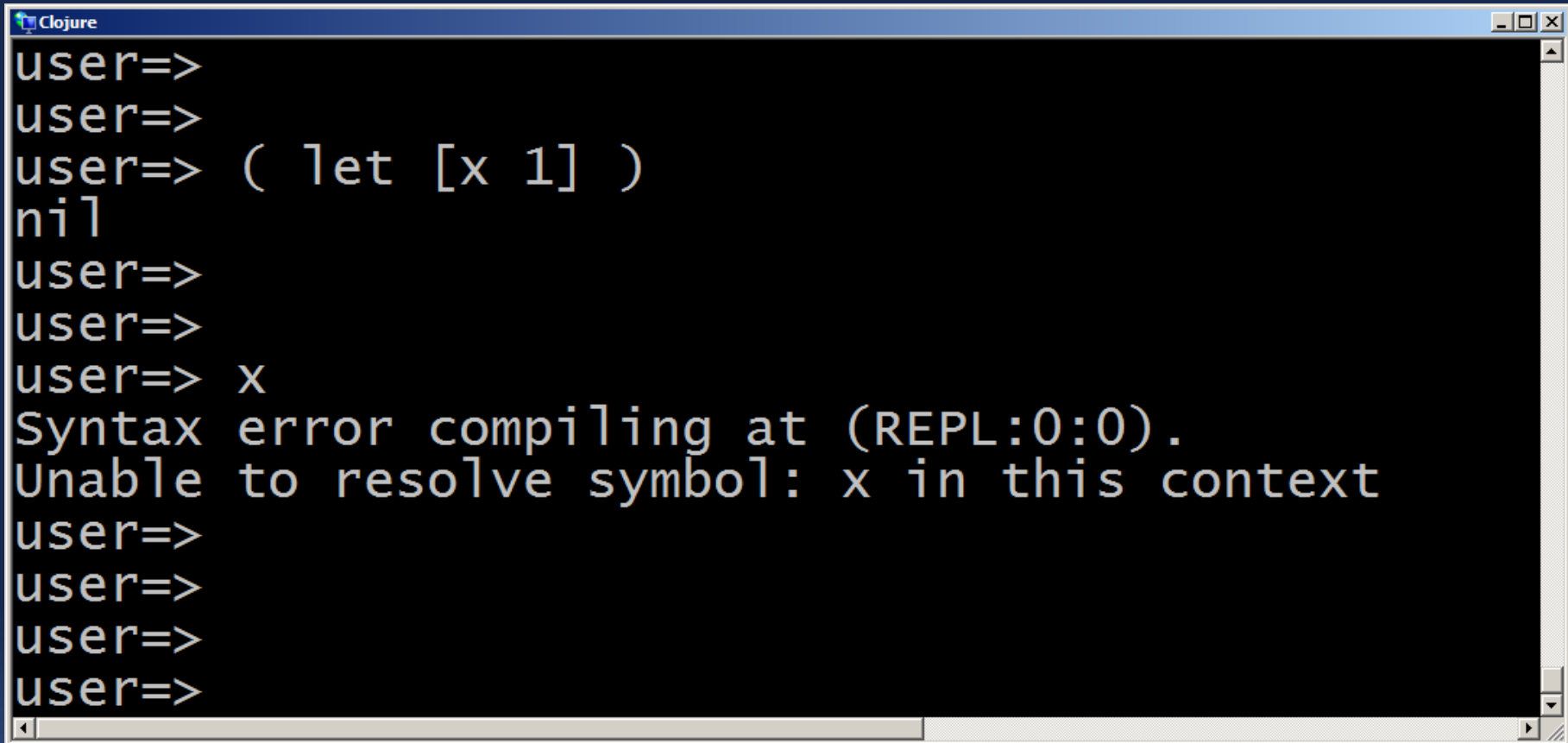
- ✓ Usado para se ligar (**binding**) símbolos a determinados valores;

```
Clojure
user=>
user=>
user=> (def programadortop "Mauricio")
#'user/programadortop
user=>
user=>
user=>
user=> (println (str " O " programadortop " é um excelente programador") )
O Mauricio é um excelente programador
nil
user=>
user=>
user=> (def programadortop "Mauricio Szabo" )
#'user/programadortop
user=>
user=>
user=>
user=> (println (str " O " programadortop " é um excelente programador") )
O Mauricio Szabo é um excelente programador
nil
user=>
```

let

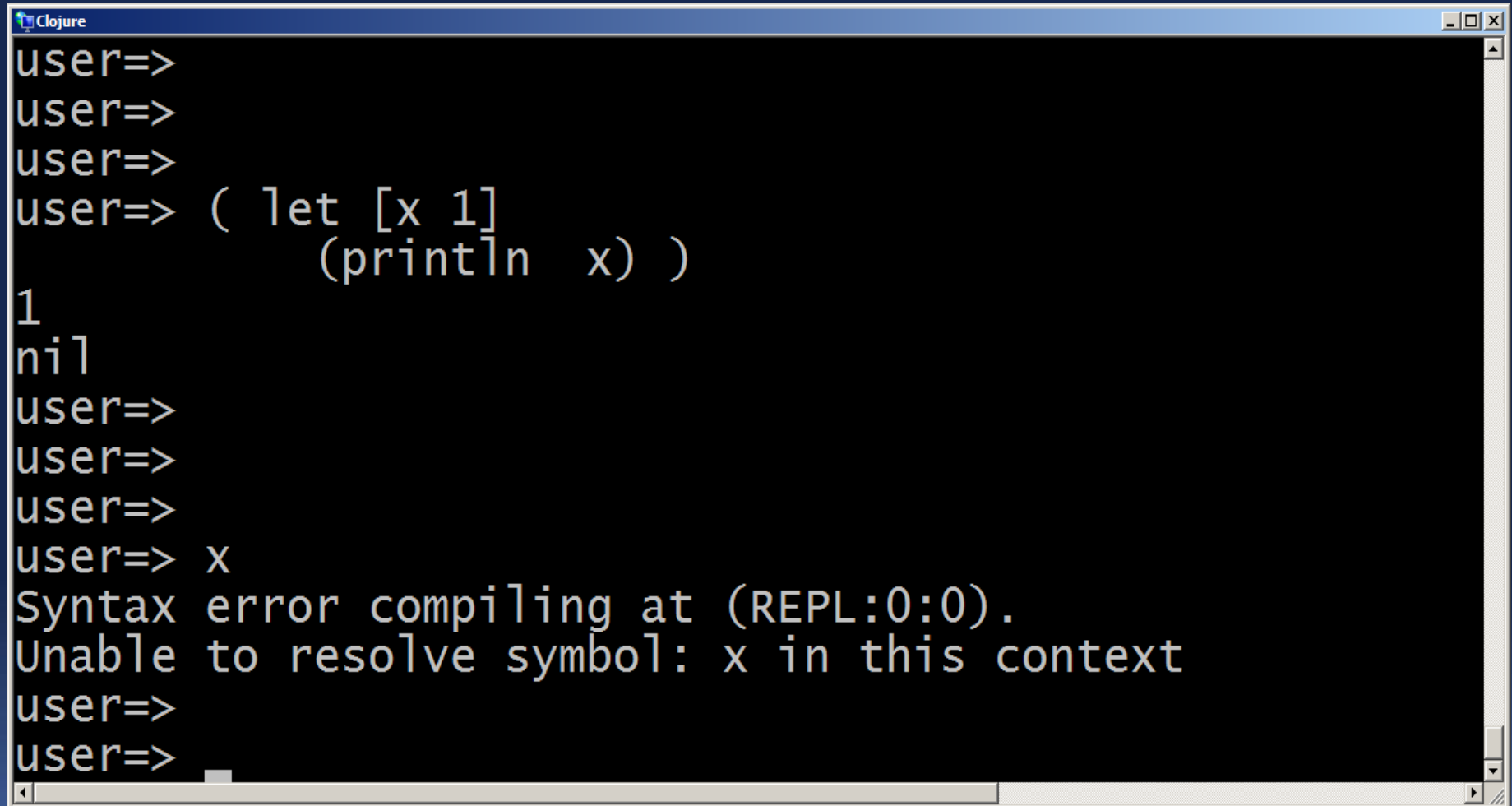
- ✓ Usado para se ligar (**binding**) símbolos a determinados valores;
- ✓ Opera com um **vetor** o qual associa um **símbolo** especificado no primeiro elemento a um **valor** especificado no segundo elemento;
- ✓ Os símbolos definidos no **let não** podem ser usados fora do let. Este comportamento é semelhante à variáveis **private** em outras linguagens;
- ✓ Avalia a expressão em um contexto léxico.

let



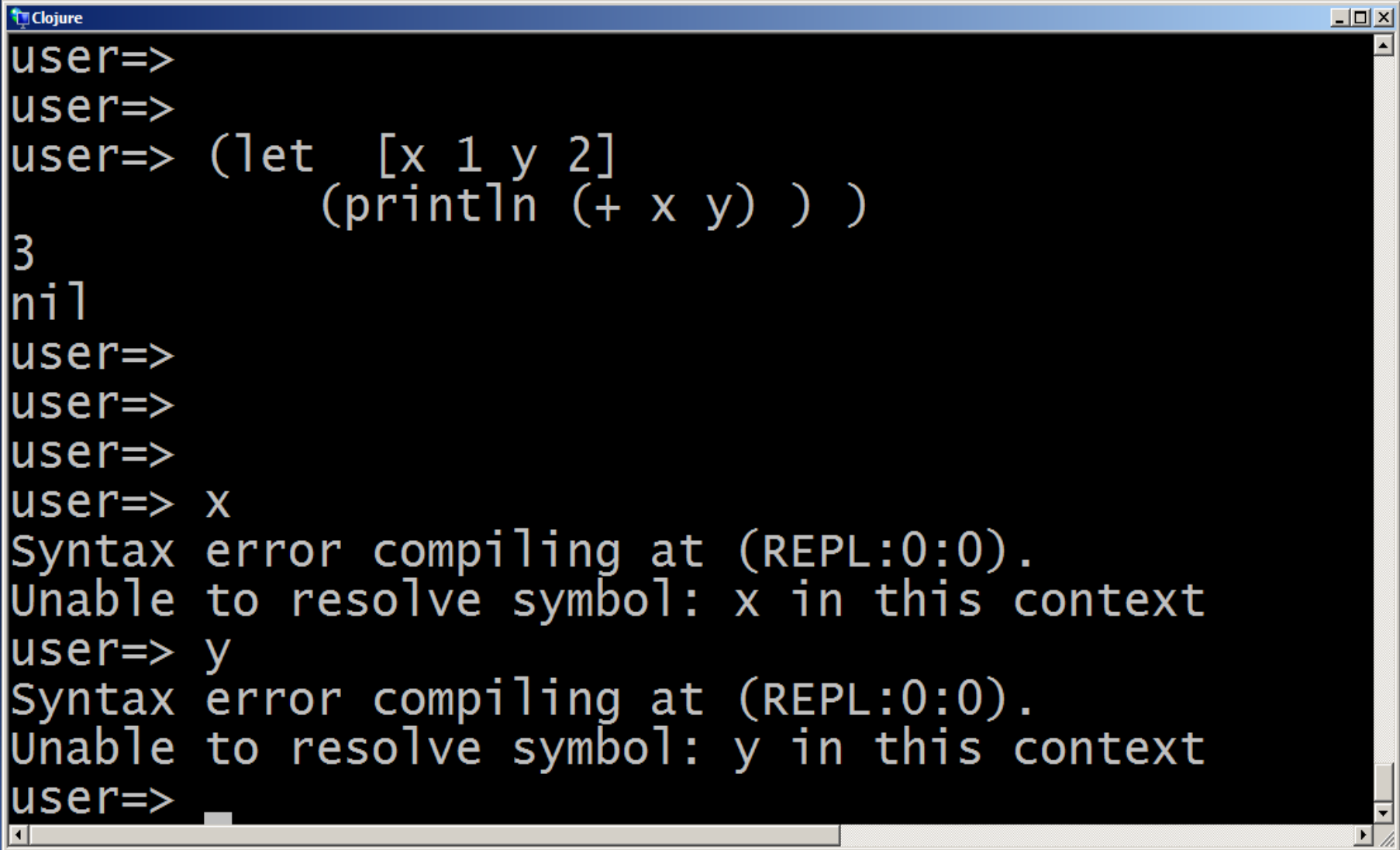
```
Clojure
user=>
user=>
user=> ( let [x 1] )
nil
user=>
user=>
user=> x
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: x in this context
user=>
user=>
user=>
user=>
```

let



```
Clojure
user=>
user=>
user=>
user=> ( let [x 1]
          (println x) )
1
nil
user=>
user=>
user=>
user=> x
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: x in this context
user=>
user=>
```

let



```
Clojure
user=>
user=>
user=> (let [x 1 y 2]
          (println (+ x y) ) )
3
nil
user=>
user=>
user=>
user=> x
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: x in this context
user=> y
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: y in this context
user=>
```

let

```
Clojure
user=>
user=> (let [m "Maurício" a "Aparecido"] (println (str m " e " a) ) )
Maurício e Aparecido
nil
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> m
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: m in this context
user=>
user=>
user=> a
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: a in this context
user=>
```

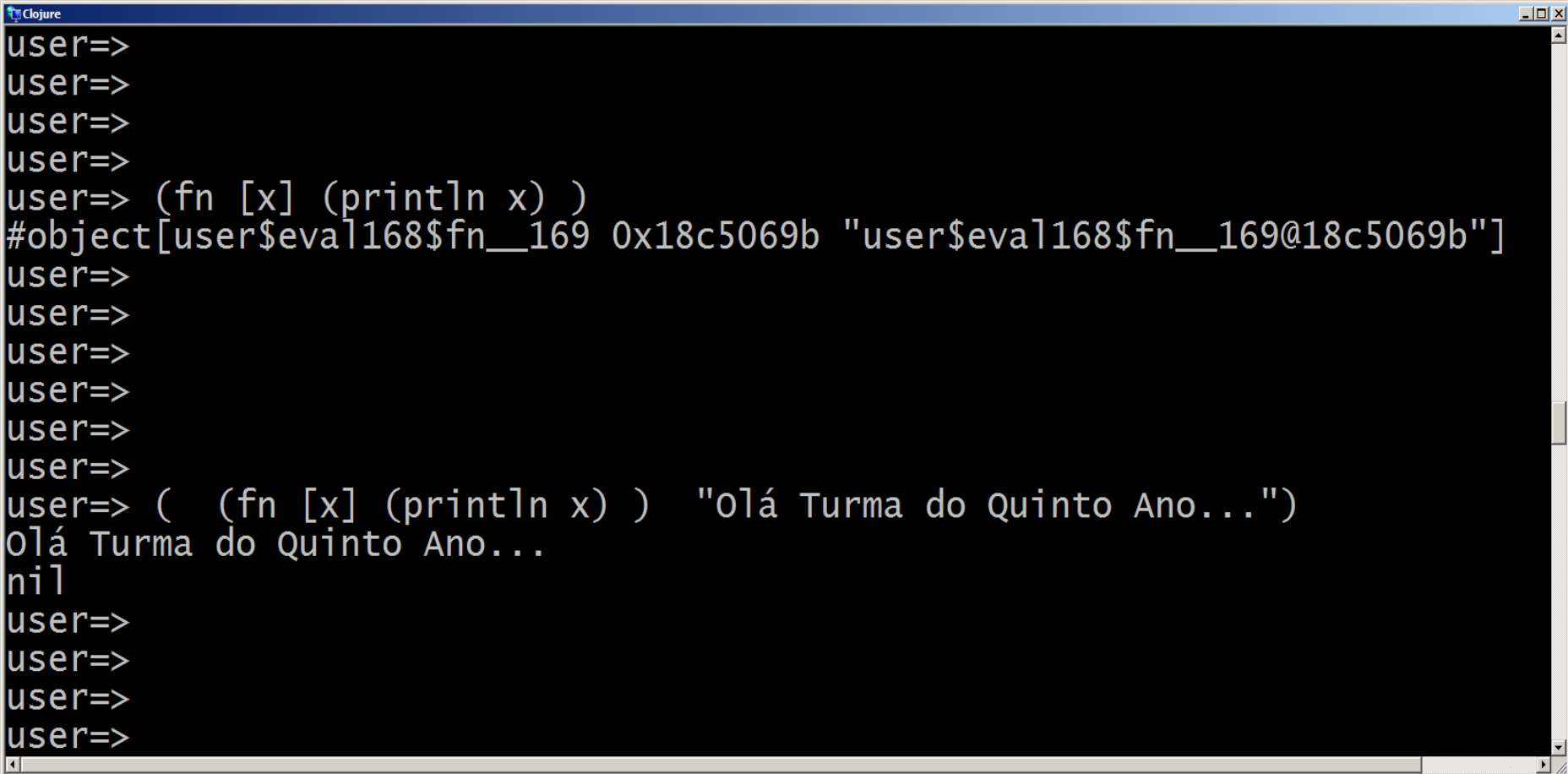
fn

- ✓ Funções são **objetos de primeira ordem em Clojure**; Isso significa que em **Clojure** pode-se fazer todas as operações básicas com funções;
- ✓ Ou seja, pode-se passar funções à outras funções, retornar funções de outras funções e ligar (binding) funções à símbolos (variáveis);
- ✓ A macro **fn** é usada para se criar **funções anônimas**;
- ✓ **#()** é o shortcut para **fn**;
- ✓ **%** será substituído pelos argumentos passados para a função. Quando houver múltiplos argumentos, **%1** para o primeiro argumento, **%2** para o segundo argumento e assim por diante.

Macro – **fn**

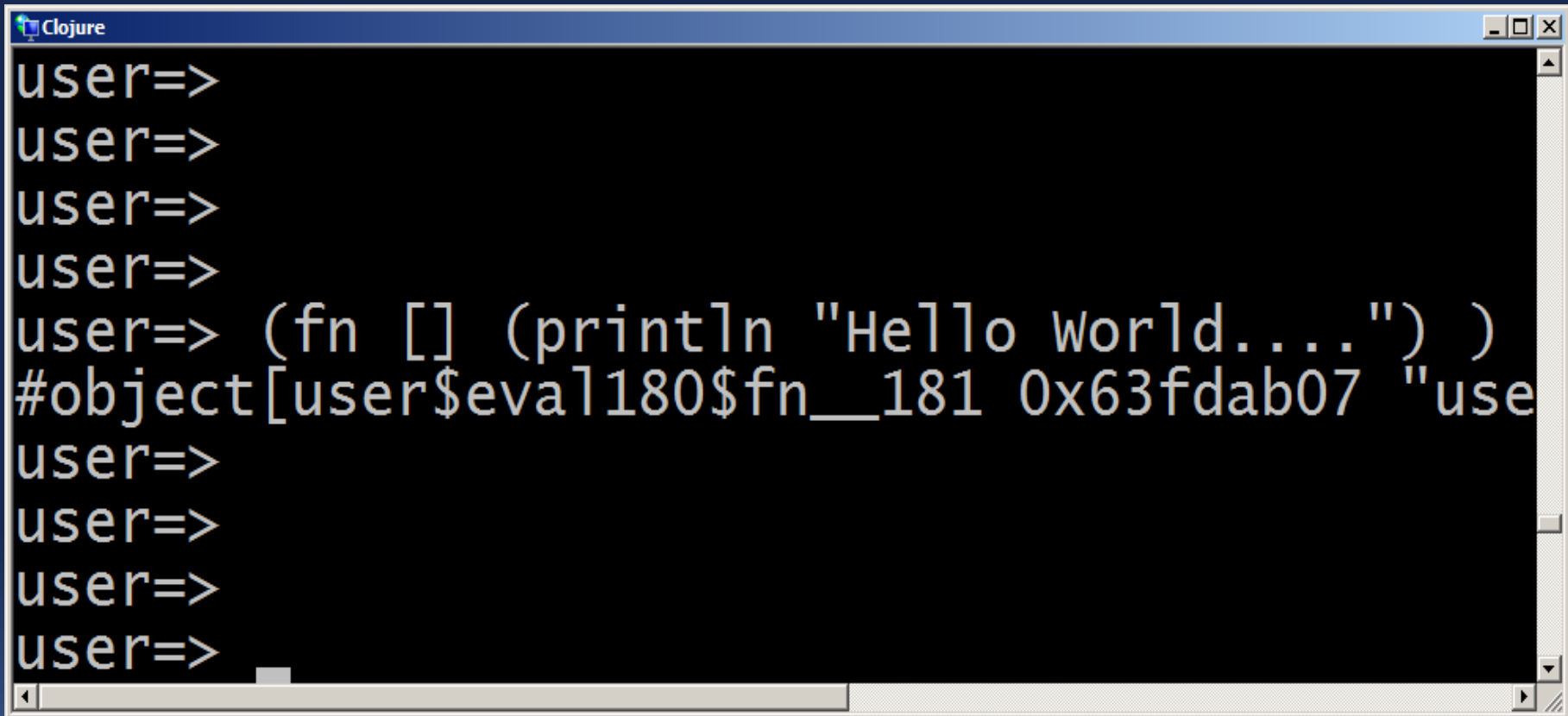
```
user=>  
user=>  
user=> (fn [x] (println x) )  
#object[user$eval1176$fn__177 0x1755e85b "user$eval1176$fn__177@1755e85b"]  
user=>
```

Macro – fn



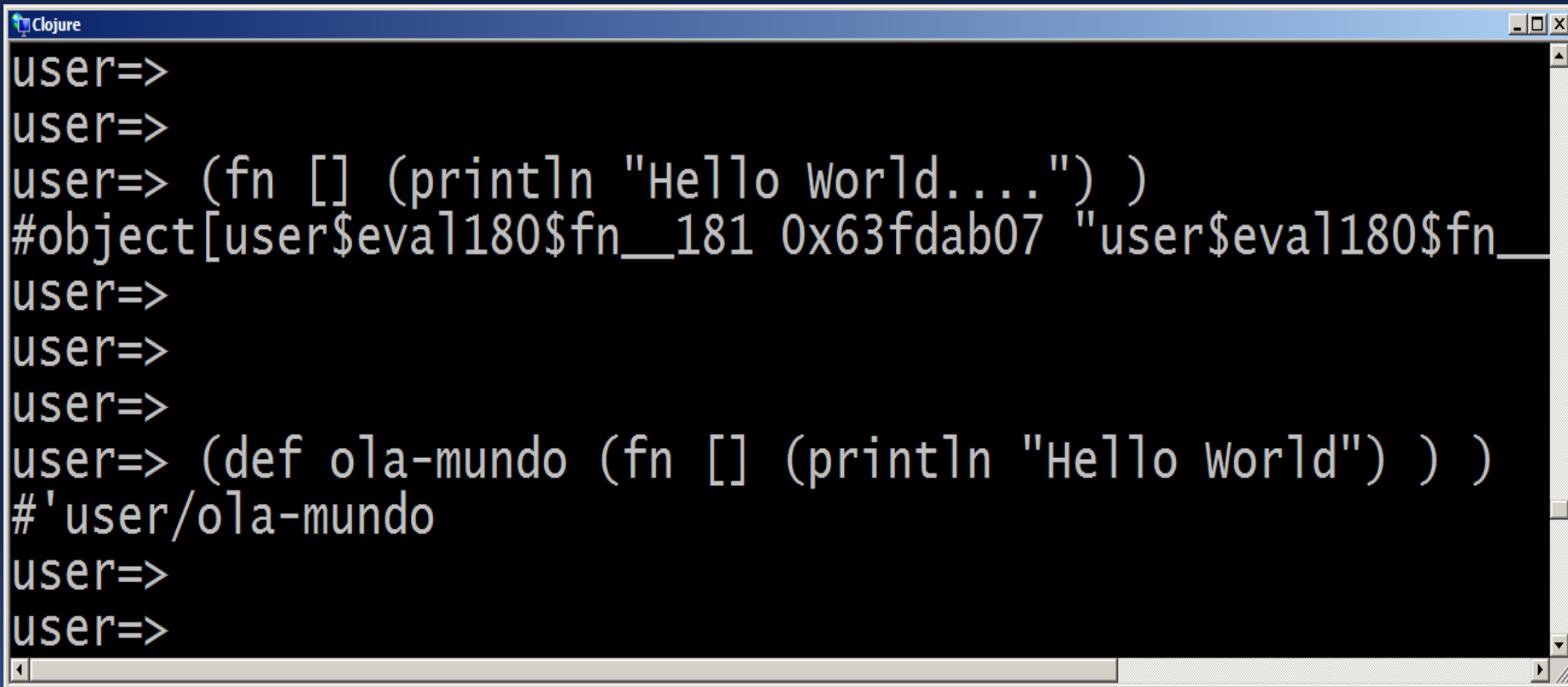
```
Clojure
user=>
user=>
user=>
user=>
user=> (fn [x] (println x) )
#object[user$eval168$fn__169 0x18c5069b "user$eval168$fn__169@18c5069b"]
user=>
user=>
user=>
user=>
user=>
user=> ( (fn [x] (println x) ) "Olá Turma do Quinto Ano...")
Olá Turma do Quinto Ano...
nil
user=>
user=>
user=>
user=>
```

Macro – **fn**



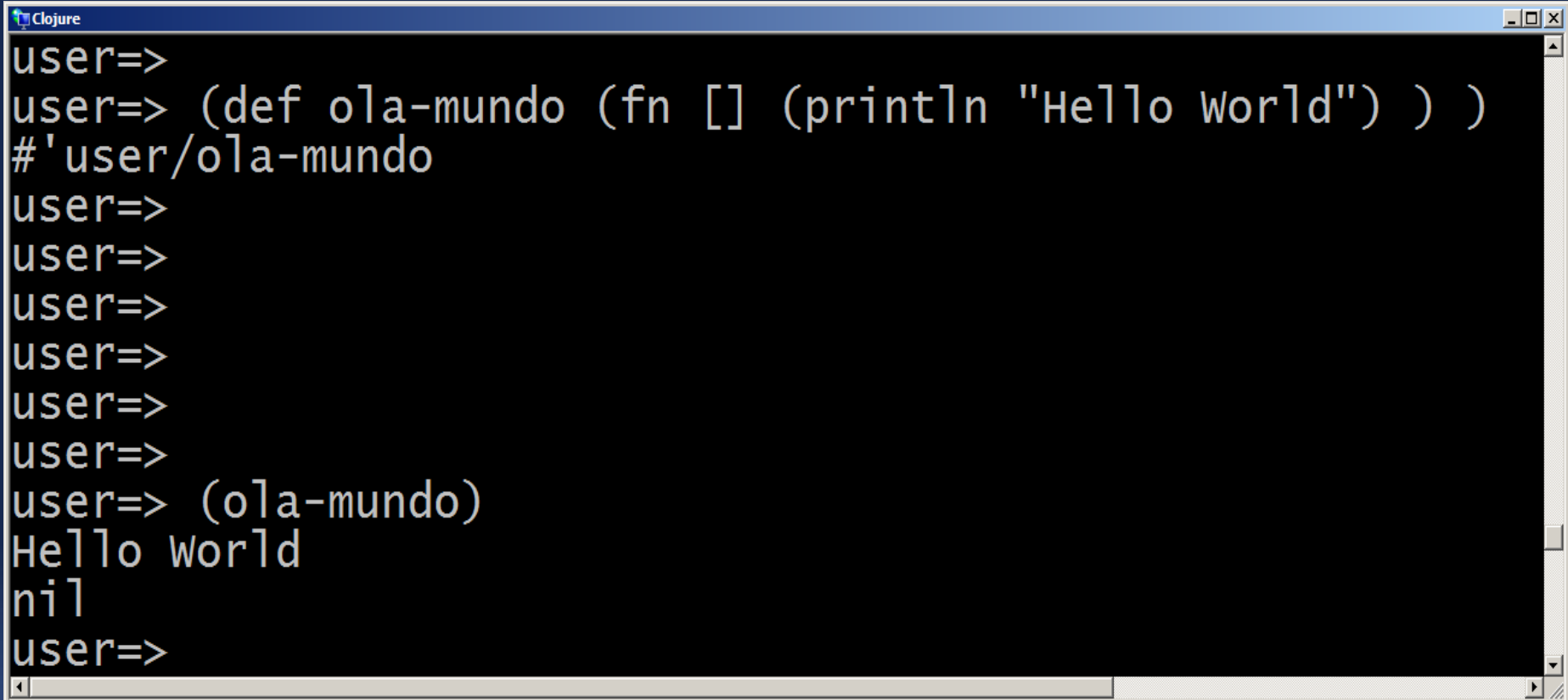
```
Clojure
user=>
user=>
user=>
user=>
user=> (fn [] (println "Hello World...."))
#object[user$eval180$fn__181 0x63fdab07 "use
user=>
user=>
user=>
user=>
```

Macro – fn



```
Clojure
user=>
user=>
user=> (fn [] (println "Hello world...."))
#object[user$eval180$fn__181 0x63fdab07 "user$eval180$fn__181"]
user=>
user=>
user=>
user=> (def ola-mundo (fn [] (println "Hello world")))
#'user/ola-mundo
user=>
user=>
```

Macro – fn



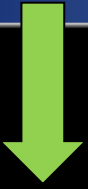
```
Clojure
user=>
user=> (def ola-mundo (fn [] (println "Hello World") ) )
#'user/ola-mundo
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (ola-mundo)
Hello World
nil
user=>
```

Macro – fn



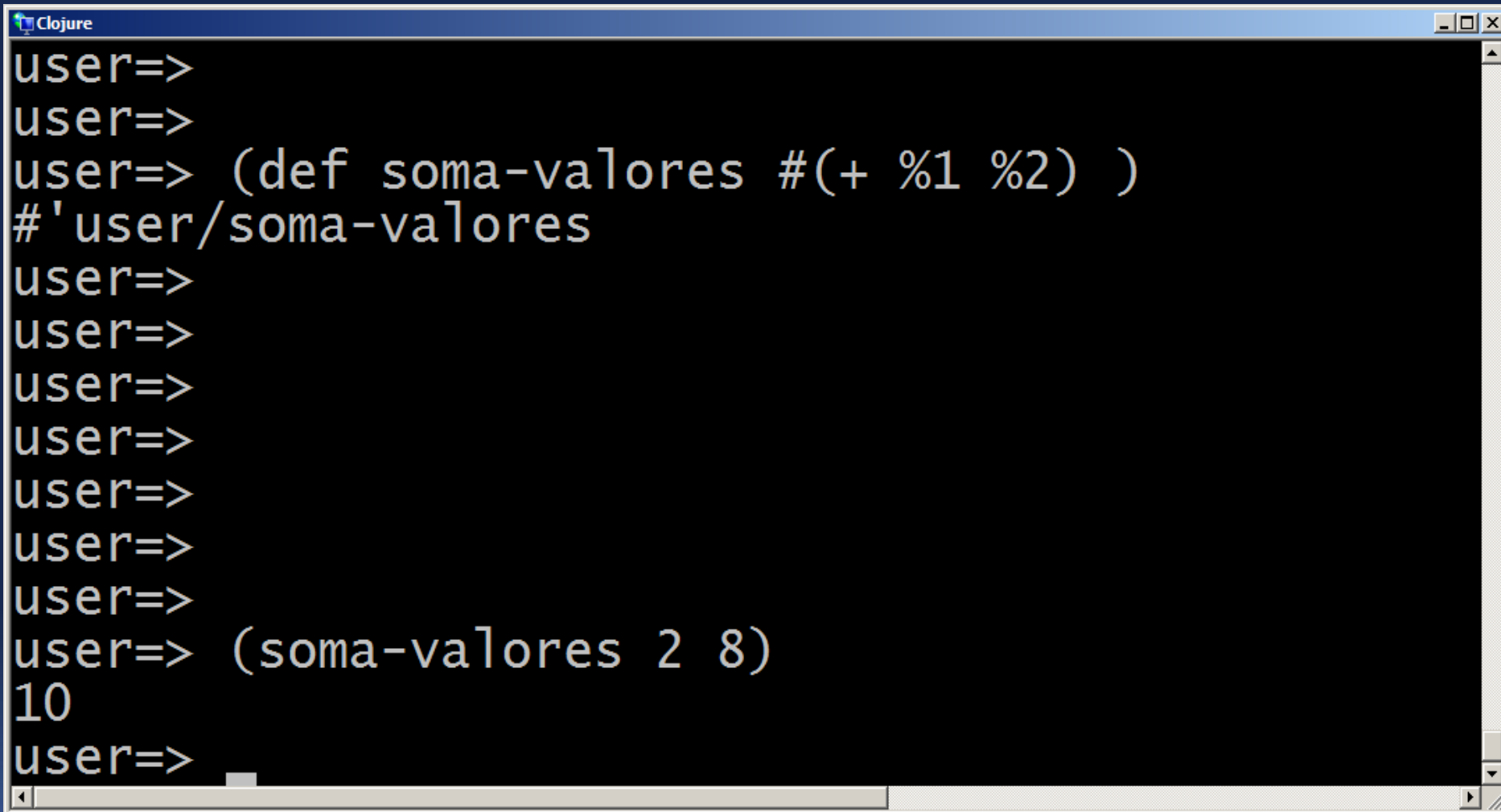
```
Clojure
user=>
user=>
user=> #(println %)
#object[user$eval1189$fn__190 0x1506f20f "user$eval1189$fn__
user=>
user=>
user=>
user=> ( #(println %) "Olá quinto ano...")
Olá quinto ano...
nil
user=>
user=>
user=>
```

Macro – **fn**



```
Clojure
user=>
user=>
user=>
user=> #(+ %1 %2)
#object[user$eval1203$fn__204 0xc5ee75e "user$eval1203$fn__204" 1]
user=>
user=>
user=>
user=>
user=> ( #(+ %1 %2) 7 4)
11
user=>
user=>
user=>
```

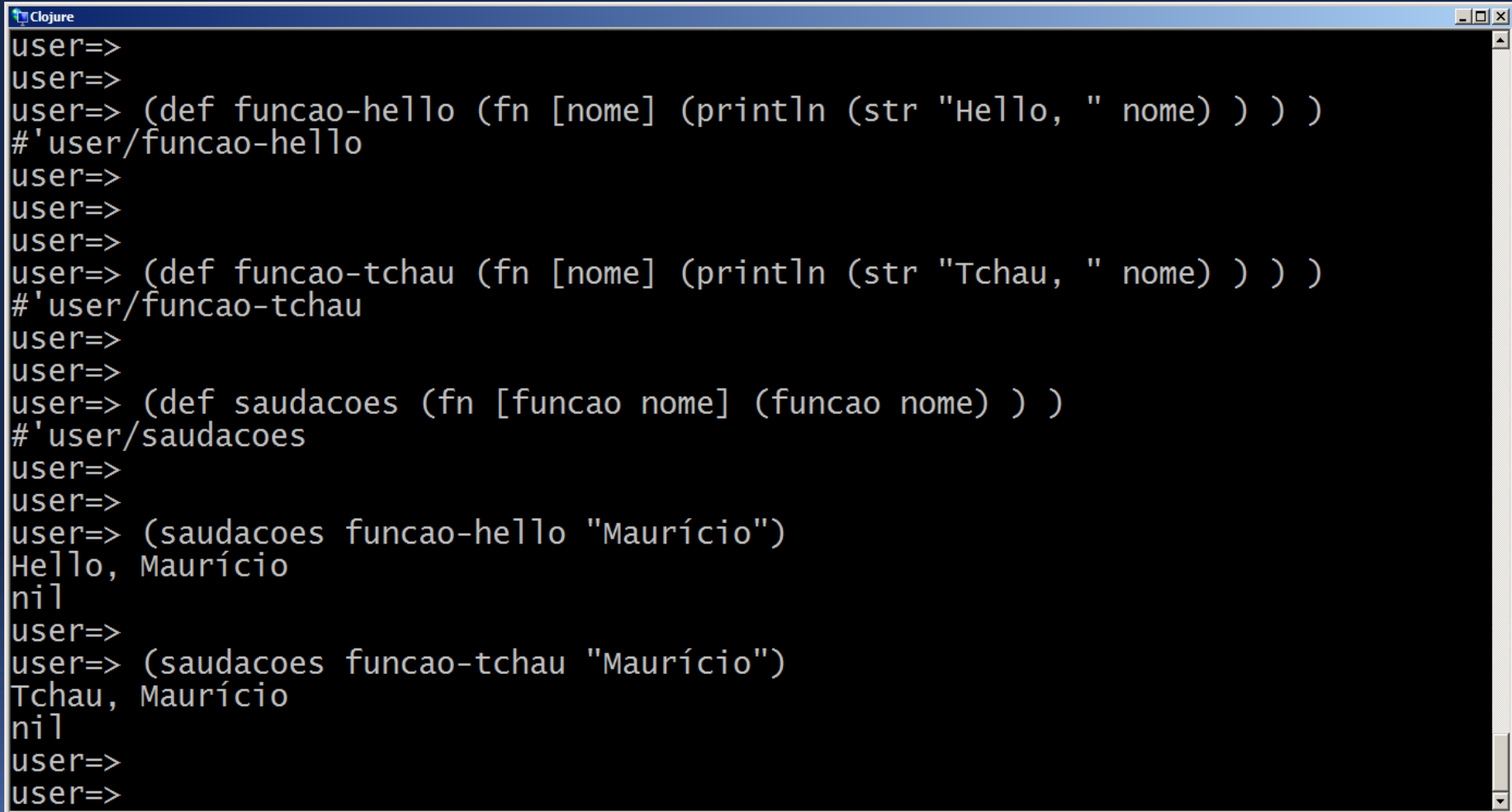
Macro – fn



```
Clojure
user=>
user=>
user=> (def soma-valores #(+ %1 %2) )
#'user/soma-valores
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (soma-valores 2 8)
10
user=>
```

Macro – **fn**

Passando funções à funções

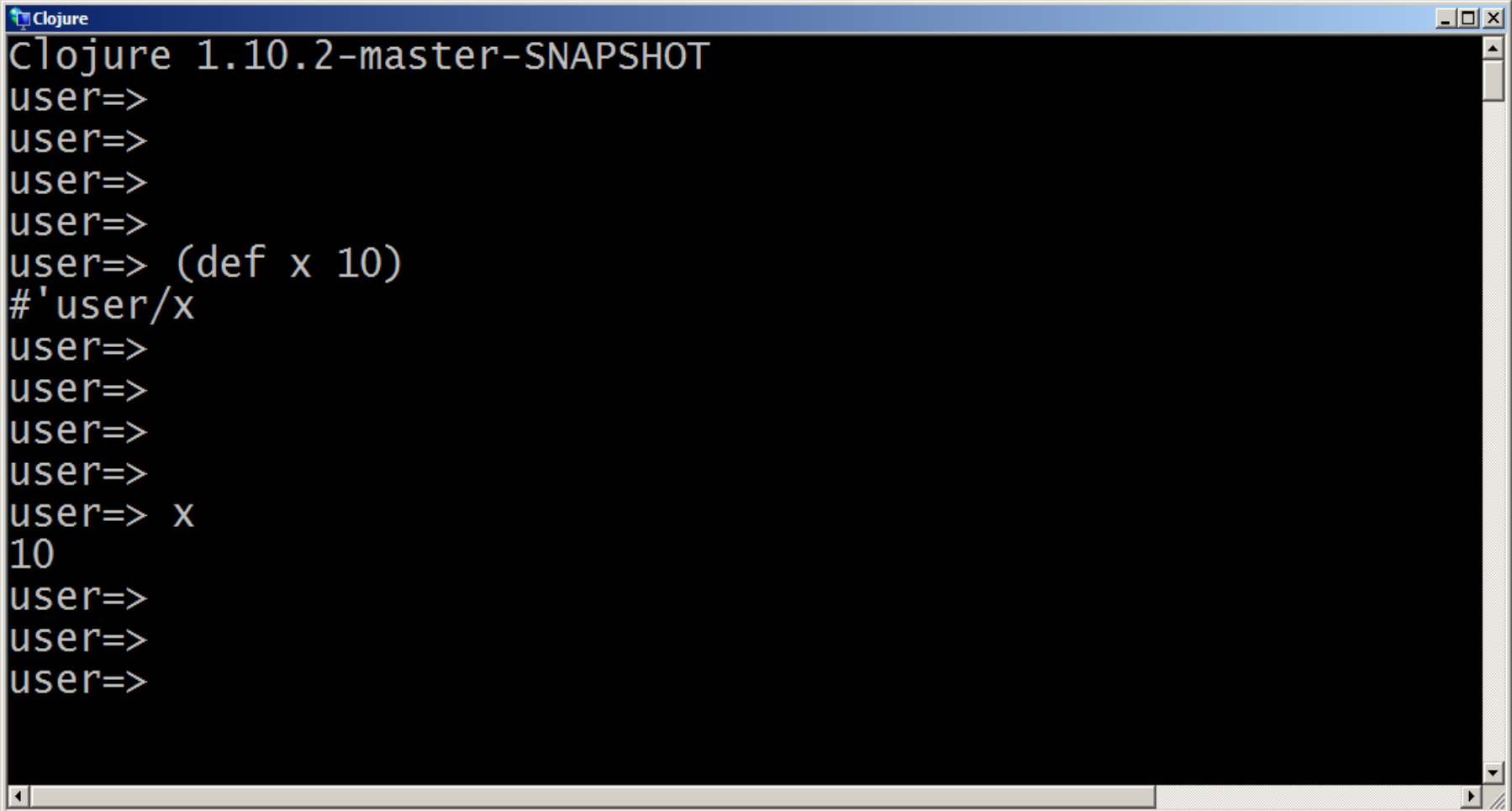


```
Clojure
user=>
user=>
user=> (def funcao-hello (fn [nome] (println (str "Hello, " nome) ) ) )
#'user/funcao-hello
user=>
user=>
user=>
user=> (def funcao-tchau (fn [nome] (println (str "Tchau, " nome) ) ) )
#'user/funcao-tchau
user=>
user=>
user=> (def saudacoes (fn [funcao nome] (funcao nome) ) )
#'user/saudacoes
user=>
user=>
user=> (saudacoes funcao-hello "Maurício")
Hello, Maurício
nil
user=>
user=> (saudacoes funcao-tchau "Maurício")
Tchau, Maurício
nil
user=>
user=>
```

Tarefa 7

- ✓ Abra o **REPL**;
- ✓ Com o emprego da special form **def**, escreva o código que efetua a ligação do símbolo `x` ao valor 10;
- ✓ Em seguida, avalie o valor do símbolo `x`;
- ✓ Encerre o **REPL**.

Tarefa 7



```
Clojure 1.10.2-master-SNAPSHOT
user=>
user=>
user=>
user=>
user=> (def x 10)
#'user/x
user=>
user=>
user=>
user=>
user=> x
10
user=>
user=>
user=>
```

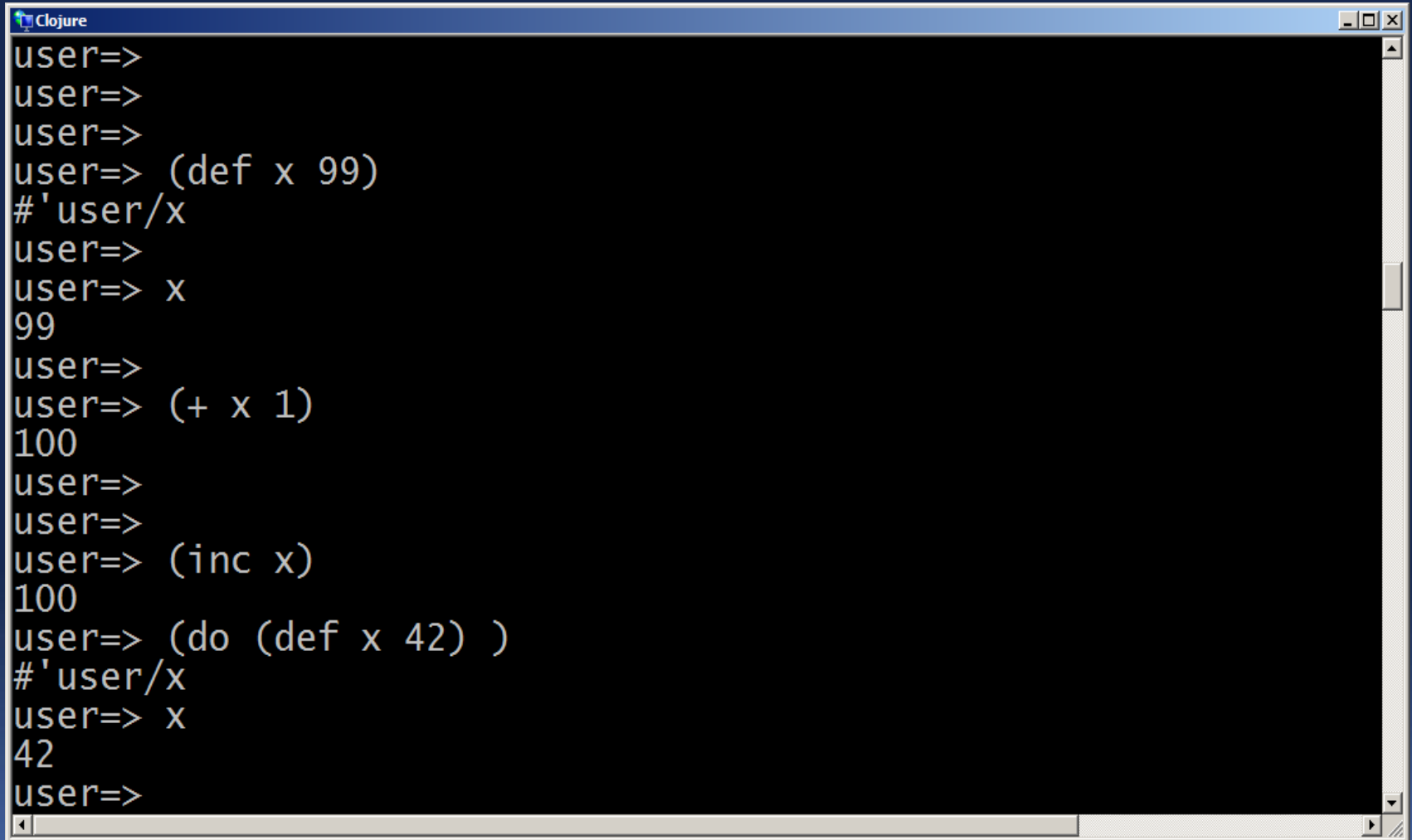
- ✓ Observação: Quando **REPL** retorna **#'user/x**, ele está retornando uma referência ao símbolo **x** (variável) criado. A parte **user** indica o **namespace** onde o símbolo **x** foi definido

Binding

- ✓ É possível redefinir um símbolo criado previamente ?
- ✓ Sim, é possível modificar-se o binding por meio de: **(def x 20)**
- ✓ Mas, **não** se recomenda redefinir-se símbolos em nossos programas uma vez que tal procedimento dificulta a leitura e a manutenção do código;
- ✓ Assim, em **Clojure** uma boa prática é considerar a operação de binding como constante.

Binding

- ✓ Uma vez definido um símbolo por meio de **binding**, pode-se utilizá-lo em avaliação de expressões e redefiní-lo ainda por **binding**;

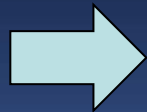


```
Clojure
user=>
user=>
user=>
user=> (def x 99)
#'user/x
user=>
user=> x
99
user=>
user=> (+ x 1)
100
user=>
user=>
user=> (inc x)
100
user=> (do (def x 42) )
#'user/x
user=> x
42
user=>
```

Binding e Escopo

- ✓ **Bindings** criados pela special form **def** têm **escopo dinâmico** e podem ser considerados como “**globais**”. Eles são, dessa forma, automaticamente **namespaced**, o qual é útil para evitar conflitos com nomes existentes;
- ✓ Para se definir bindings com escopo **local** (ou **escopo léxico**), deve-se usar a special form **let**.

y está
fora do
escopo
definido
em let !!!

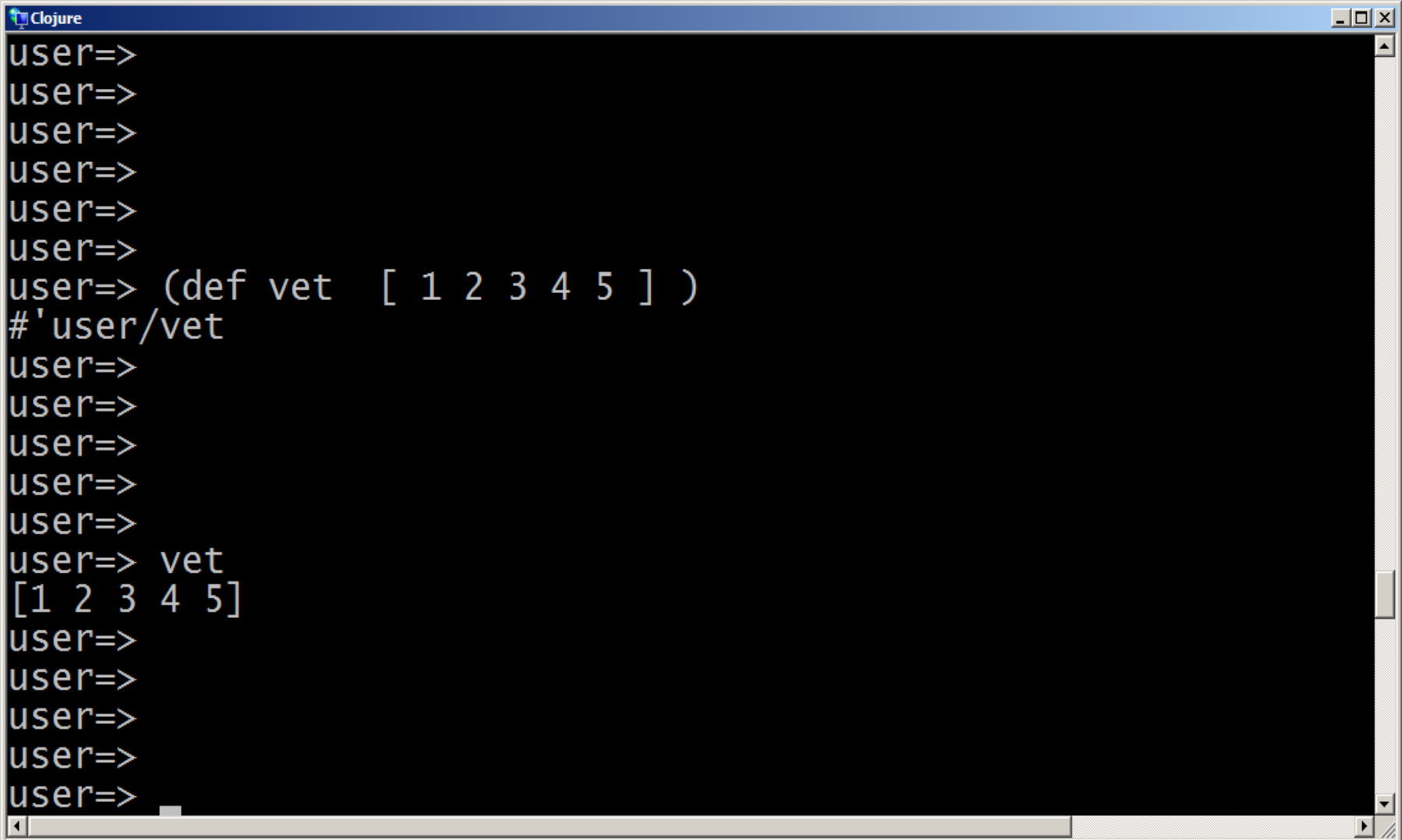


```
Clojure
user=>
user=>
user=>
user=>
user=>
user=> (let [y 3] (println y) (* 10 y) )
3
30
user=>
user=>
user=>
user=>
user=>
user=> y
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: y in this context
user=>
user=>
user=>
user=>
```

Observação let

- ✓ A special form **let** emprega um vetor como um parâmetro para criar bindings locais e uma série de expressões que serão avaliadas como se estivessem em um bloco;
- ✓ Um **vetor** é similar a uma **lista**, no sentido em que ambos correspondem a uma coleção sequencial de valores;
- ✓ Na próxima unidade veremos com mais detalhes os vetores;
- ✓ Por ora, apenas é importante saber que **vetores** podem ser criados por **colchetes**;
- ✓ Por exemplo: **[1 2 3 4 5]**

Definindo vetores



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=> (def vet [ 1 2 3 4 5 ] )
#'user/vet
user=>
user=>
user=>
user=>
user=>
user=> vet
[1 2 3 4 5]
user=>
user=>
user=>
user=>
user=>
```

Escopos Global e Local



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (def x 99)
#'user/x
user=> x
99
user=> (let [x 5] (println x) )
5
nil
user=> x
99
user=>
user=>
user=>
user=>
```

Macro let – Escopo local

Escopo Local



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (let [x 10 y 99] (str "x=" x " y=" y) )
"x=10 y=99"
user=>
user=>
user=>
user=>
user=> x
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: x in this context
user=>
user=>
user=>
user=> y
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: y in this context
user=>
user=>
```

Macro let – Escopo local

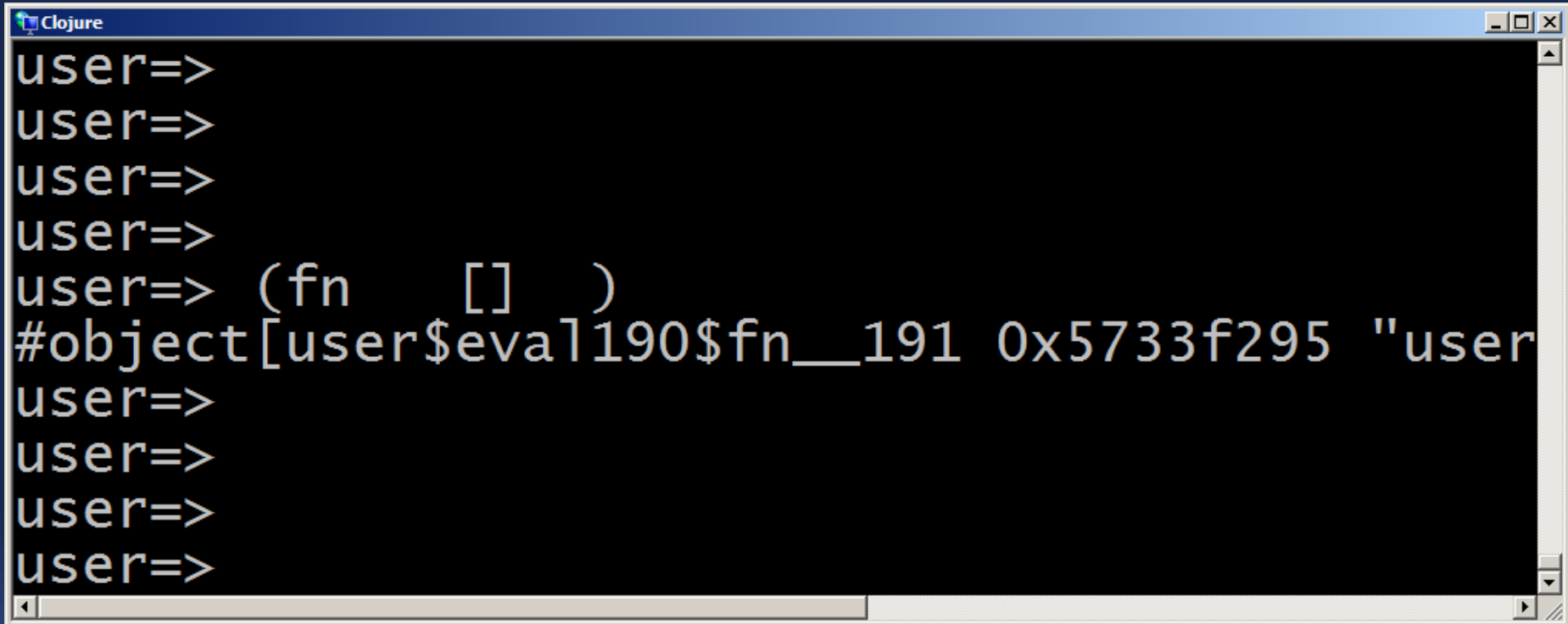
```
Clojure
user=>
user=>
user=>
user=>
user=> (def mensagem "olá a todos")
#'user/mensagem
user=>
user=>
user=>
user=> ( let [ x (* 3 10) y 20 z 4] (println mensagem) (+ x y z ) )
Olá a todos
54
user=>
user=>
user=>
user=>
user=> x
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: x in this context
user=> y
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: y in this context
user=> z
Syntax error compiling at (REPL:0:0).
Unable to resolve symbol: z in this context
user=> _
```

Tarefa 8

- ✓ Abra o **REPL**;
- ✓ Com a **macro** **fn** defina uma função que **não** recebe argumentos e que também **nada** faz;
- ✓ Encerre o **REPL**.

Atividade 8

(fn [])

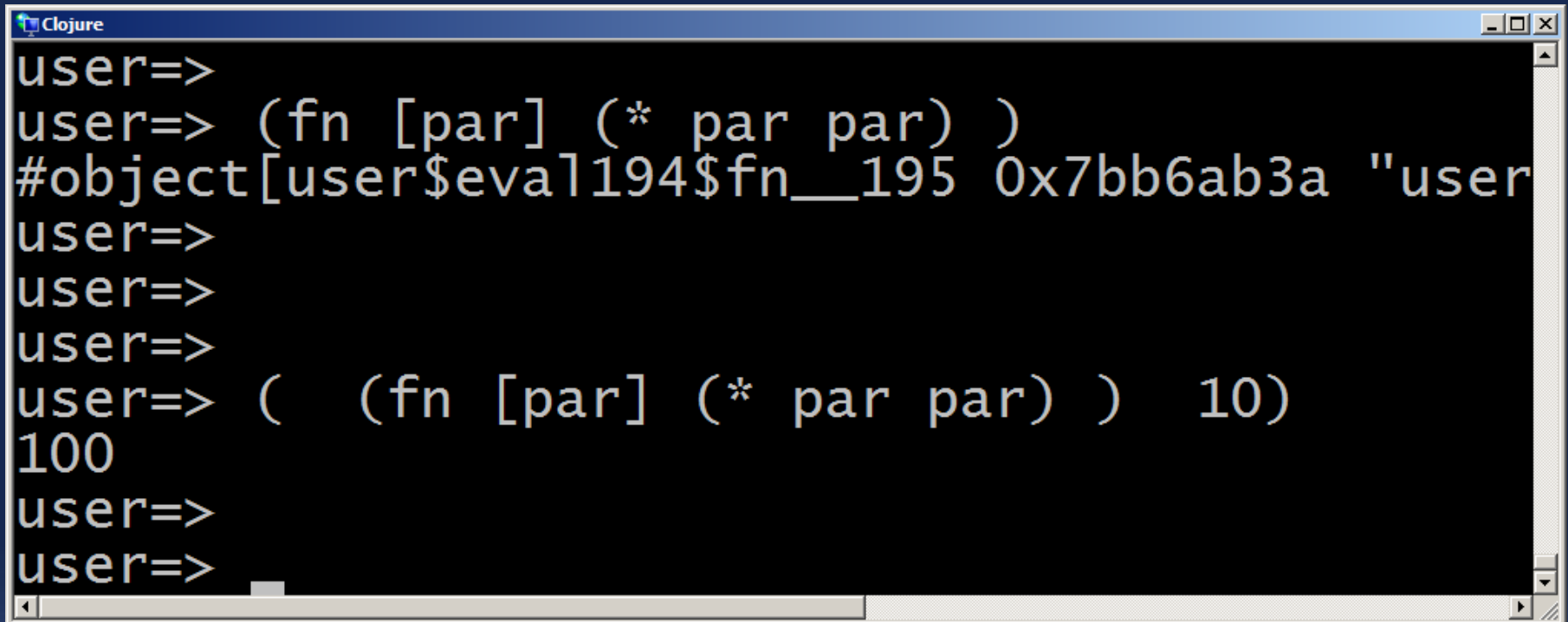


```
Clojure
user=>
user=>
user=>
user=>
user=> (fn [] )
#object[user$eval1190$fn__191 0x5733f295 "user
user=>
user=>
user=>
user=>
```

Tarefa 9

- ✓ Abra o **REPL**;
- ✓ Com a **macro** **fn** defina uma função que recebe um parâmetro chamado **par** e retorna seu valor ao **quadrado** (**par** multiplicado por ele mesmo)
- ✓ Encerre o **REPL**.

Tarefa 9

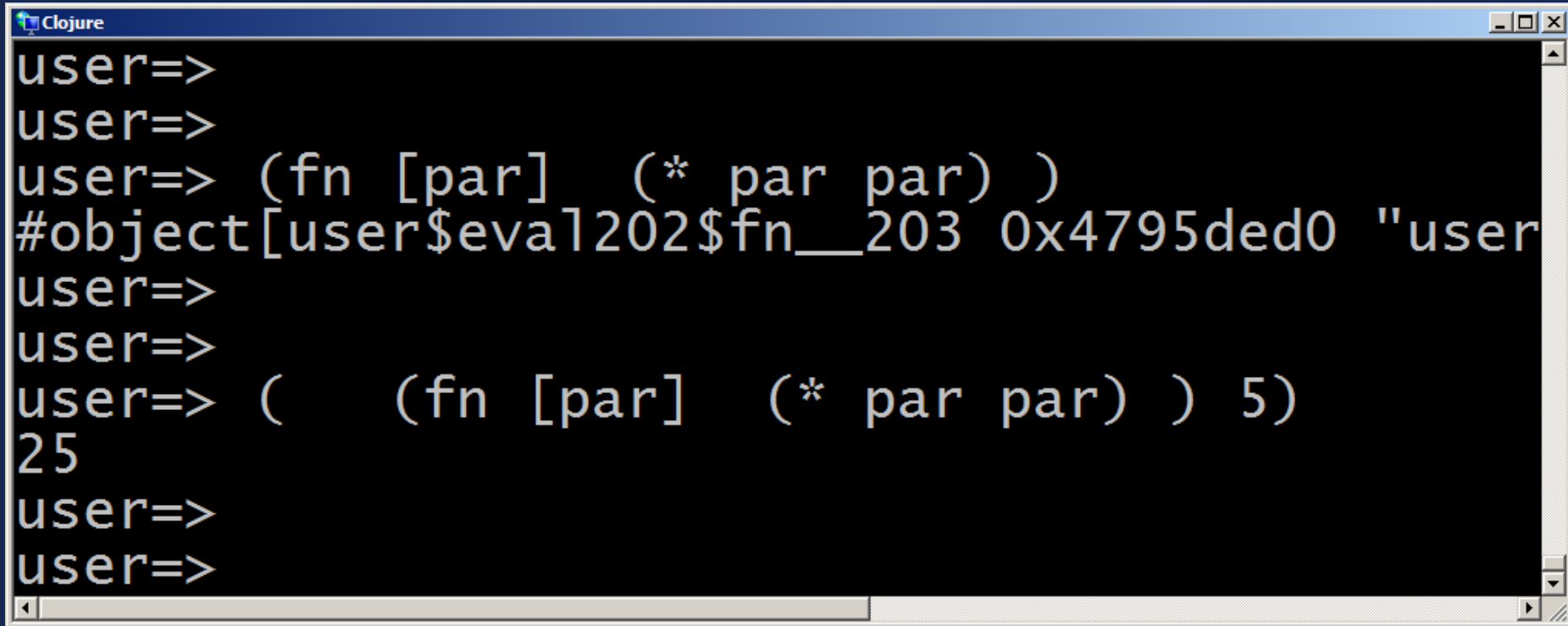


```
Clojure
user=>
user=> (fn [par] (* par par) )
#object[user$eva1194$fn__195 0x7bb6ab3a "user"
user=>
user=>
user=>
user=> ( (fn [par] (* par par) ) 10)
100
user=>
user=>
```

Tarefa 10

- ✓ Abra o **REPL**;
- ✓ Com a **macro** **fn** defina uma função que recebe um parâmetro chamado **par** e retorna seu valor ao **quadrado** (**par** multiplicado por ele mesmo).
- ✓ Após a definição da função anônima (com **fn**) efetuar a aplicação da função passando a ela um argumento com o valor 5.
- ✓ Encerre o **REPL**.

Tarefa 10



```
Clojure
user=>
user=>
user=> (fn [par] (* par par) )
#object[user$eva1202$fn__203 0x4795ded0 "user"
user=>
user=>
user=> ( (fn [par] (* par par) ) 5)
25
user=>
user=>
```

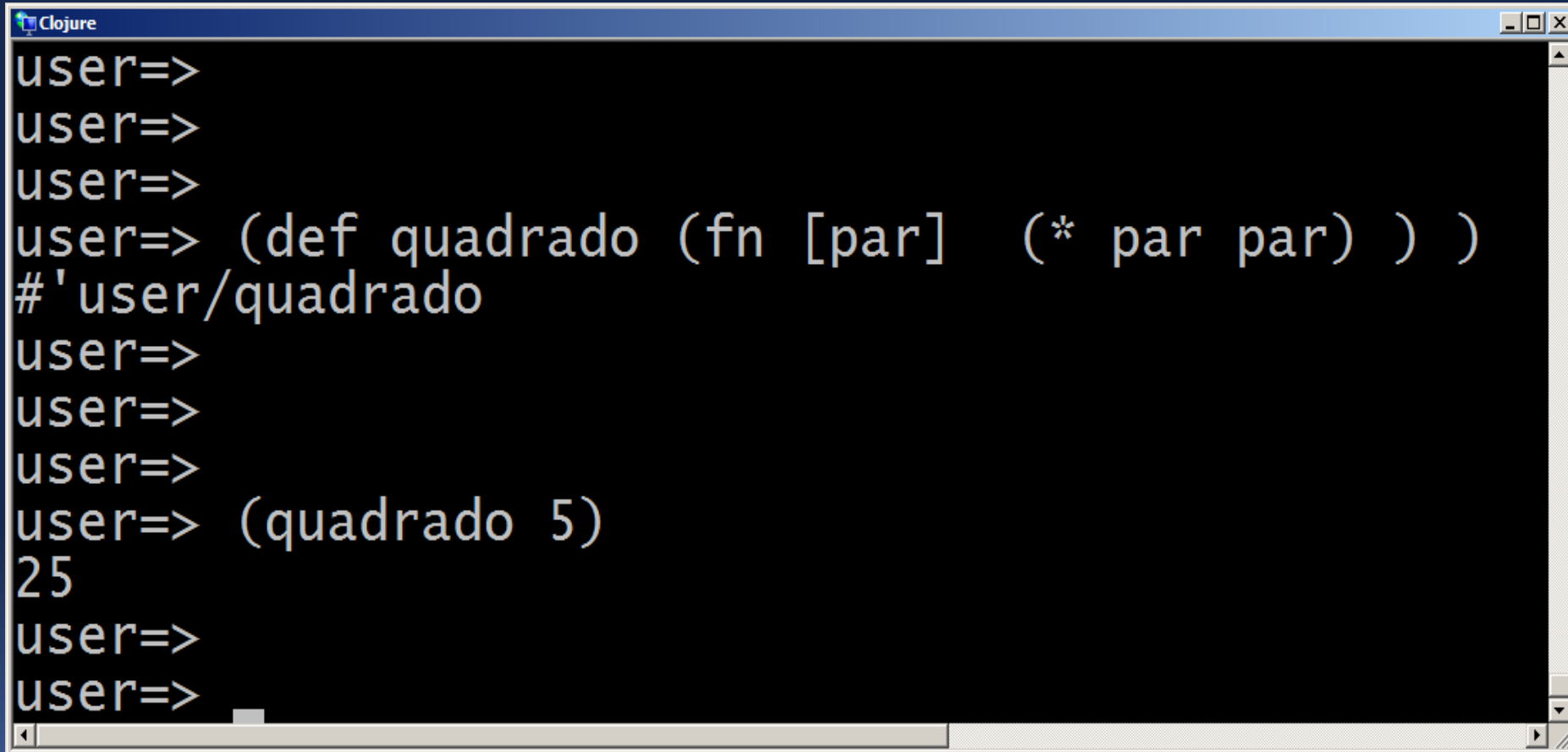
Observação: Nessa atividade, definimos uma função anônima e junto com a definição fizemos uma aplicação da função (execução) passando a ela um argumento com o valor 5. A função anônima retornou o quadrado de 5 que é 25.

Tarefa 11

- ✓ Abra o **REPL**;
- ✓ Com a **macro** **fn** desenvolvida na atividade anterior, foi possível definirmos uma função de forma anônima e executá-la por meio da aplicação da função com o argumento passado, no caso 5.
- ✓ Mas, e se quisermos reaproveitar essa função ? Ou seja, reusá-la em outra ocasião ! Para esse caso, precisaríamos associar à essa função um nome que permitisse chamá-la em outra ocasião. Repita a atividade de forma que a função deixe de ser anônima e passe assim a ter um nome.
- ✓ Encerre o **REPL**.

Dica: Nessa atividade, para definirmos função com nome devemos usar a macro `def`.

Tarefa 11

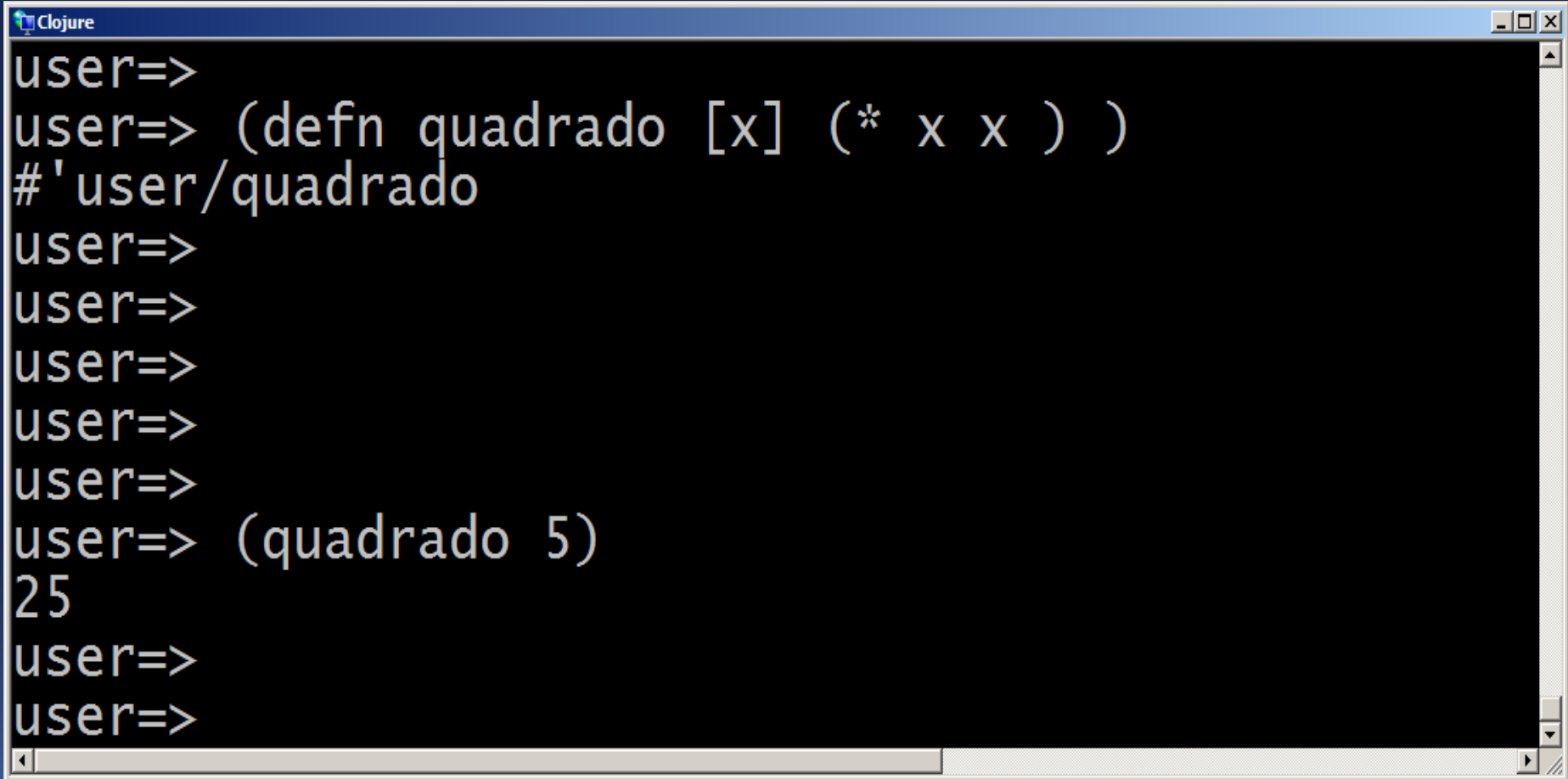
A screenshot of a Clojure REPL window. The window has a title bar that says "Clojure". The background is black with white text. The text shows a series of REPL interactions: "user=>" followed by three more "user=>" prompts, then a definition "(def quadrado (fn [par] (* par par)))", followed by "#'user/quadrado", then three more "user=>" prompts, then a function call "(quadrado 5)", and finally the result "25". The window has standard OS window controls (minimize, maximize, close) in the top right and a scrollbar on the right side.

```
Clojure
user=>
user=>
user=>
user=> (def quadrado (fn [par] (* par par) ) )
#'user/quadrado
user=>
user=>
user=>
user=> (quadrado 5)
25
user=>
user=>
```

Macro `defn`

- ✓ Vimos nas atividades anteriores que com a special form `fn` podemos criar funções anônimas.
- ✓ Vimos também que podemos retirar da função essa característica anônima, ou seja deixá-la reusável por meio da special form `def`.
- ✓ Essa combinação de `def` com `fn` é portanto bem usual;
- ✓ Em função disso, `Clojure` disponibiliza uma macro `built-in` para essa finalidade.
- ✓ Trata-se da macro `defn`.

Exemplo – Macro defn



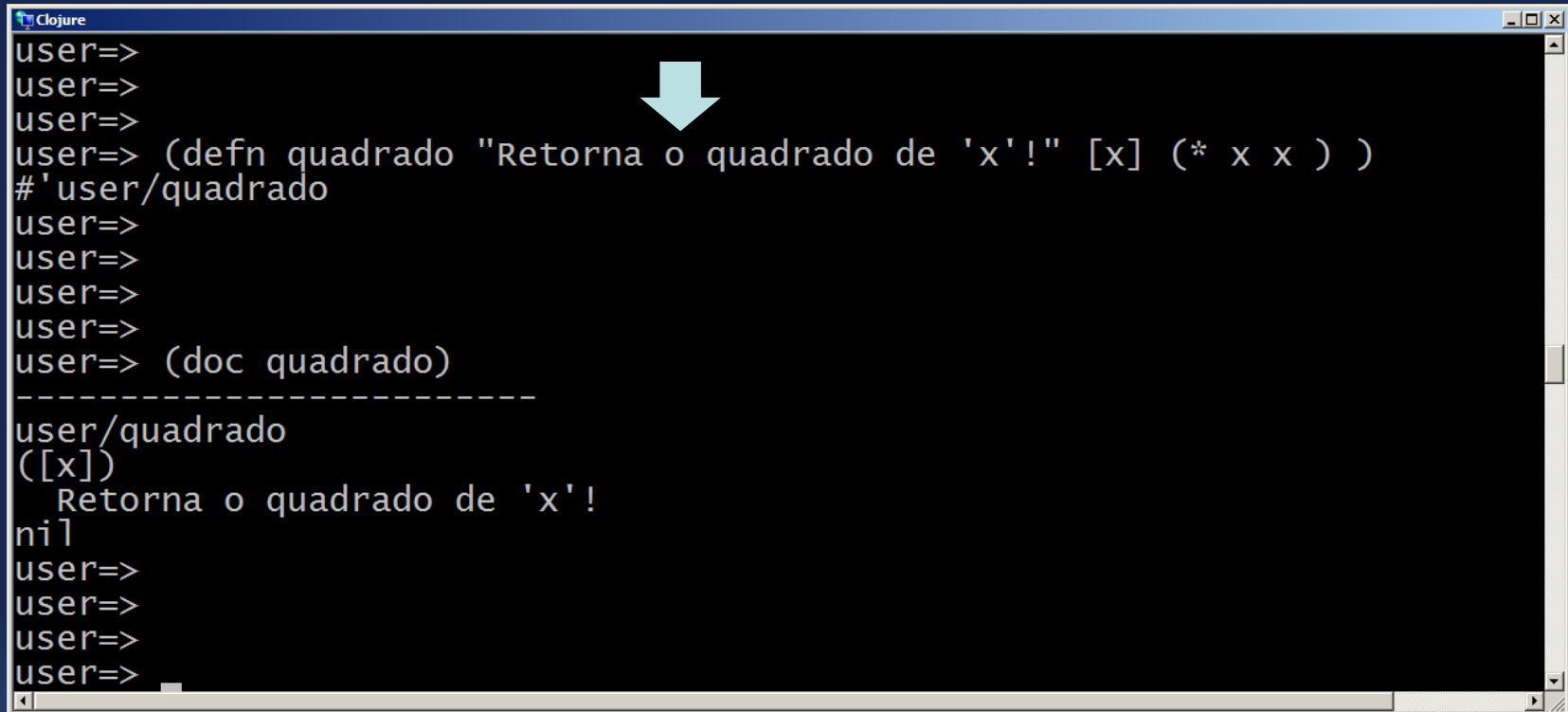
```
Clojure
user=>
user=> (defn quadrado [x] (* x x) )
#'user/quadrado
user=>
user=>
user=>
user=>
user=>
user=> (quadrado 5)
25
user=>
user=>
```

Exemplo – Macro defn

```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (defn funcao-hello [nome] (println (str "Hello, " nome) ) )
#'user/funcao-hello
user=>
user=>
user=>
user=>
user=> (funcao-hello "Maurício")
Hello, Maurício
nil
user=>
user=>
user=>
```

Adicionando documentação à função

- ✓ Para adicionar alguma documentação em uma função definida com a macro **defn**, basta acrescentar um **doc-string** antes dos argumentos da função !



```
Clojure
user=>
user=>
user=>
user=> (defn quadrado "Retorna o quadrado de 'x'!" [x] (* x x) )
#'user/quadrado
user=>
user=>
user=>
user=>
user=> (doc quadrado)
-----
user/quadrado
([x])
  Retorna o quadrado de 'x'!
nil
user=>
user=>
user=>
user=>
```

Observação: O doc-string deve vir antes da definição dos parâmetros da função. Se for escrito após, será avaliado sequencialmente como parte do corpo da função, resultando naturalmente em erro!

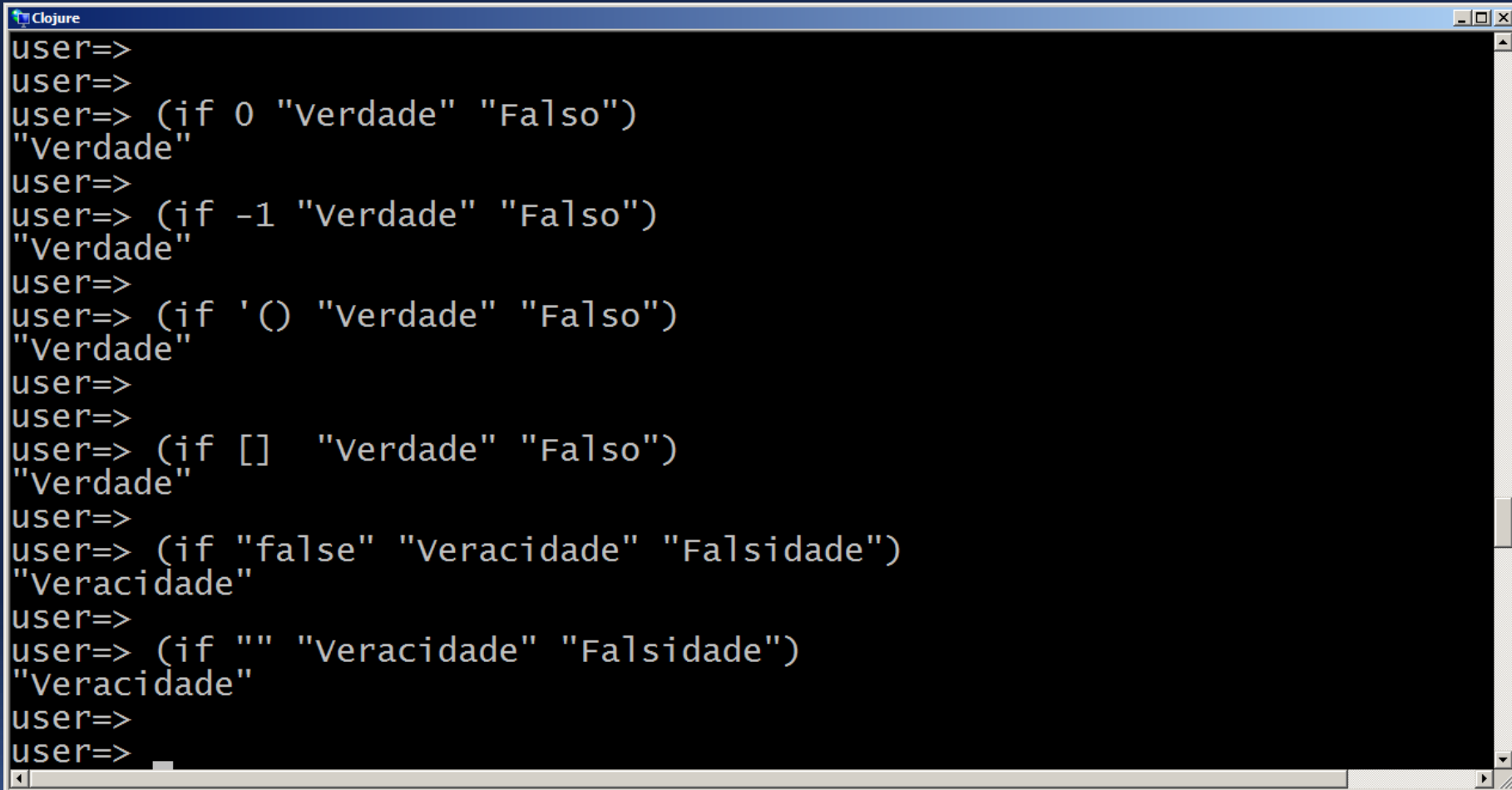
Veracidade, Falsidade e nil

- ✓ Na linguagem **Clojure**, **nil** representa ausência de valor; Corresponde à **NULL** em outras linguagens de programação;
- ✓ Representar ausência de valor é útil, pois permitir saber que "algo" está faltando;
- ✓ Em **Clojure**, **nil** se comporta como "**false**" quando for avaliado em uma expressão booleana;
- ✓ Em **Clojure**, **false** e **nil** são os **únicos** valores que são tratados como "**falsidade**". Tudo o mais é verdadeiro. Essa simples regra torna código **Clojure** mais robusto e confiável.

Veracidade, Falsidade e nil – Exemplos

```
user=>  
user=> (if nil "Veracidade" "Falsidade")  
"Falsidade"  
user=>  
user=> (if false "Veracidade" "Falsidade")  
"Falsidade"  
user=>  
user=>
```

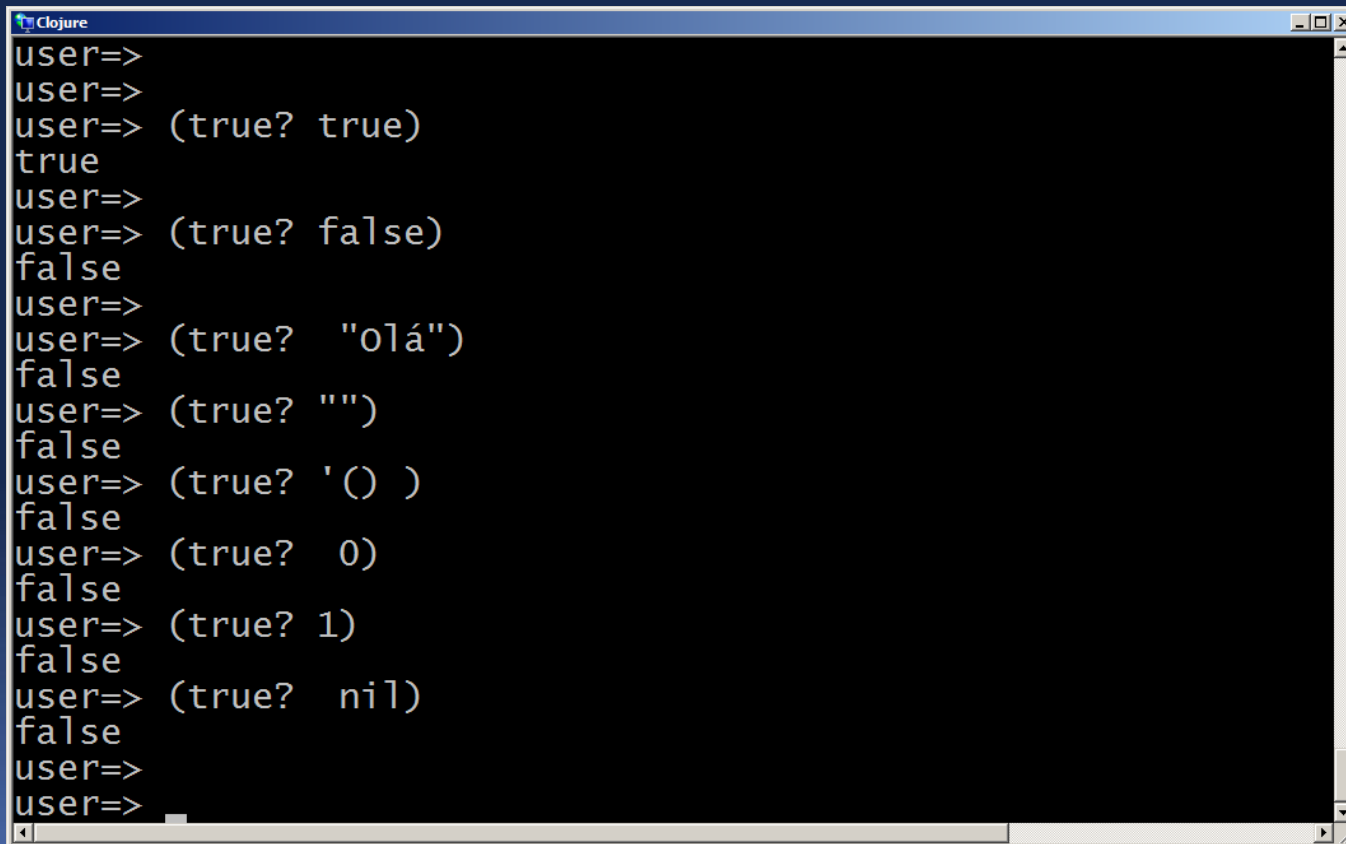
Veracidade, Falsidade e nil – Exemplos



```
Clojure
user=>
user=>
user=> (if 0 "Verdade" "Falso")
"Verdade"
user=>
user=> (if -1 "Verdade" "Falso")
"Verdade"
user=>
user=> (if '() "Verdade" "Falso")
"Verdade"
user=>
user=>
user=> (if [] "Verdade" "Falso")
"Verdade"
user=>
user=> (if "false" "Veracidade" "Falsidade")
"Veracidade"
user=>
user=> (if "" "Veracidade" "Falsidade")
"Veracidade"
user=>
user=>
```

Função true?

- ✓ A função **true?** retorna um valor booleano **true** ou **false** e permite checar se o parâmetro é **exatamente true** e não simplesmente tratado como **veracidade** ou **falsidade**.
- ✓ Assim, **true?** retorna true apenas quando o parâmetro for verdadeiramente **true**; Do contrário, sempre retornará **false**.



```
Clojure
user=>
user=>
user=> (true? true)
true
user=>
user=> (true? false)
false
user=>
user=> (true? "olá")
false
user=>
user=> (true? "")
false
user=>
user=> (true? '() )
false
user=>
user=> (true? 0)
false
user=>
user=> (true? 1)
false
user=>
user=> (true? nil)
false
user=>
user=>
```

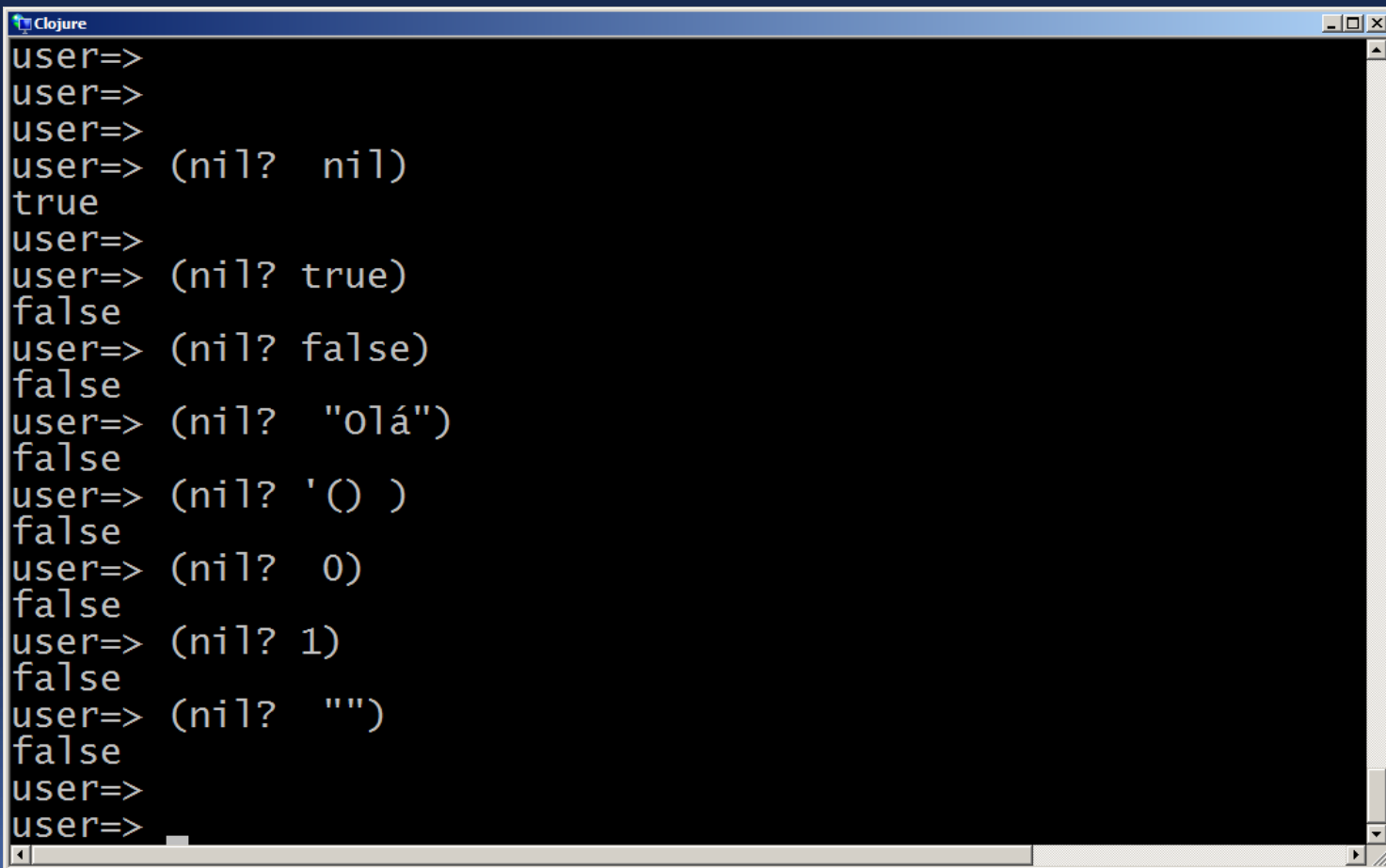
Função `false?`

- ✓ A função `false?` retorna um valor booleano `true` ou `false` e permite checar se o parâmetro é **exatamente false** e não simplesmente tratado como **veracidade** ou **falsidade**.
- ✓ Assim, `false?` retorna `true` apenas quando o parâmetro for verdadeiramente `false`; Do contrário, sempre retornará `false`.

```
user=>  
user=>  
user=> (false? false)  
true  
user=> (false? true)  
false  
user=> (false? "olá")  
false  
user=> (false? '() )  
false  
user=> (false? 0)  
false  
user=> (false? nil)  
false  
user=> (false? "")  
false  
user=>  
user=>
```

Função nil?

- ✓ A função **nil?** retorna um valor booleano **true** ou **false** e permite checar se o parâmetro é **exatamente nil** e não simplesmente tratado como **veracidade** ou **falsidade**.
- ✓ Assim, **nil?** retorna true apenas quando o parâmetro for verdadeiramente **nil**; Do contrário, sempre retornará **false**.



```
Clojure
user=>
user=>
user=>
user=> (nil? nil)
true
user=>
user=> (nil? true)
false
user=> (nil? false)
false
user=> (nil? "olá")
false
user=> (nil? '() )
false
user=> (nil? 0)
false
user=> (nil? 1)
false
user=> (nil? "")
false
user=>
user=>
```

Exemplo interessante

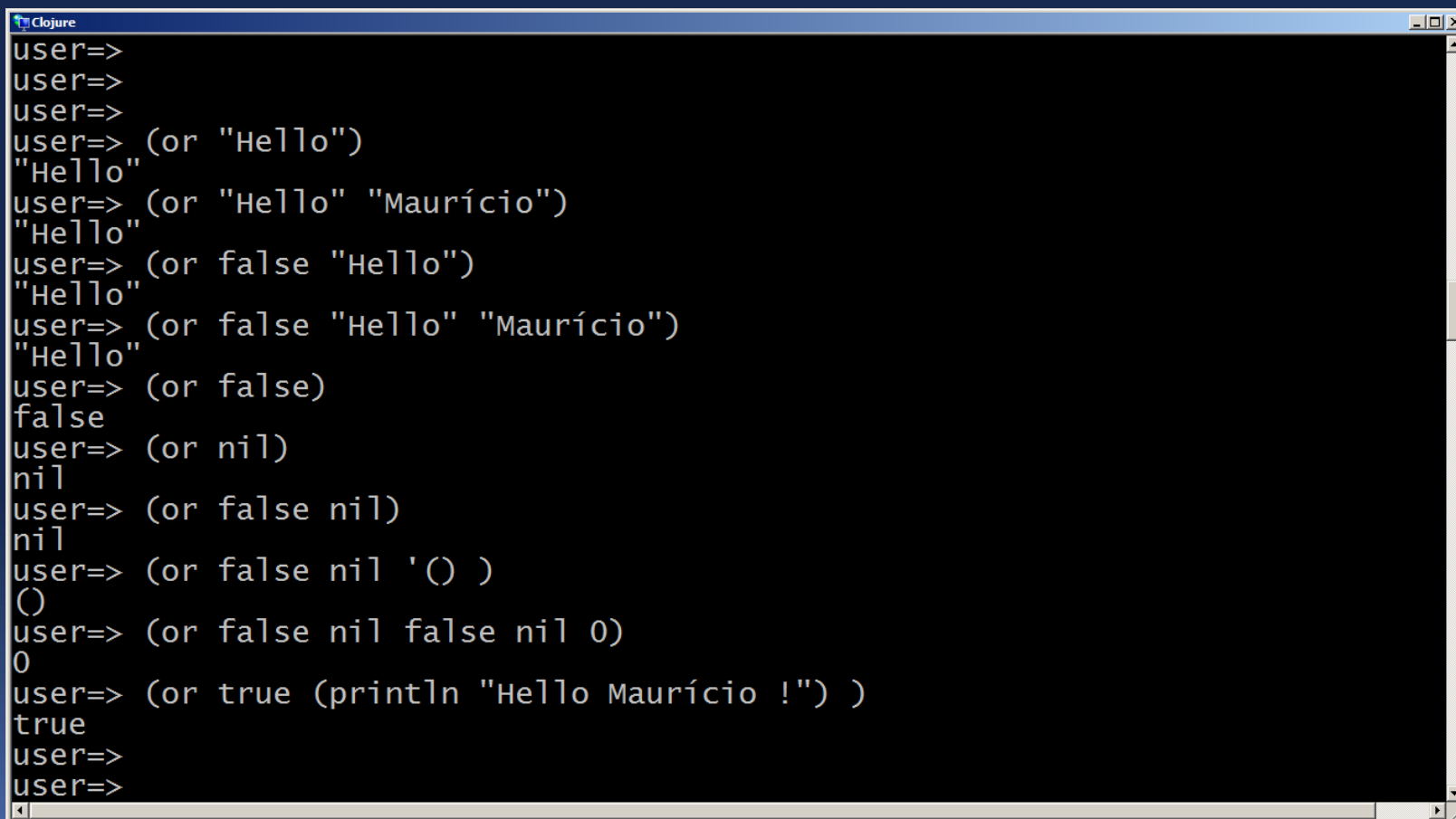
```
user=>  
user=> (println "Olá  Maurício !")  
Olá  Maurício !  
nil  
user=>  
user=>  
user=> (nil?  (println "Olá  Maurício !") )  
Olá  Maurício !  
true  
user=>
```

Macro and

- ✓ A macro **and** retorna a primeira **falsidade** que encontrar (da esquerda para a direita) e **não** avaliará o restante da expressão quando for o caso;
- ✓ Quando todos os valores passados para a macro **and** resultarem em “veracidade”, a macro **and** retorna o valor da última expressão.

```
Clojure
user=>
user=>
user=>
user=>
user=> (and "Hello Maurício !!!")
"Hello Maurício !!!"
user=>
user=> (and "Hello" "Maurício")
"Maurício"
user=>
user=> (and false "Hello" "Maurício")
false
user=>
user=> (and "Hello" false "Maurício")
false
user=>
user=> (and nil "Hello" "Maurício")
nil
user=>
user=> (and (println "Hello Maurício") (println "Você está bem? ") )
Hello Maurício
nil
user=>
user=>
user=>
```

- ✓ A macro **or** retorna a primeira **veracidade** que encontrar (da esquerda para a direita) e **não** avaliará o restante da expressão quando for o caso;
- ✓ Quando todos os valores passados para a macro **or** resultarem em “falsidade”, a macro **or** retorna false ou nil, dependendo do caso.



```
Clojure
user=>
user=>
user=>
user=> (or "Hello")
"Hello"
user=> (or "Hello" "Maurício")
"Hello"
user=> (or false "Hello")
"Hello"
user=> (or false "Hello" "Maurício")
"Hello"
user=> (or false)
false
user=> (or nil)
nil
user=> (or false nil)
nil
user=> (or false nil '() )
()
user=> (or false nil false nil 0)
0
user=> (or true (println "Hello Maurício !") )
true
user=>
user=>
```

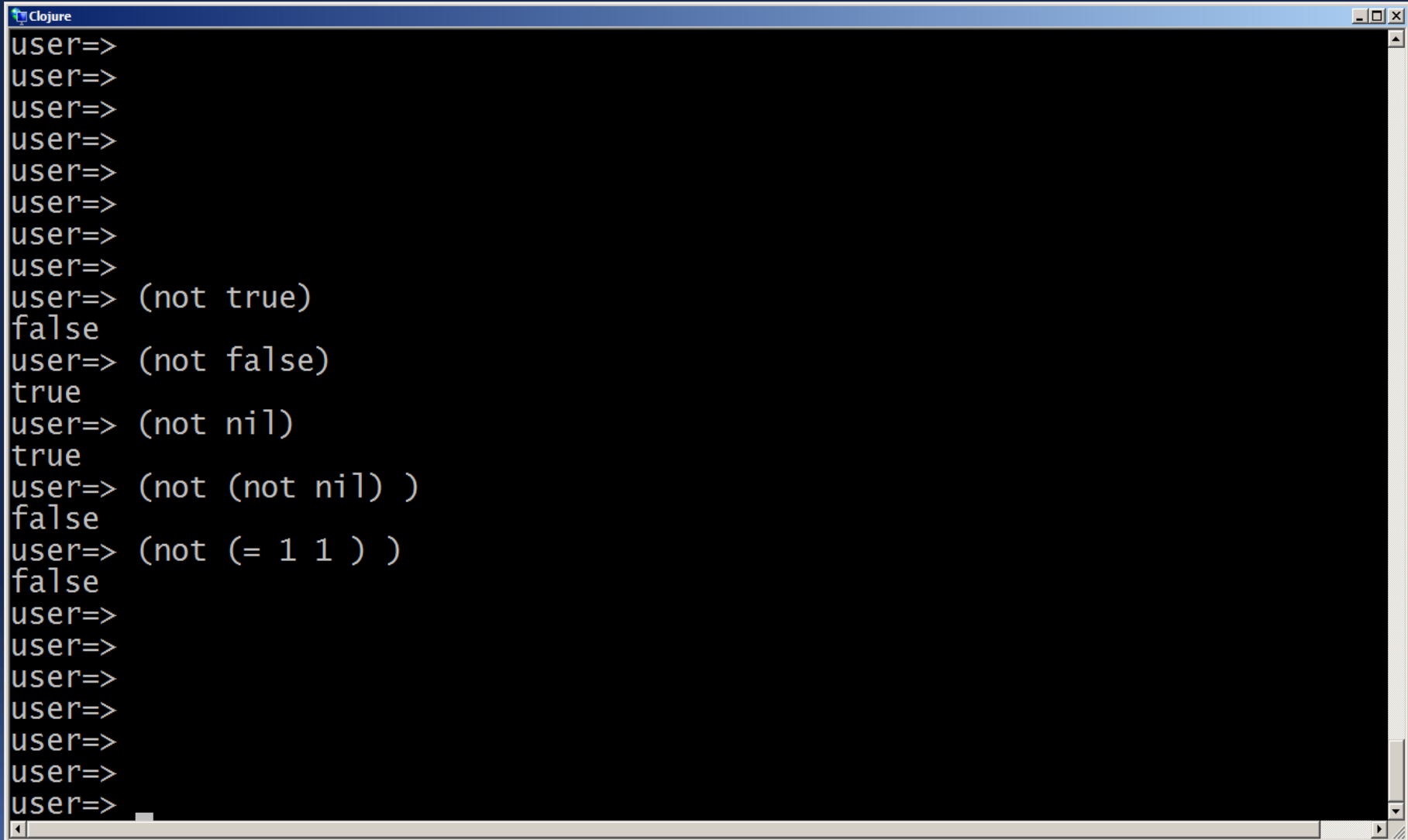
Operações de Igualdade

- ✓ Na maioria das linguagens de programação o símbolo **=** é usado para operações de atribuição. Porém em **Clojure**, bindings de símbolos à valores são feitos pelas special forms **def** e **defn**;
- ✓ Em **Clojure**, o símbolo **=** está associado à uma função para igualdade e retorna **true** se todos os argumentos forem **iguais**.
- ✓ Todas as outras operações de igualdade (**>** , **>=** , **<** , **<=**) são também implementações de funções.

Operações de Igualdade

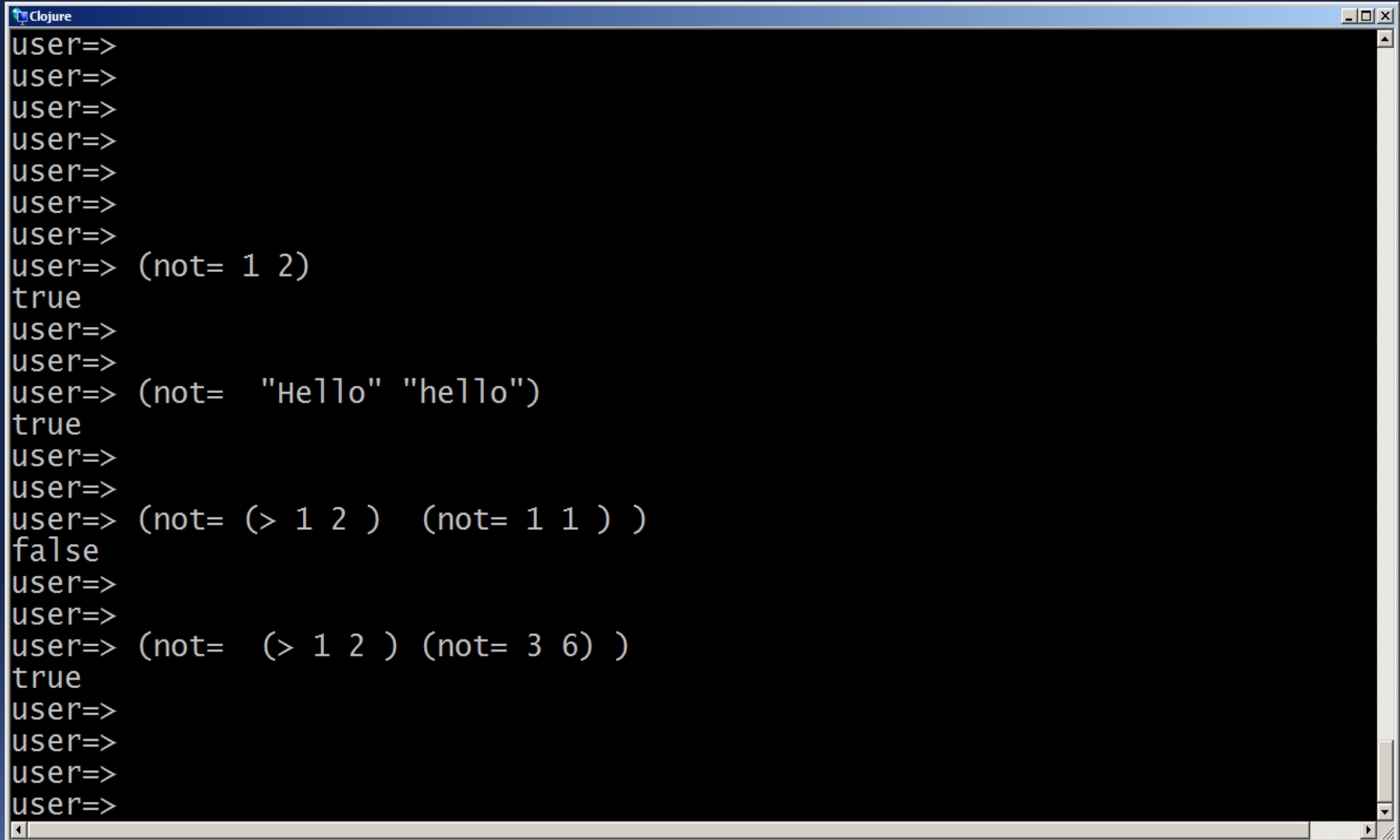
```
Clojure
user=>
user=> (= 1 1 )
true
user=> (= 2 4)
false
user=> (= 1 1 1 )
true
user=> (= 1 1 1 1 2)
false
user=> (not= 1 2 )
true
user=> (not= "Hello" "Hello")
false
user=> (not= "Hello" "hello")
true
user=> (= nil nil)
true
user=> (= nil false)
false
user=> (= nil true)
false
user=>
user=> (< 1 2 )
true
user=>
```

Operações de Igualdade



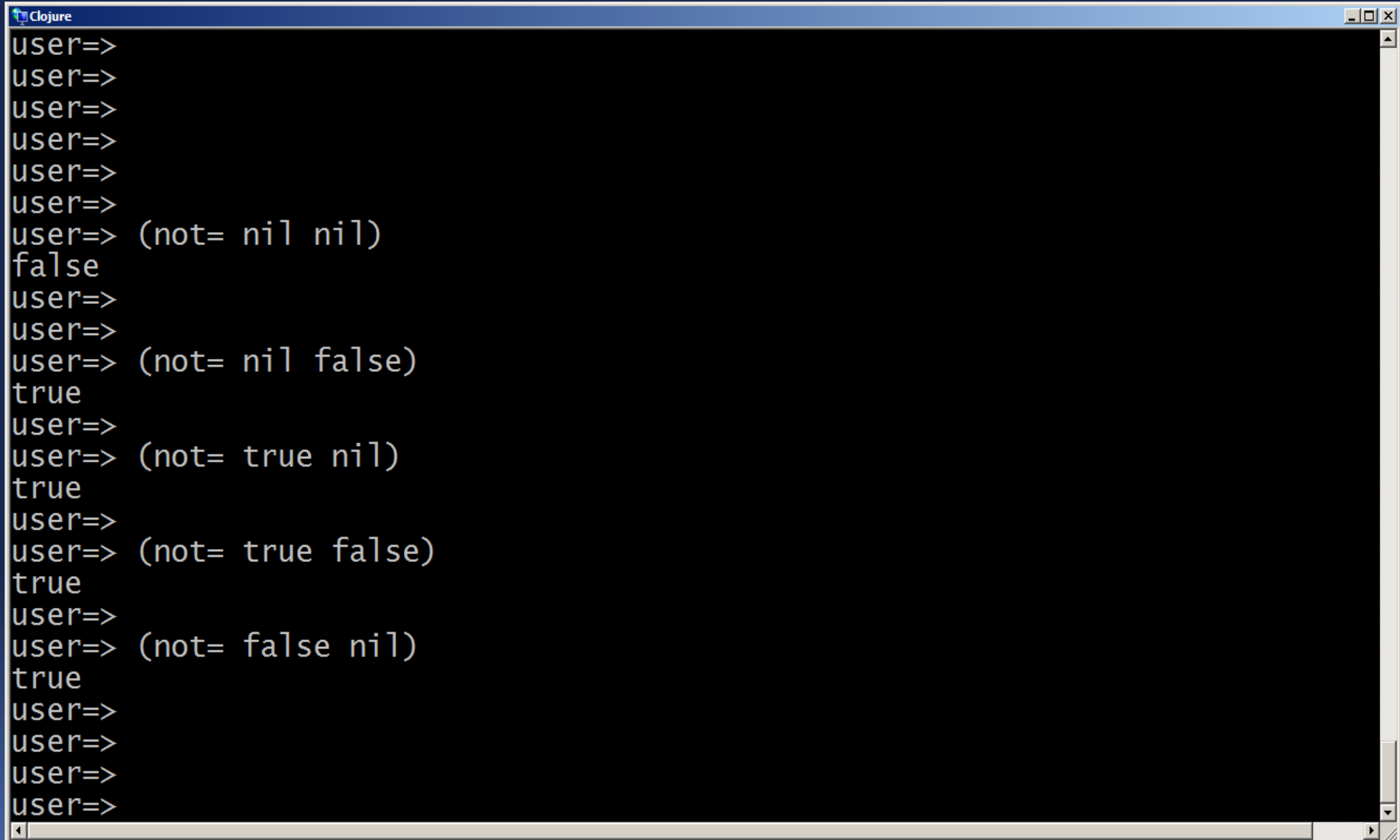
```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (not true)
false
user=> (not false)
true
user=> (not nil)
true
user=> (not (not nil) )
false
user=> (not (= 1 1) )
false
user=>
user=>
user=>
user=>
user=>
user=>
```

Operações de Igualdade



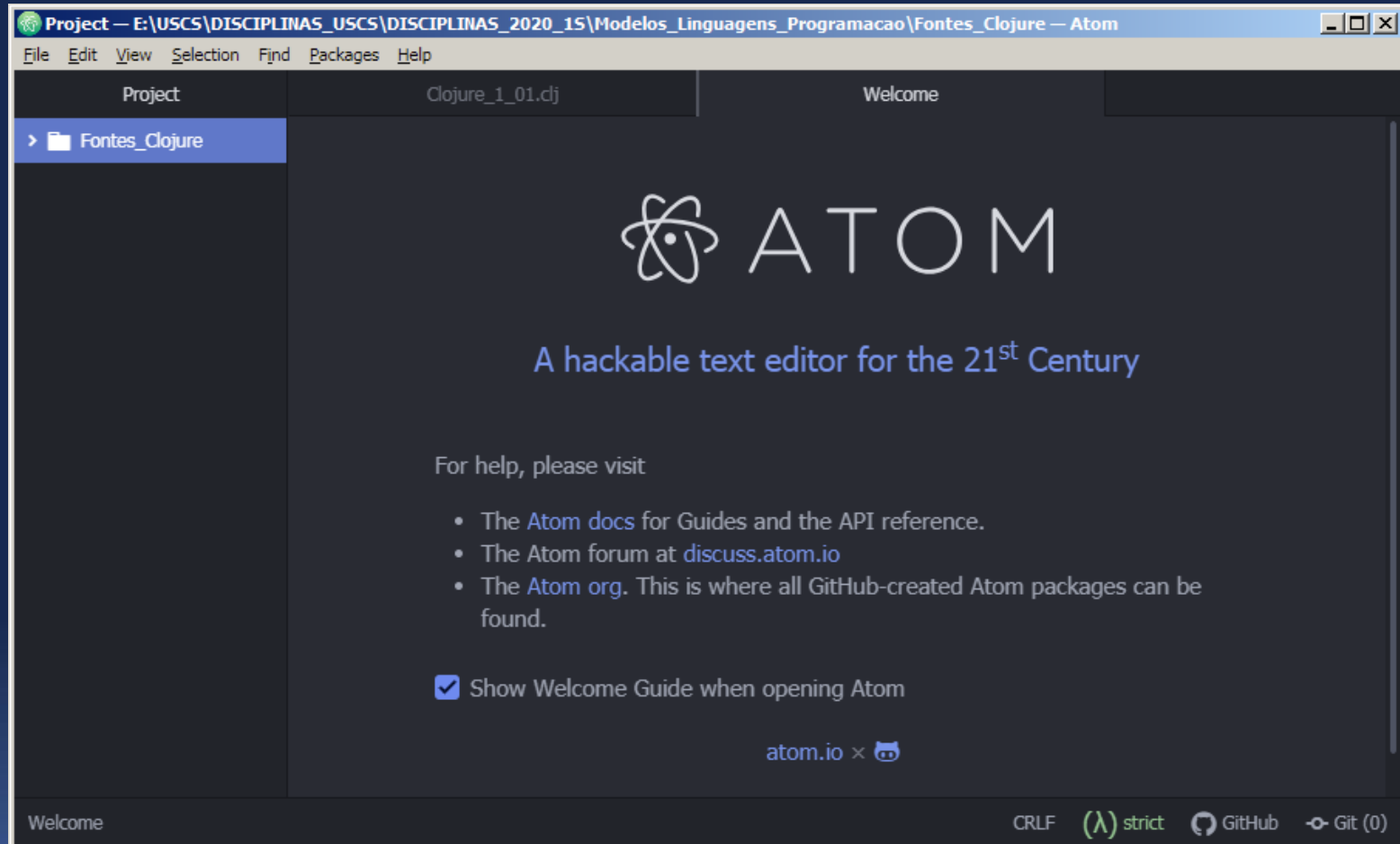
```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=>
user=> (not= 1 2)
true
user=>
user=>
user=> (not= "Hello" "hello")
true
user=>
user=>
user=> (not= (> 1 2) (not= 1 1) )
false
user=>
user=>
user=> (not= (> 1 2) (not= 3 6) )
true
user=>
user=>
user=>
user=>
```

Operações de Igualdade



```
Clojure
user=>
user=>
user=>
user=>
user=>
user=>
user=> (not= nil nil)
false
user=>
user=>
user=> (not= nil false)
true
user=>
user=> (not= true nil)
true
user=>
user=> (not= true false)
true
user=>
user=> (not= false nil)
true
user=>
user=>
user=>
user=>
```

Execução de Código Clojure no Atom Package Chlorine

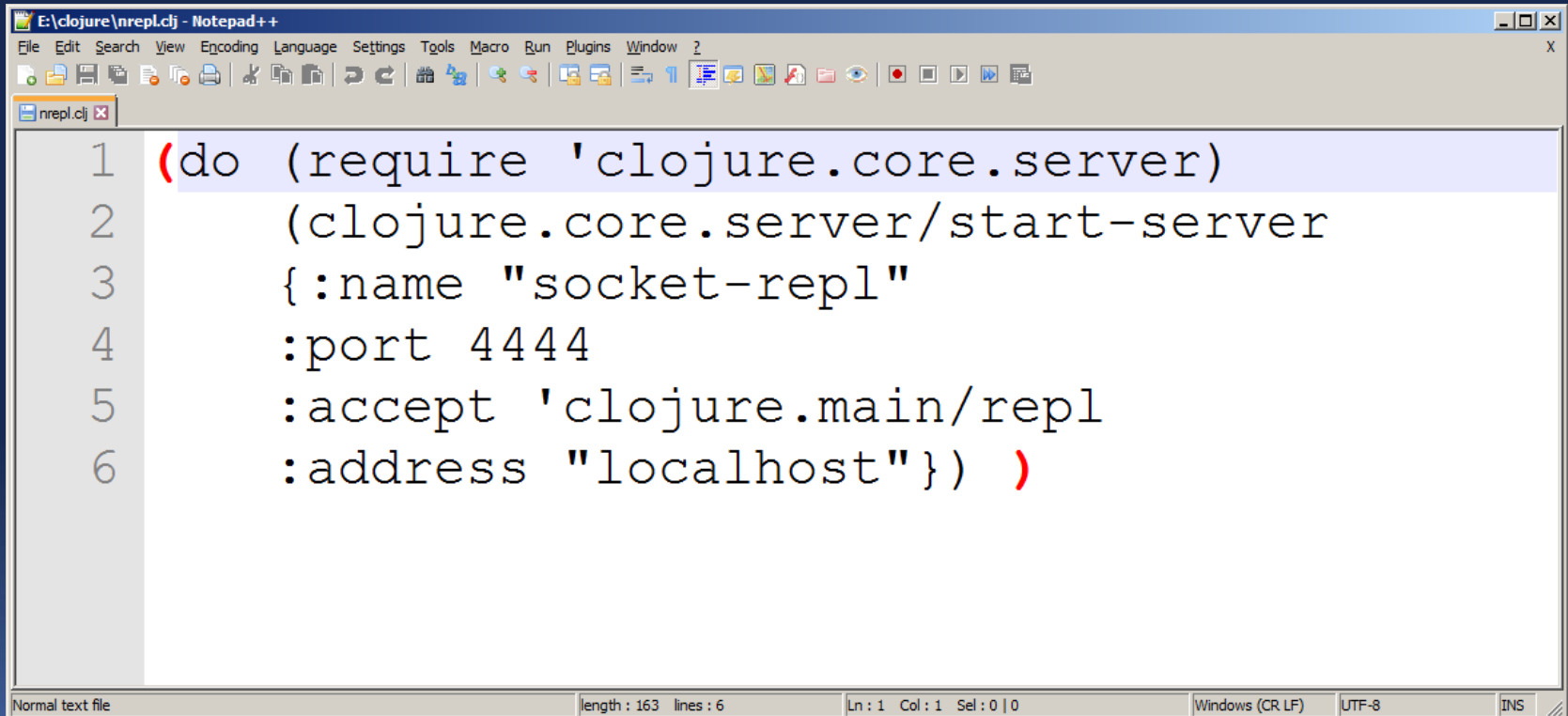


Ativando REPL Remoto

```
(do (require 'clojure.core.server)
    (clojure.core.server/start-server
     {:name "socket-repl"
      :port 4444
      :accept 'clojure.main/repl
      :address "localhost"}))
```

Salvando código num file .clj

- ✓ Criar arquivo .clj com código
- ✓ Estamos chamando o arquivo de nrepl.clj
- ✓ Será salvo no diretório: E:/clojure



```
E:\clojure\nrepl.clj - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
nrepl.clj
1 (do (require 'clojure.core.server)
2     (clojure.core.server/start-server
3       { :name "socket-repl"
4         :port 4444
5         :accept 'clojure.main/repl
6         :address "localhost" }) )
```

Normal text file | length : 163 lines : 6 | Ln : 1 Col : 1 Sel : 0 | 0 | Windows (CR LF) | UTF-8 | INS

Ativando REPL Remoto

Digitando-se o código na console

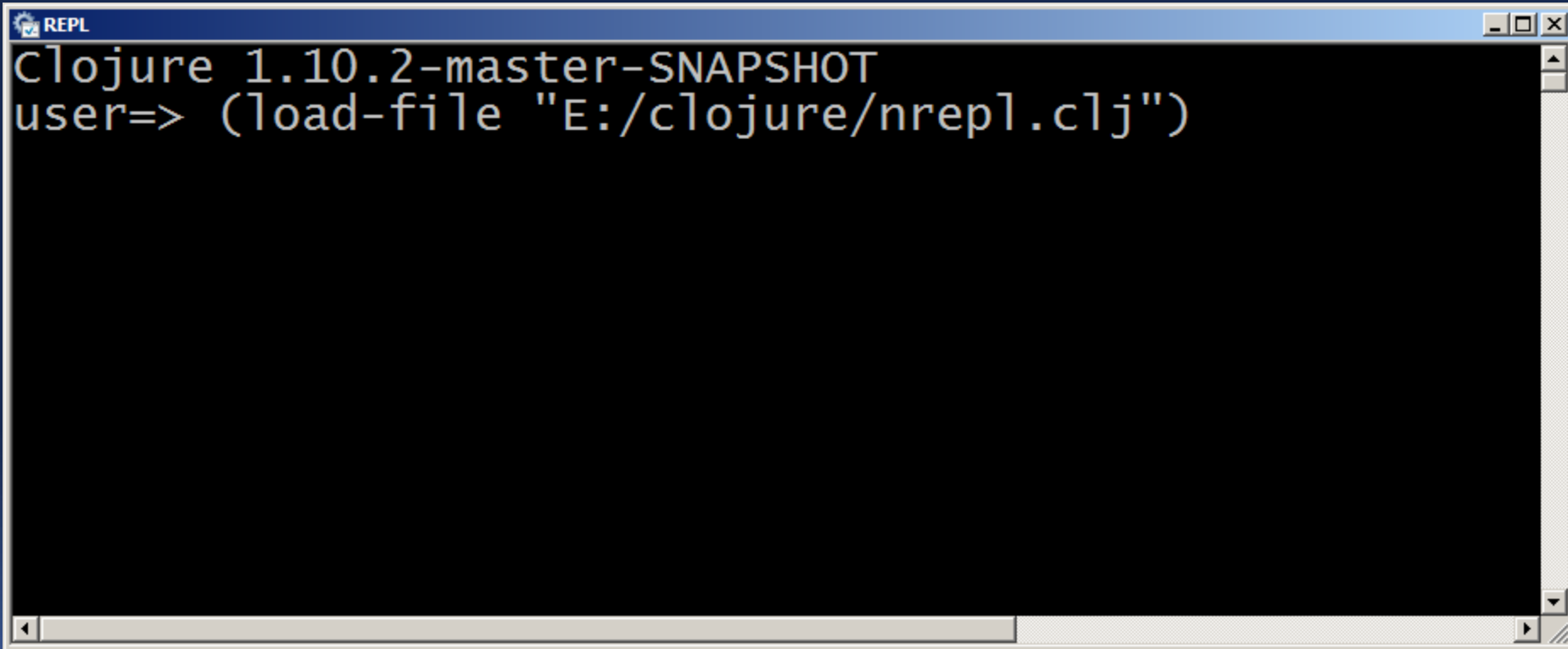


```
Clojure 1.10.2-master-SNAPSHOT
user=> (do (require 'clojure.core.server)
(clojure.core.server/start-server
{:name "socket-repl"
:port 4444
:accept 'clojure.main/repl
:address "localhost"}))
#object[java.net.ServerSocket 0x43f82e78 "ServerSocket[addr=localhost/127.0.0.1,
localport=4444]"]
user=> _
```

Ativando REPL Remoto

Carregando-se arquivo digitado anteriormente

> (load-file "E:/clojure/nrepl.clj")

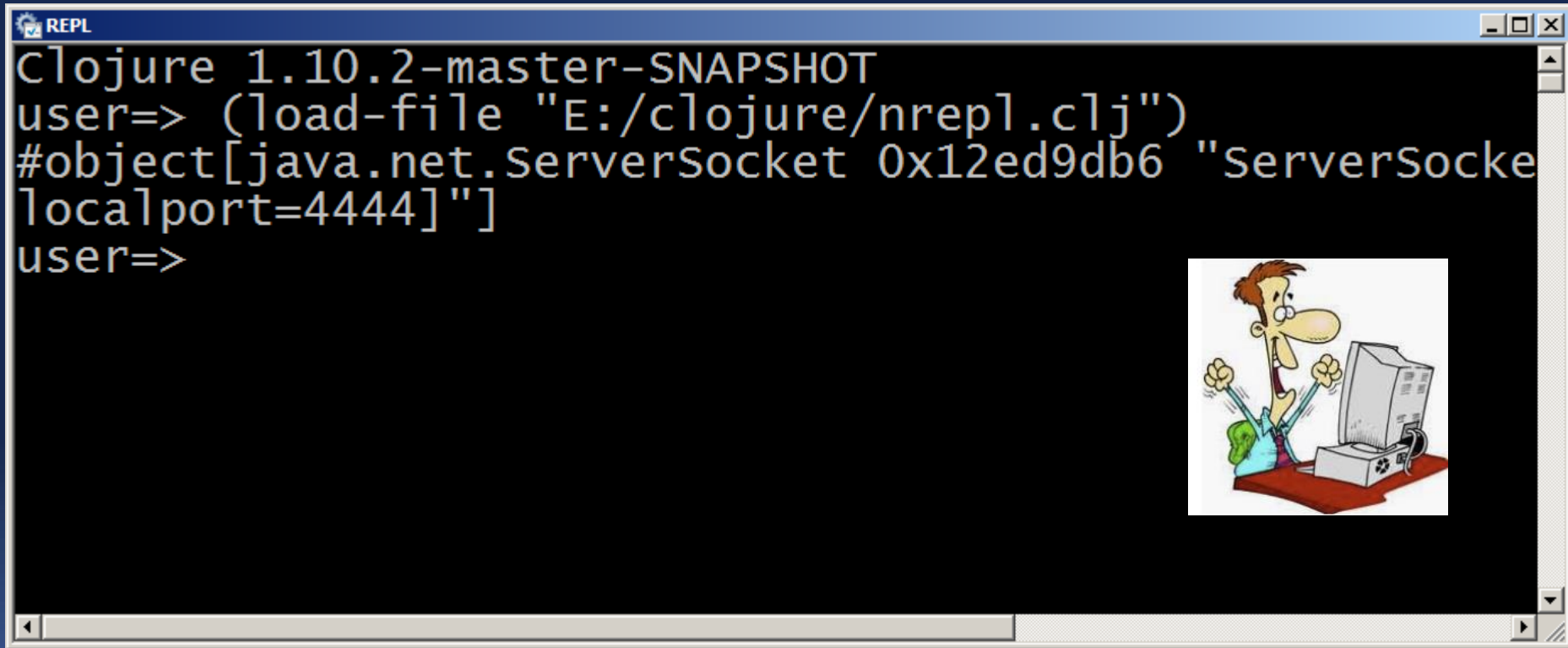


```
REPL
Clojure 1.10.2-master-SNAPSHOT
user=> (load-file "E:/clojure/nrepl.clj")
```

Ativando REPL Remoto

Carregando-se arquivo digitado anteriormente

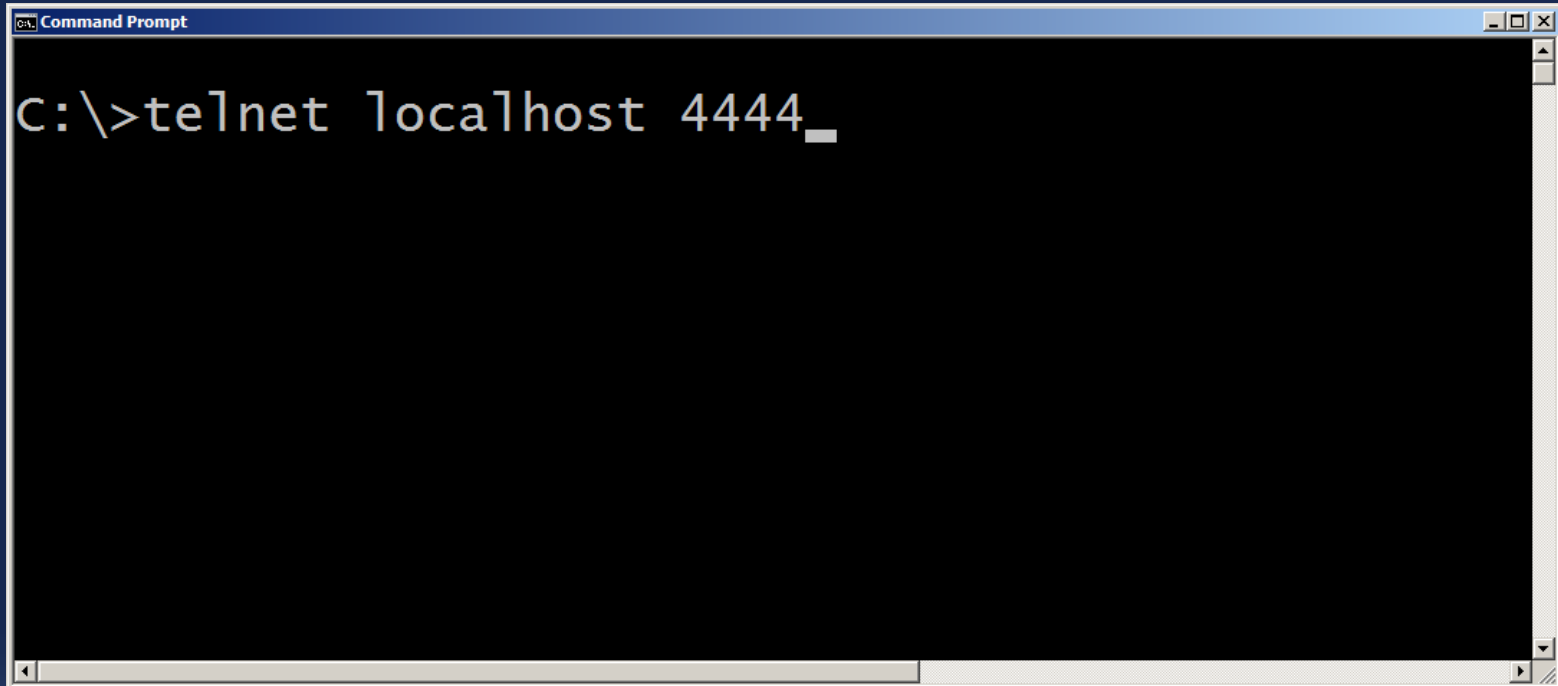
> (load-file "E:/clojure/nrepl.clj")



```
REPL
Clojure 1.10.2-master-SNAPSHOT
user=> (load-file "E:/clojure/nrepl.clj")
#object[java.net.ServerSocket 0x12ed9db6 "ServerSocket
localport=4444"]
user=>
```

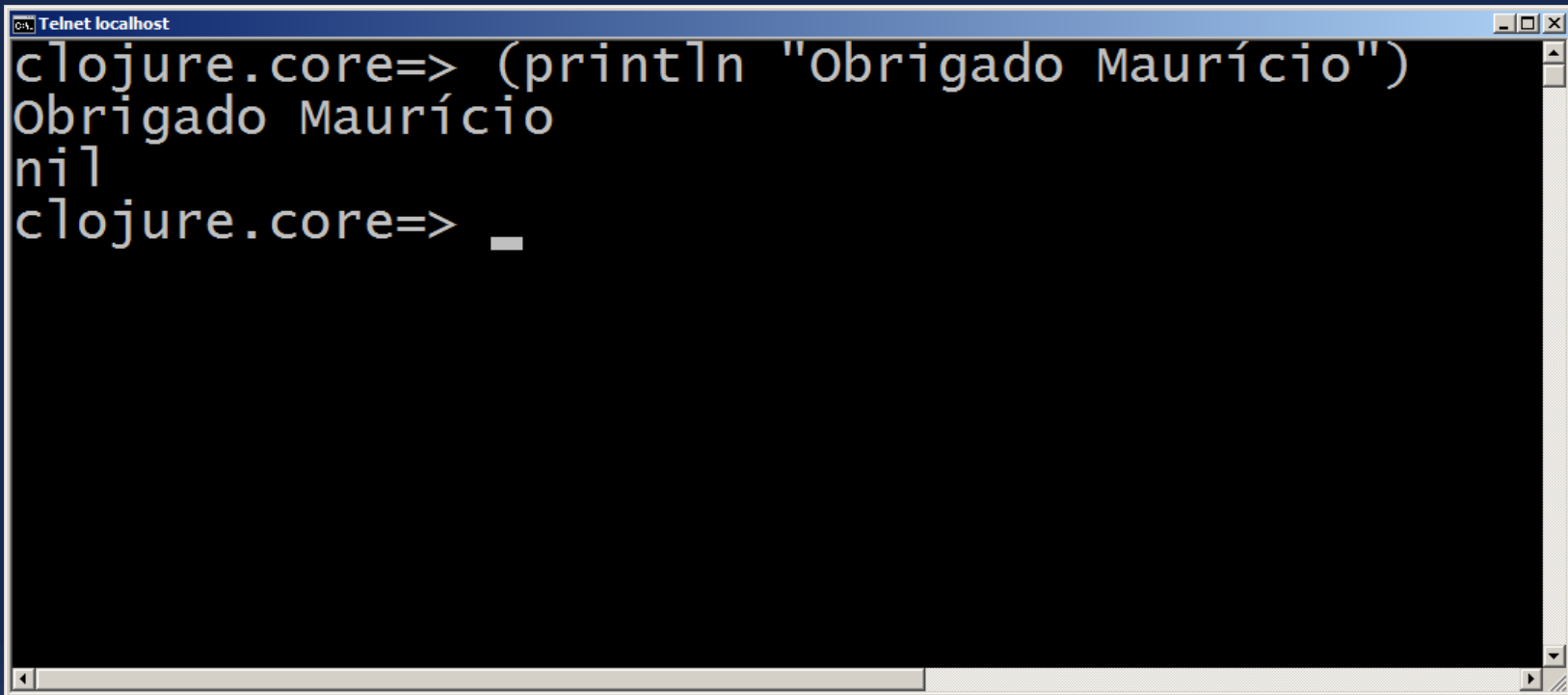


Conectando com Telnet na porta 4444



```
Command Prompt
C:\>telnet localhost 4444_
```

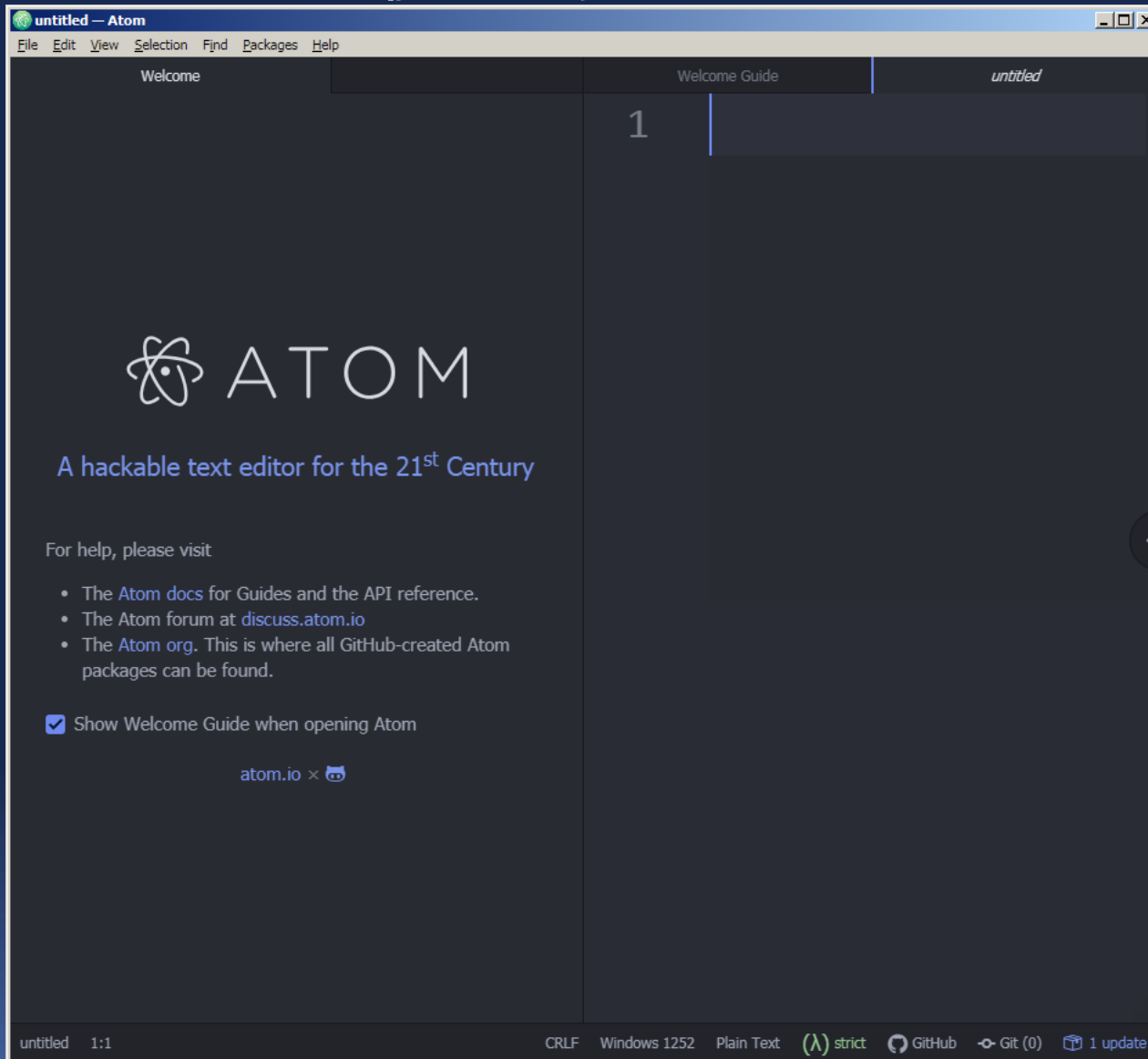
Conectando com Telnet na porta 4444



```
clojure.core=> (println "Obrigado Maurício")
Obrigado Maurício
nil
clojure.core=> _
```

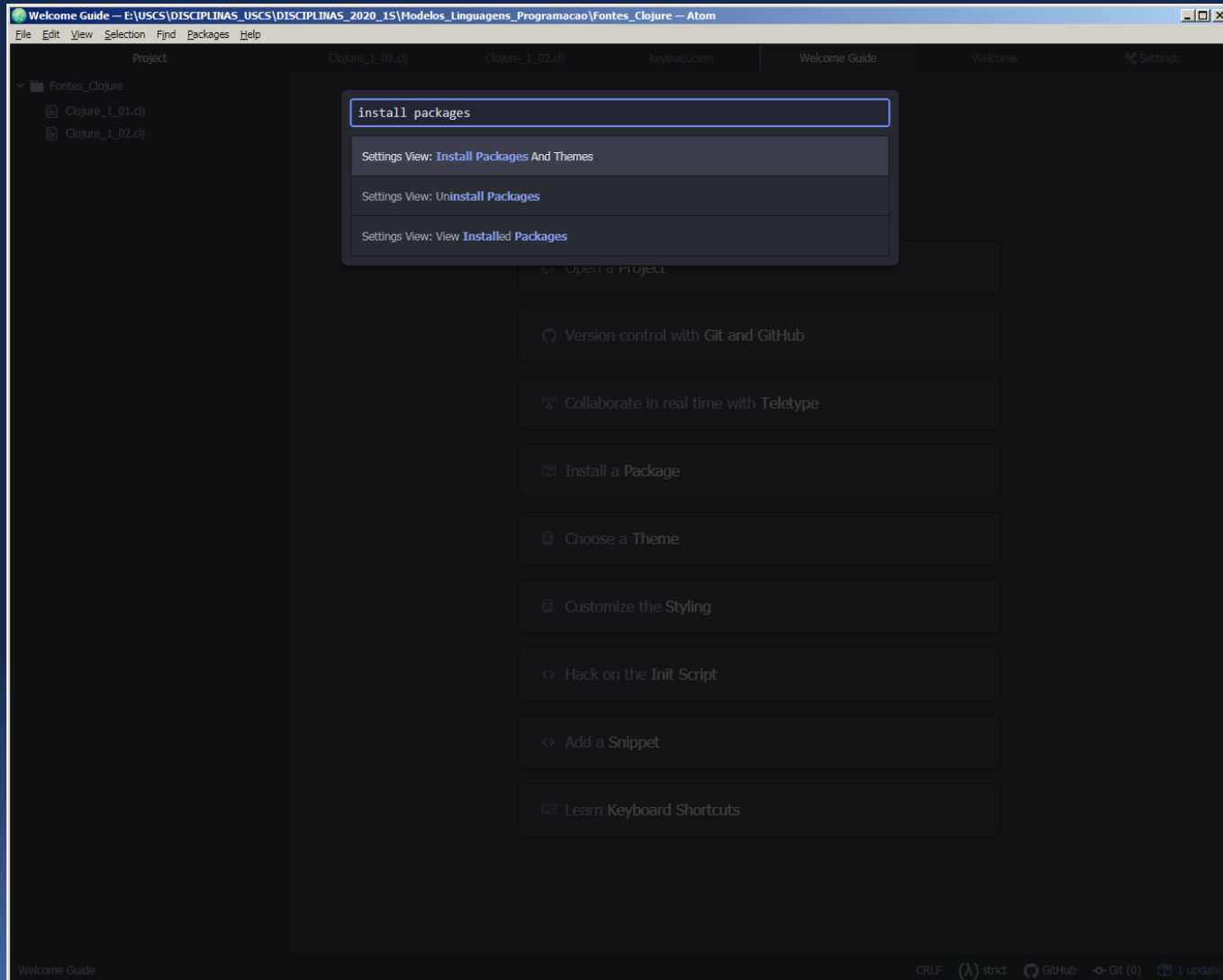
Desenvolvimento Clojure com a IDE Atom

Código Clojure no Atom



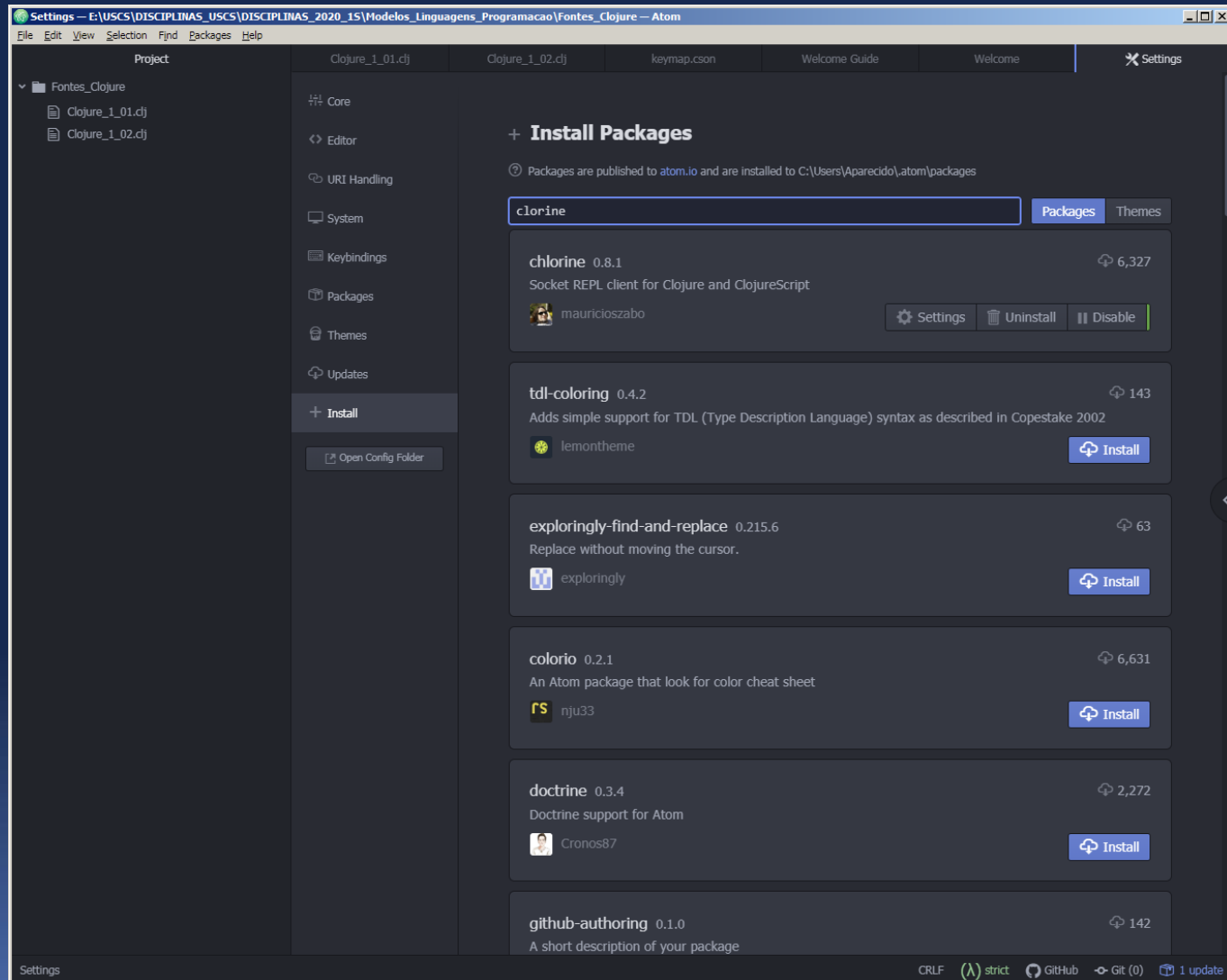
Atom – Package Chlorine

✓ **Ctrl + Shift + p** > **Install Packages and Themes**



Atom – Package Chlorine

✓ Instalar => package **Chlorine**



Chlorine – Configuração de Hot Keys

- ✓ Instalar => package **Chlorine**
- ✓ Ao se teclar **CTRL+SHIFT+p** e procurar por "**Open your keymap**", Atom irá abrir um arquivo pra customizar hotkeys;
- ✓ Nesse arquivo podem ser incluídas as configurações:

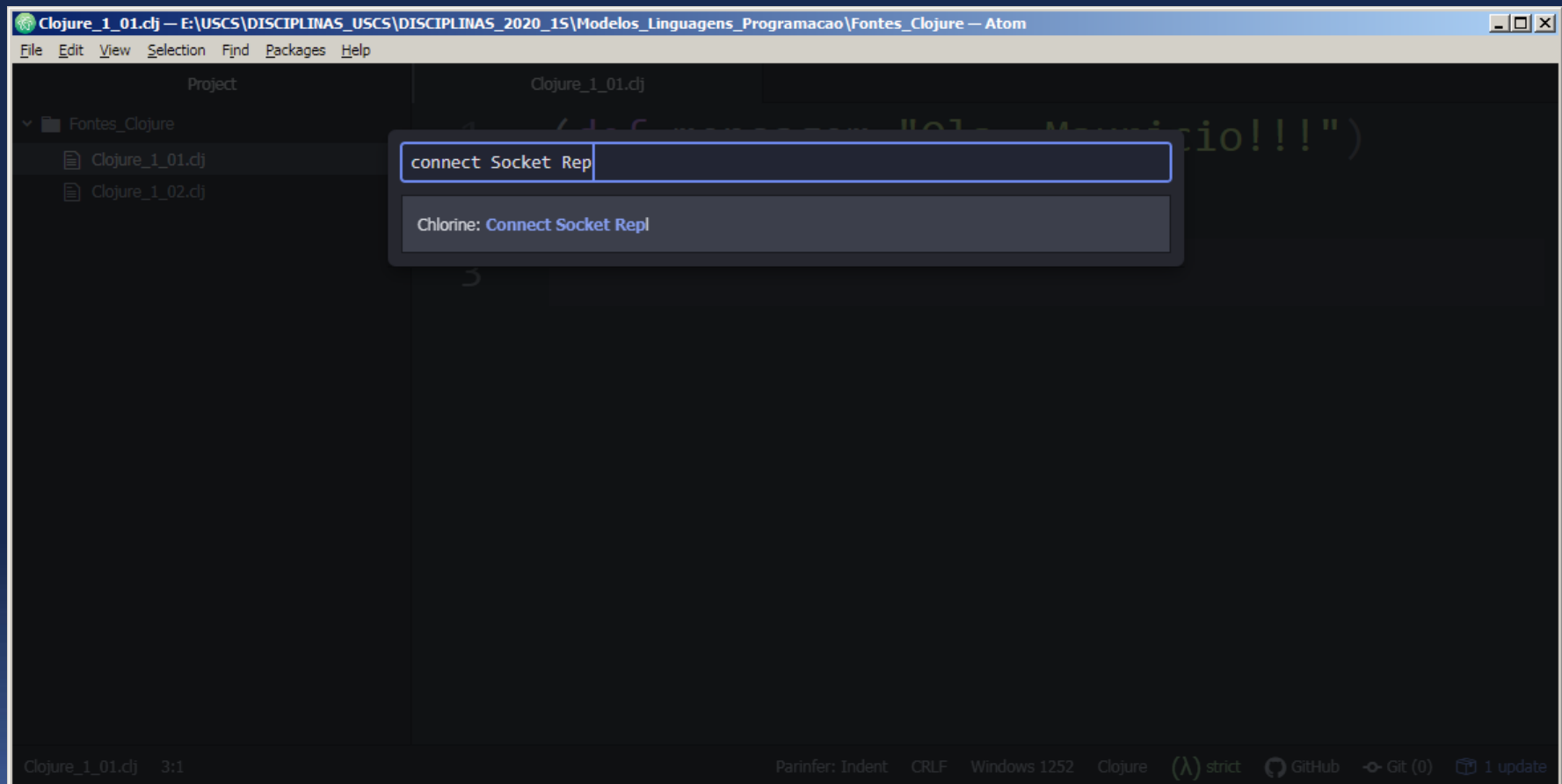
```
'atom-text-editor':  
  'ctrl-k': 'chlorine:clear-console'  
  'ctrl-shift-I': 'chlorine:load-file'  
  'ctrl-shift-l': 'chlorine:clear-inline-results'  
  'ctrl-shift-enter': 'chlorine:evaluate-block'  
  'ctrl-enter': 'chlorine:evaluate-top-block'  
  'ctrl-c': 'chlorine:break-evaluation'  
  'ctrl-d': 'chlorine:doc-for-var'
```

Chlorine – Configuração de Hot Keys

```
# Atom Flight Manual:  
# http://flight-manual.atom.io/using-atom/sections/basic-custom  
  
'atom-text-editor':  
  'ctrl-k':          'chlorine:clear-console'  
  'ctrl-shift-I':    'chlorine:load-file'  
  'ctrl-shift-L':    'chlorine:clear-inline-results'  
  'ctrl-shift-enter': 'chlorine:evaluate-block'  
  'ctrl-enter':      'chlorine:evaluate-top-block'  
  'ctrl-c':          'chlorine:break-evaluation'  
  'ctrl-d':          'chlorine:doc-for-var'
```

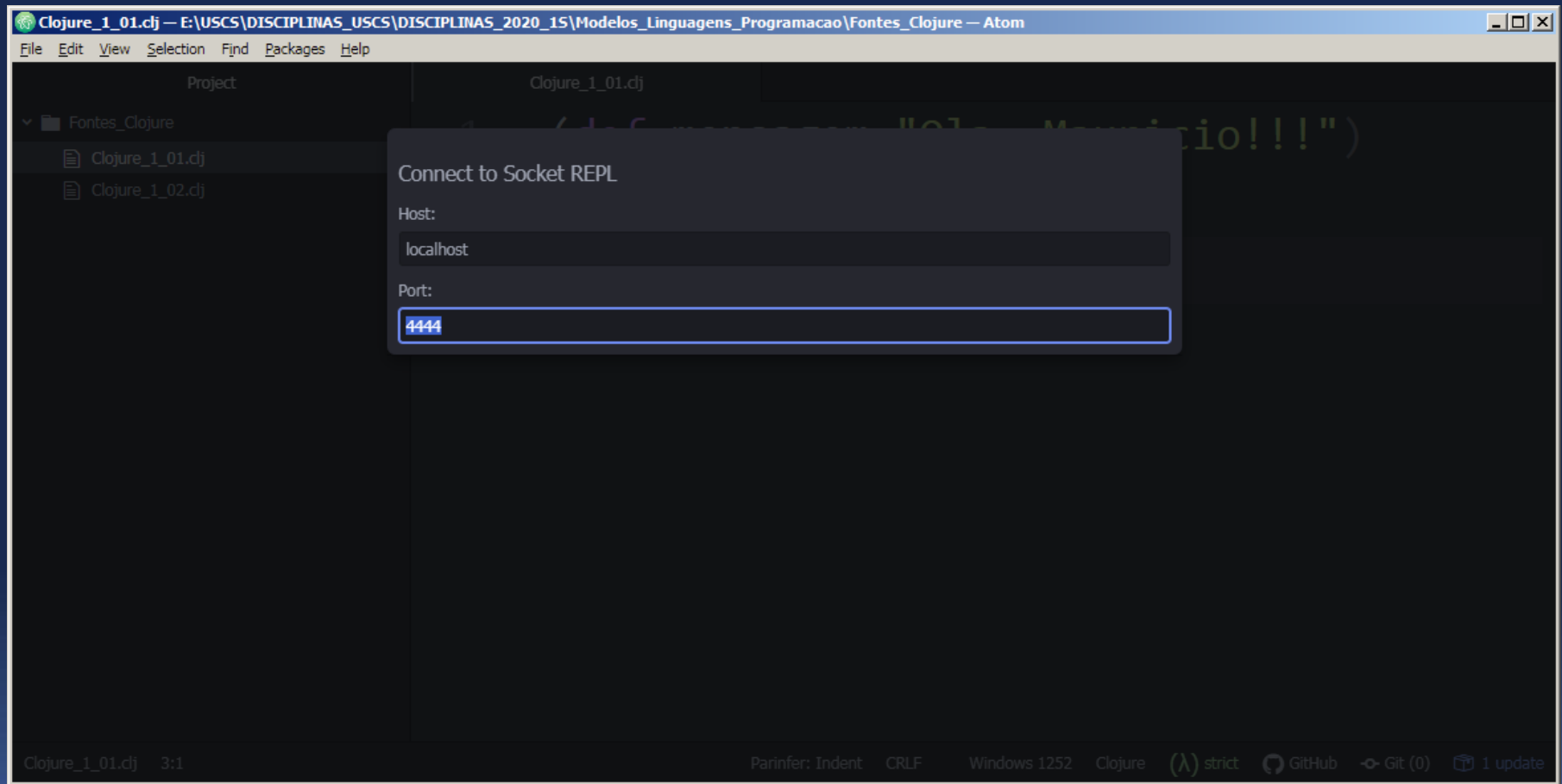
Atom – Package Chlorine

- ✓ Após instalação do Chlorine e subida do socket com REPL remoto
- ✓ **Ctrl + Shift + p** > **Connect Socket Repl**



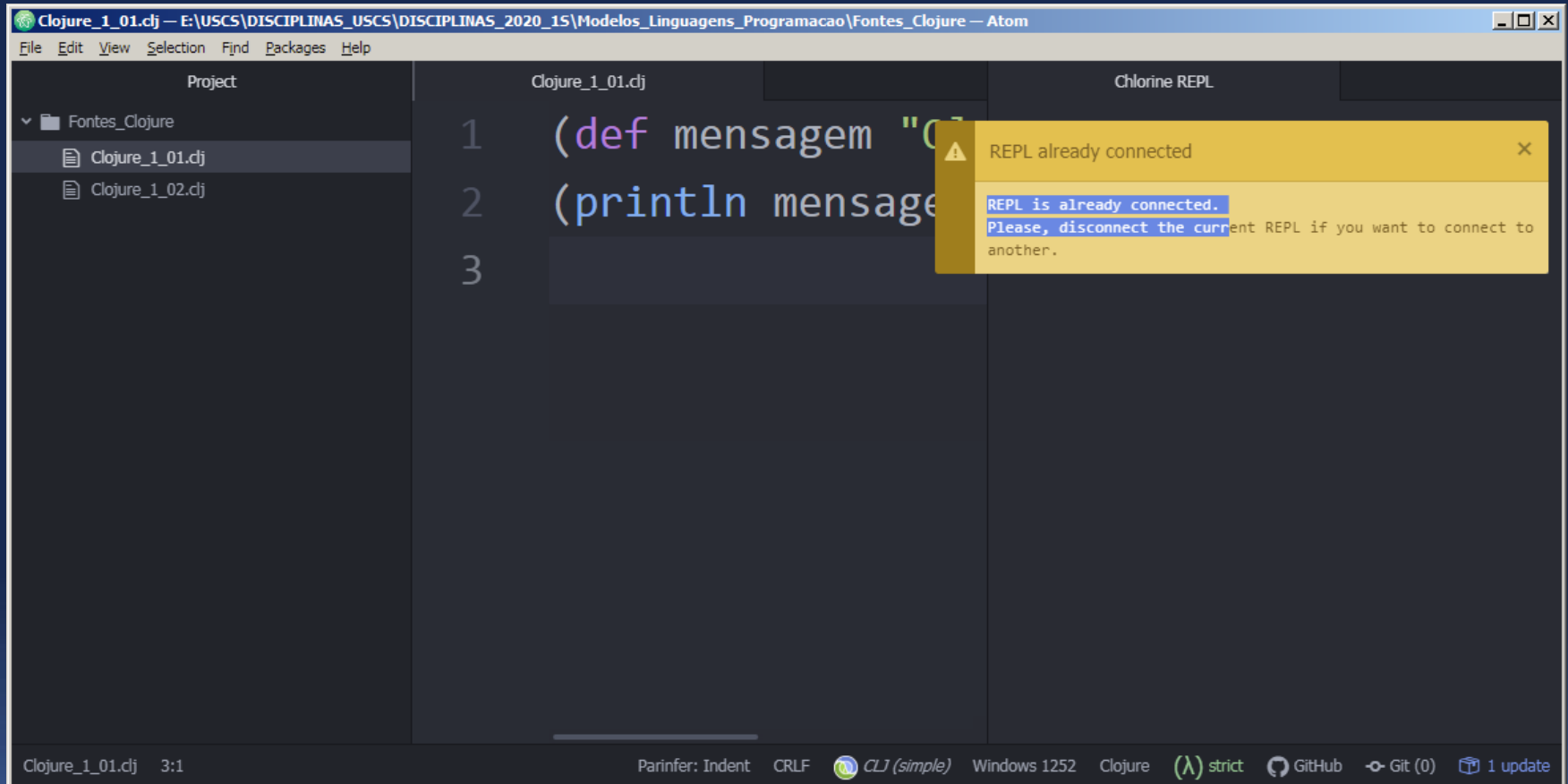
Atom – Package Chlorine

- ✓ Informar Host e porta onde o REPL remoto está sendo executado



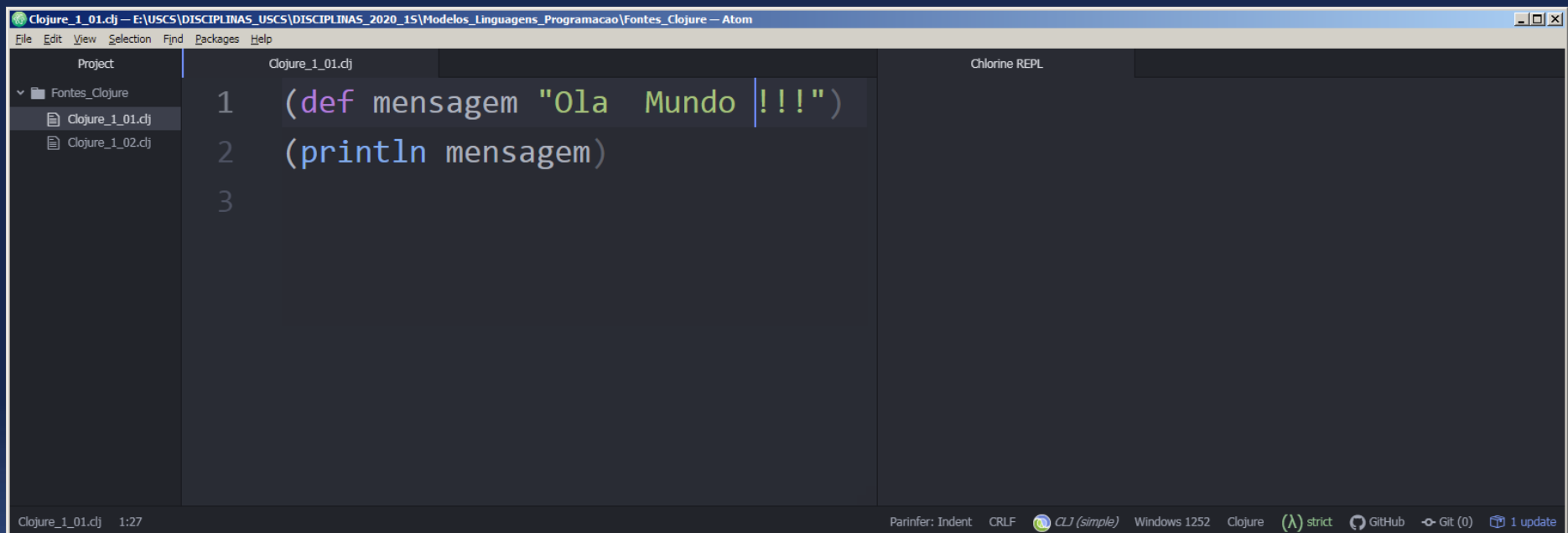
Atom – Package Chlorine

✓REPL connected !!!



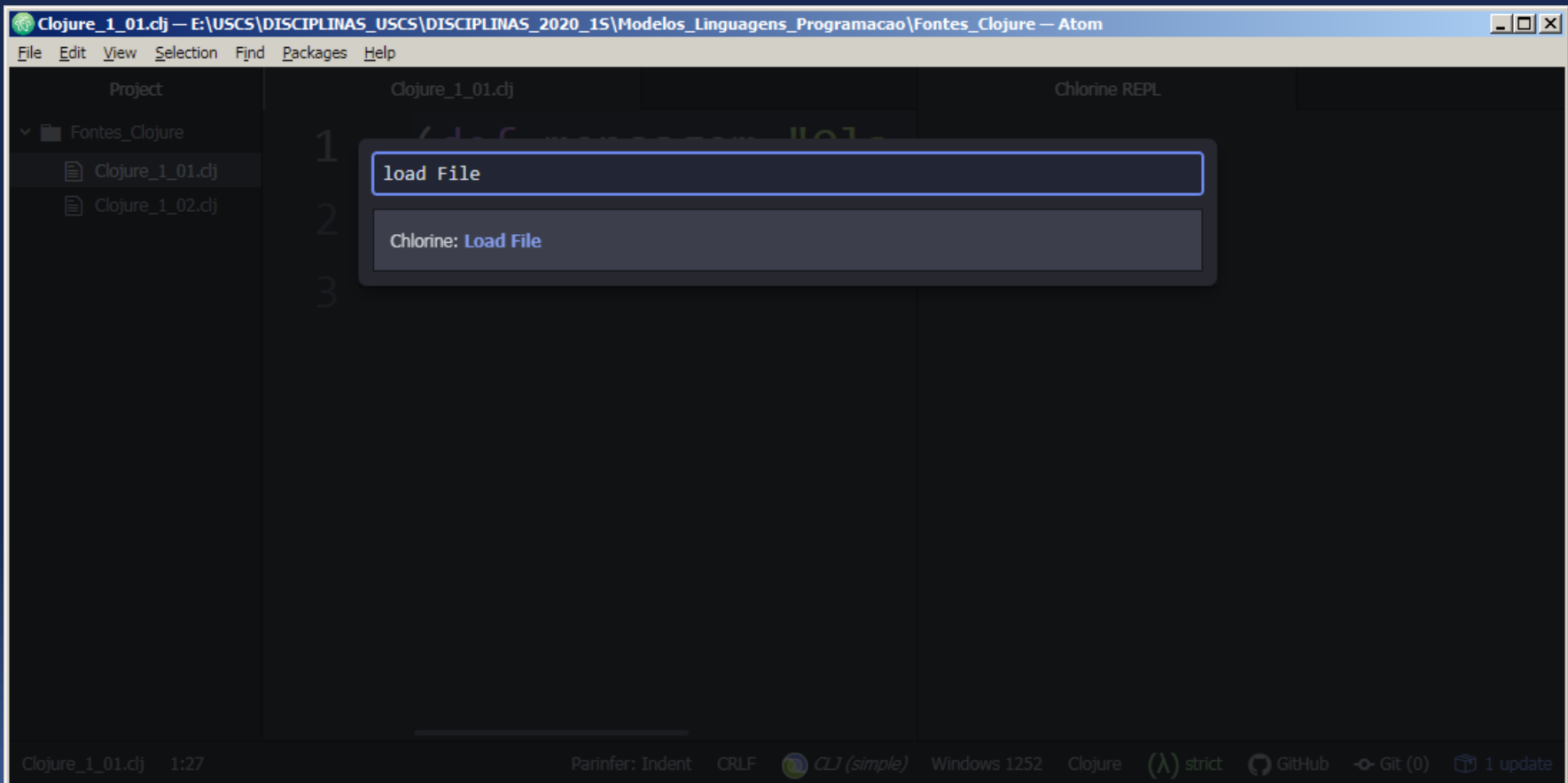
Atom – Package Chlorine

✓ Escrever código **Clojure** (extensão .clj)



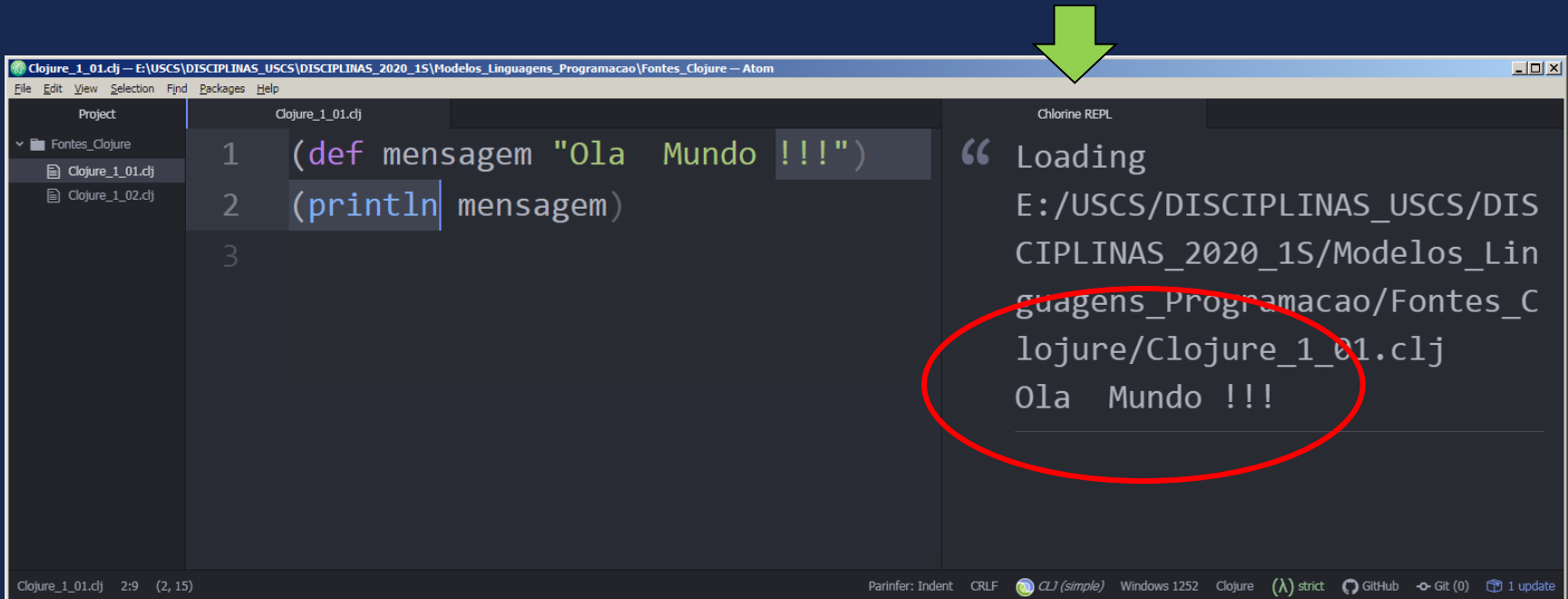
Atom – Package Chlorine

- ✓ Carregar código para Repl remoto
- ✓ **Ctrl + Shift + p** > **load File**



Atom – Package Chlorine

✓ Carga do código Clojure para o REPL



The screenshot shows the Atom editor interface with the Chlorine REPL package installed. The editor is open to a file named `Clojure_1_01.clj` in the project `Fontes_Clojure`. The code in the editor is:

```
1 (def mensagem "Ola Mundo !!!")
2 (println mensagem)
3
```

The Chlorine REPL output shows the following text:

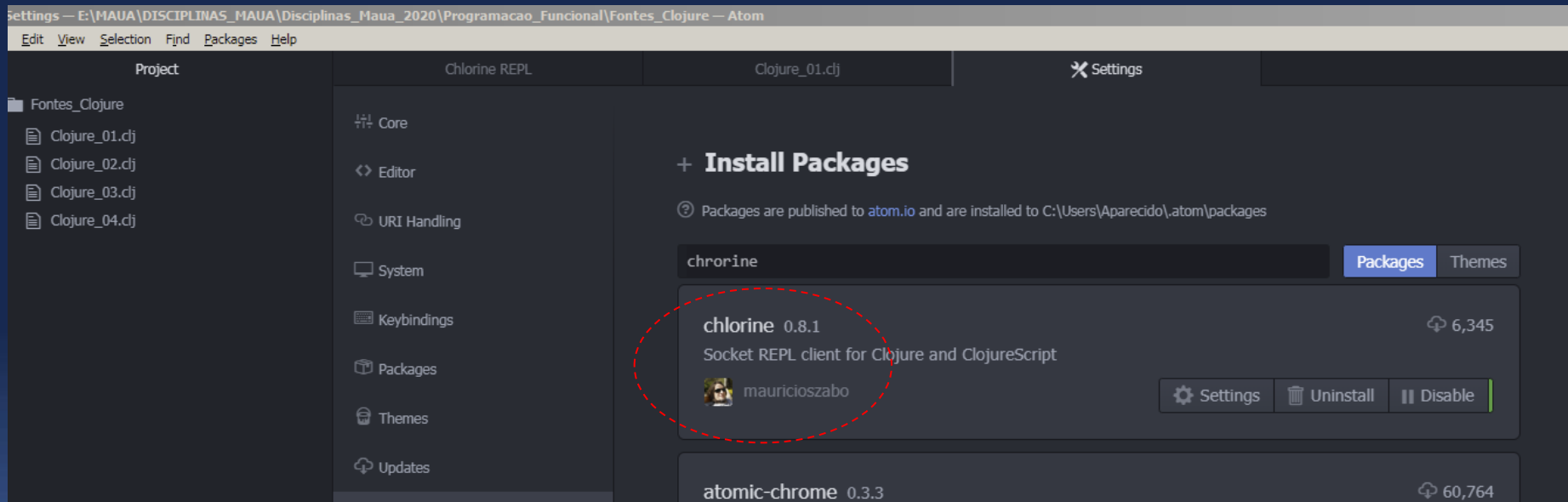
```
“ Loading
E:/USCS/DISCIPLINAS_USCS/DISCIPLINAS_2020_1S/Modelos_Linguagens_Programacao/Fontes_Clojure/Clojure_1_01.clj
Ola Mundo !!!
```

The output text is circled in red, indicating the successful execution of the code. The status bar at the bottom shows the current file is `Clojure_1_01.clj` at line 2, column 15. The bottom right corner of the status bar shows various icons and the text "1 update".

Atom – Package Chlorine + nREPL (Leiningen)

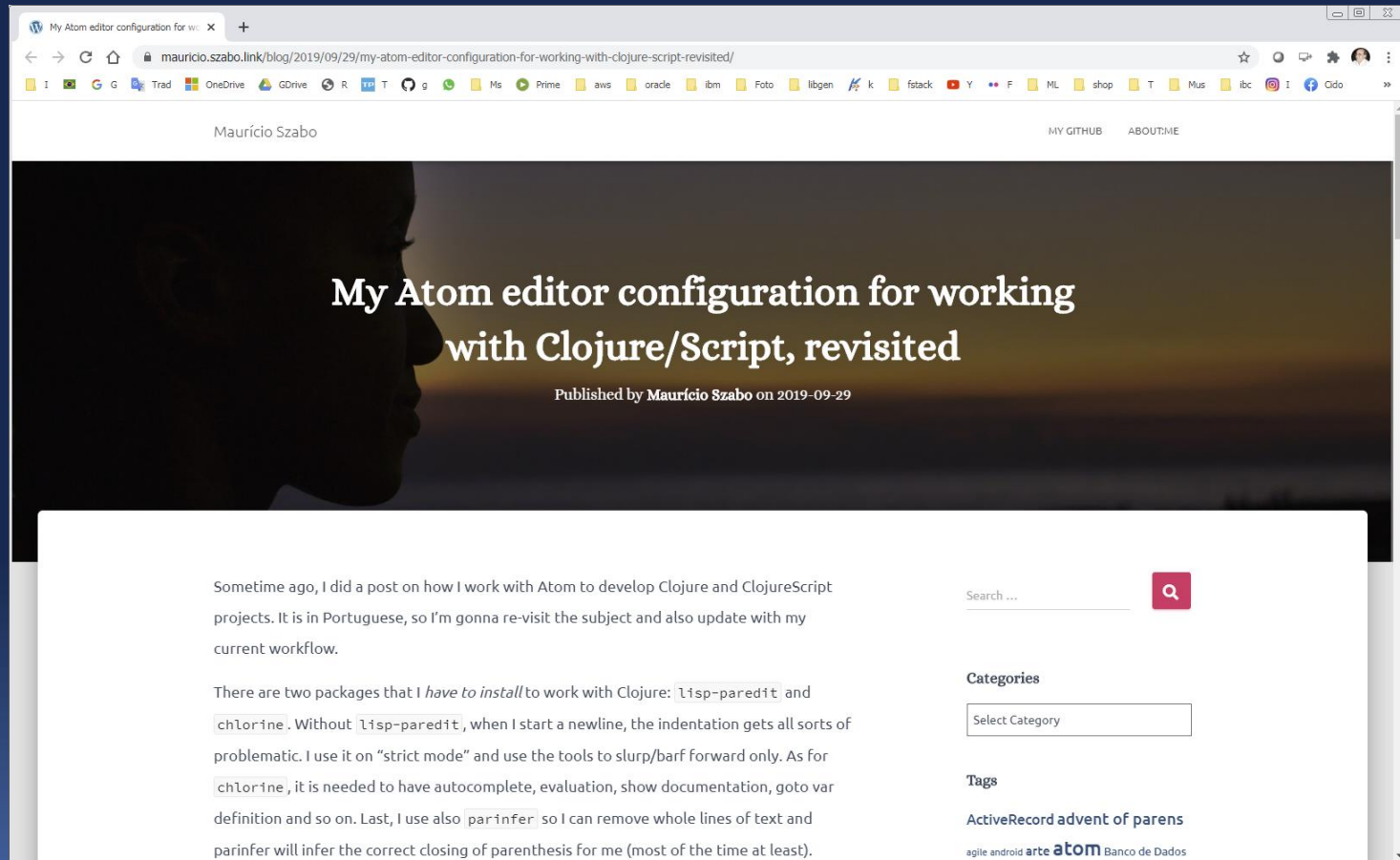
Configuração do Package Chlorine no Atom

- ✓ Instalação do Package **Chroline** no Atom
- ✓ No menu do Atom: Packages > Settings View > Install Packages / Themes



Configuração do Package Chlorine no Atom

<https://mauricio.szabo.link/blog/2019/09/29/my-atom-editor-configuration-for-working-with-clojure-script-revisited/>



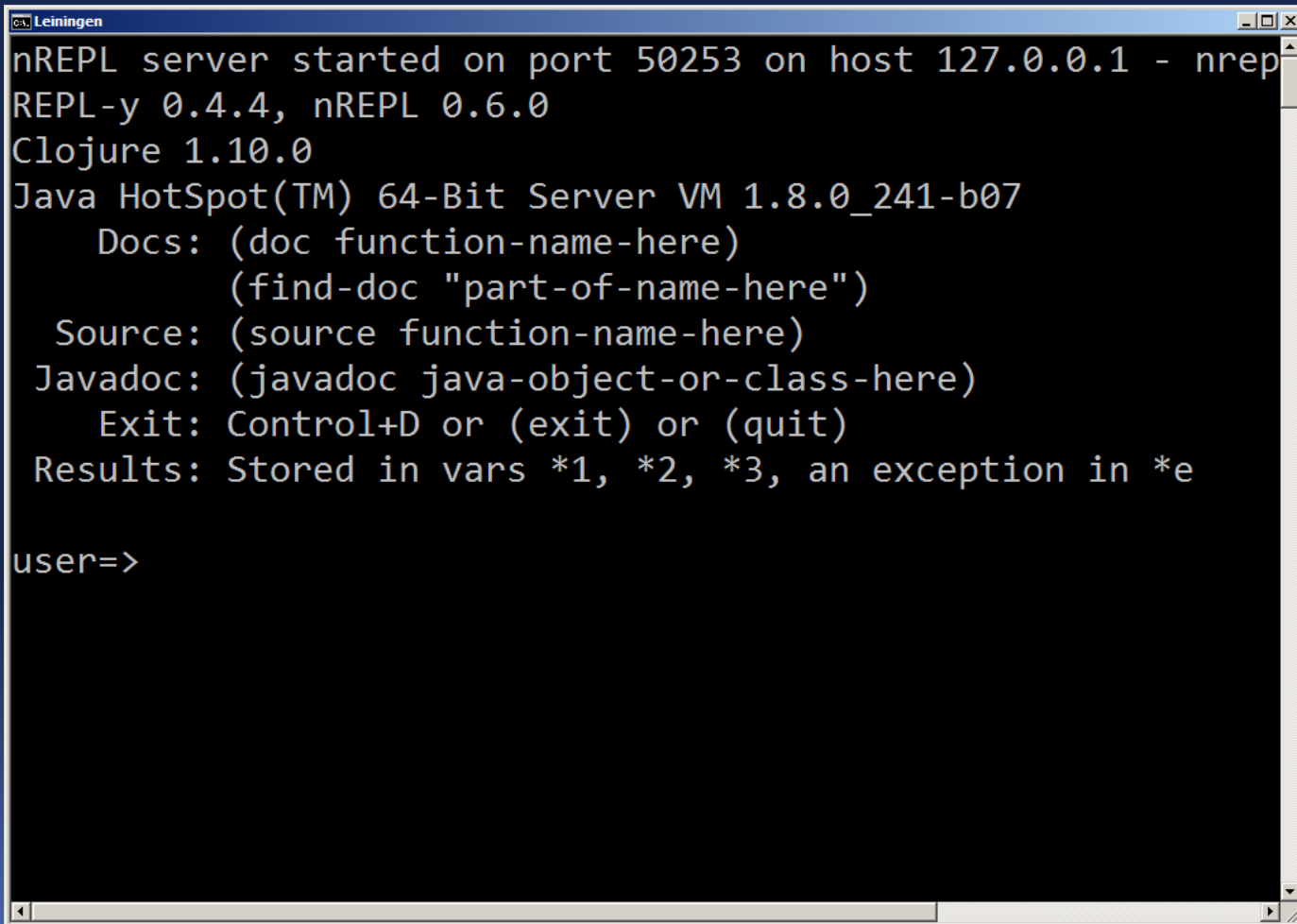
Configuração do Package Chlorine no Atom

- ✓ Sob Atom, teclar `Crtl + Shift + p` e procurar "**Open your keymap**";
- ✓ Após abertura do arquivo para customizar **keys**, incluir:

<code>'atom-text-editor':</code>	
<code>'ctrl-k':</code>	<code>'chlorine:clear-console'</code>
<code>'ctrl-shift-l':</code>	<code>'chlorine:load-file'</code>
<code>'ctrl-shift-enter':</code>	<code>'chlorine:evaluate-block'</code>
<code>'ctrl-enter':</code>	<code>'chlorine:evaluate-top-block'</code>
<code>'ctrl-c':</code>	<code>'chlorine:break-evaluation'</code>
<code>'ctrl-d':</code>	<code>'chlorine:doc-for-var'</code>

Ativar nREPL – Leiningen

- ✓ Anotar a porta de comunicação, por exemplo: **50253**

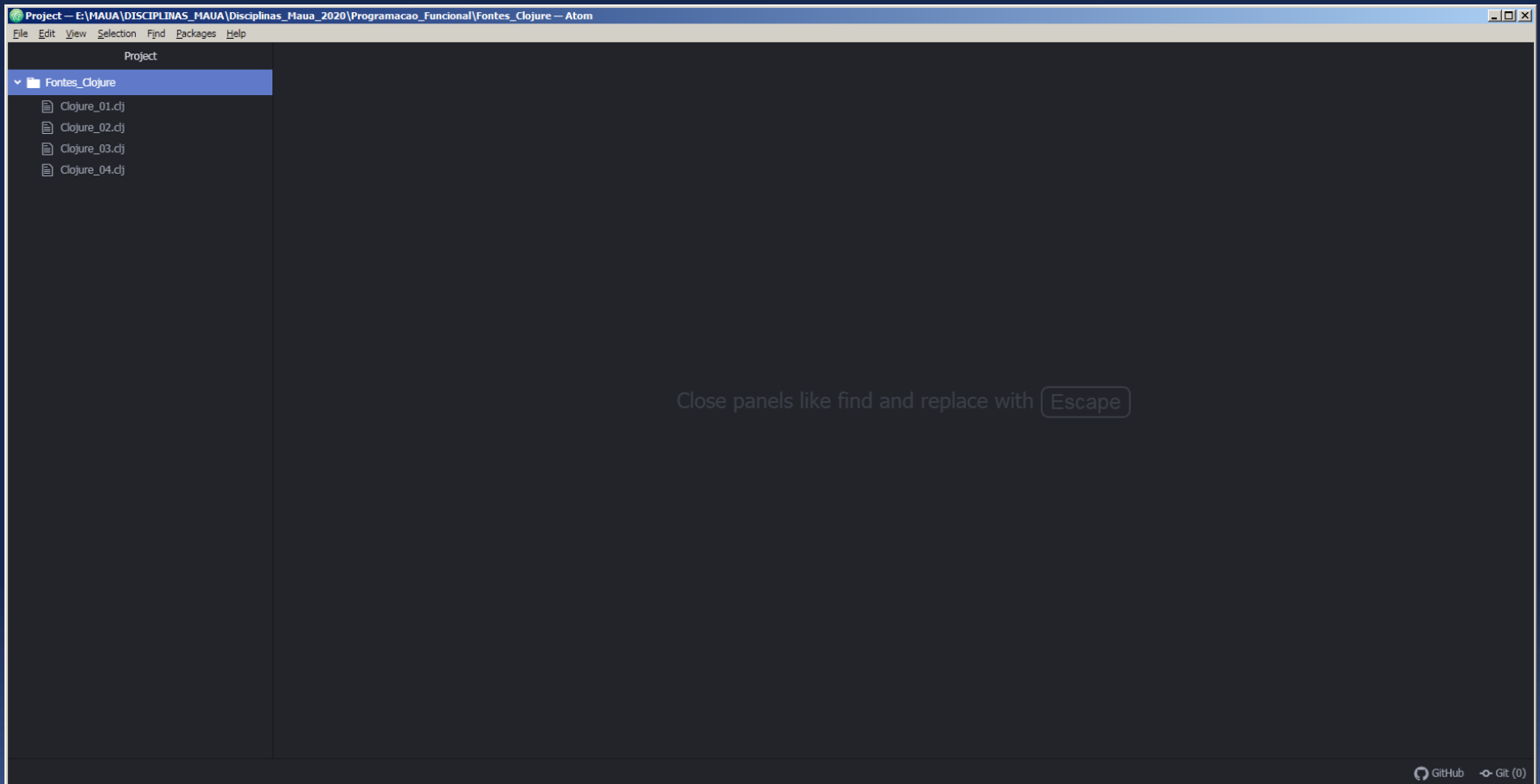


```
Leiningen
nREPL server started on port 50253 on host 127.0.0.1 - nrepl
REPL-y 0.4.4, nREPL 0.6.0
Clojure 1.10.0
Java HotSpot(TM) 64-Bit Server VM 1.8.0_241-b07
  Docs: (doc function-name-here)
        (find-doc "part-of-name-here")
  Source: (source function-name-here)
  Javadoc: (javadoc java-object-or-class-here)
  Exit: Control+D or (exit) or (quit)
  Results: Stored in vars *1, *2, *3, an exception in *e

user=>
```

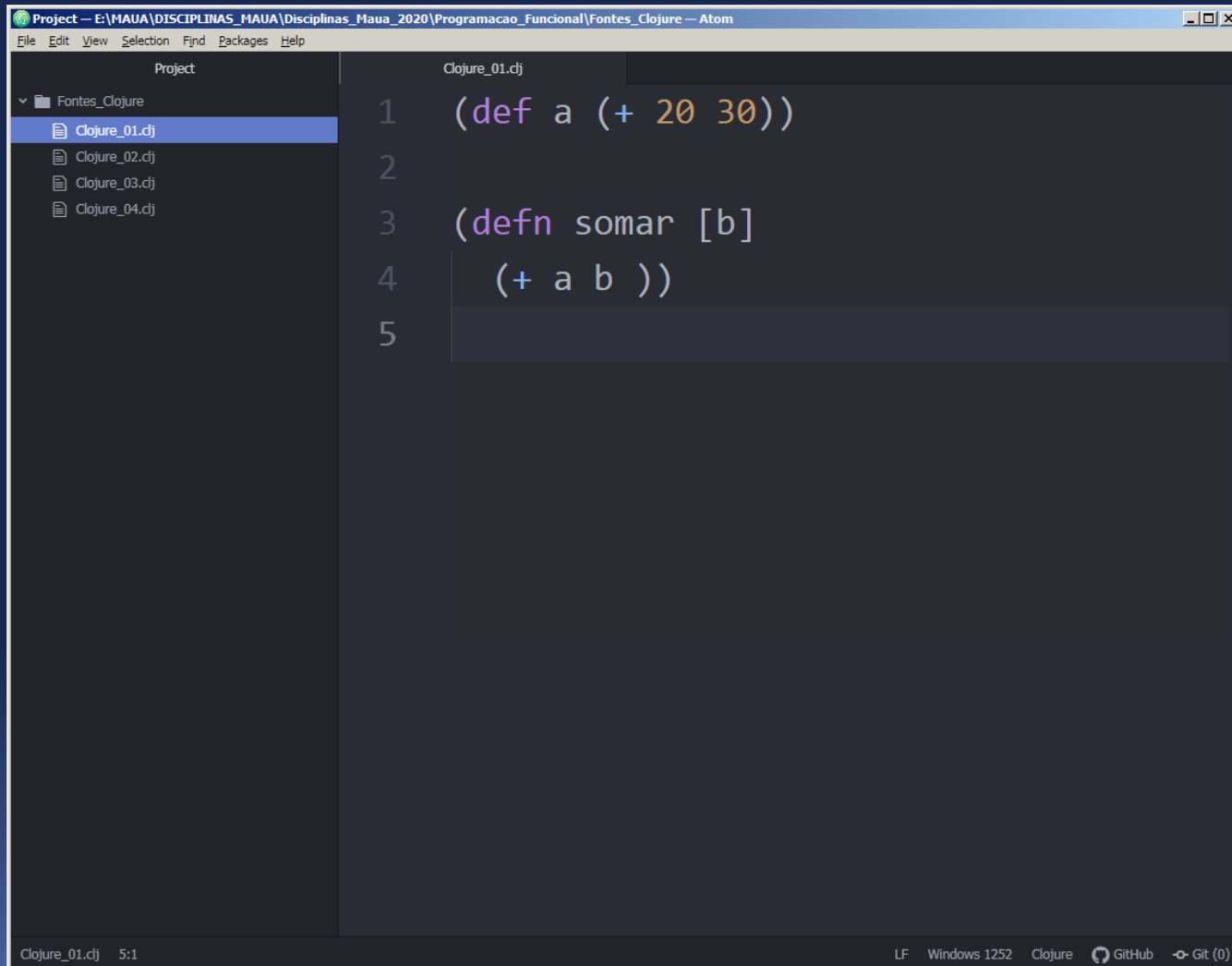
Ativar Atom

✓ File > Open Folder



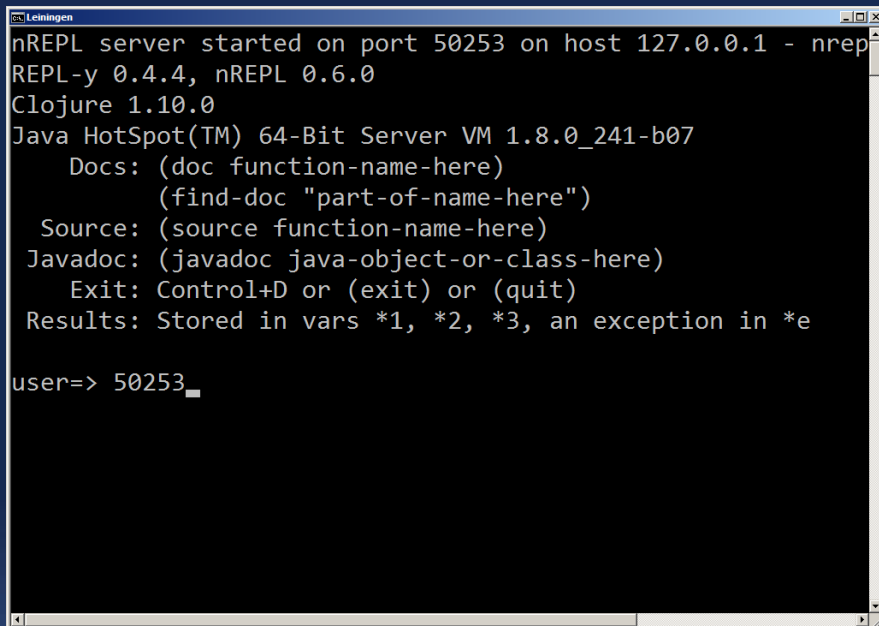
Ativar Atom

✓ Escrevendo código Clojure no Atom



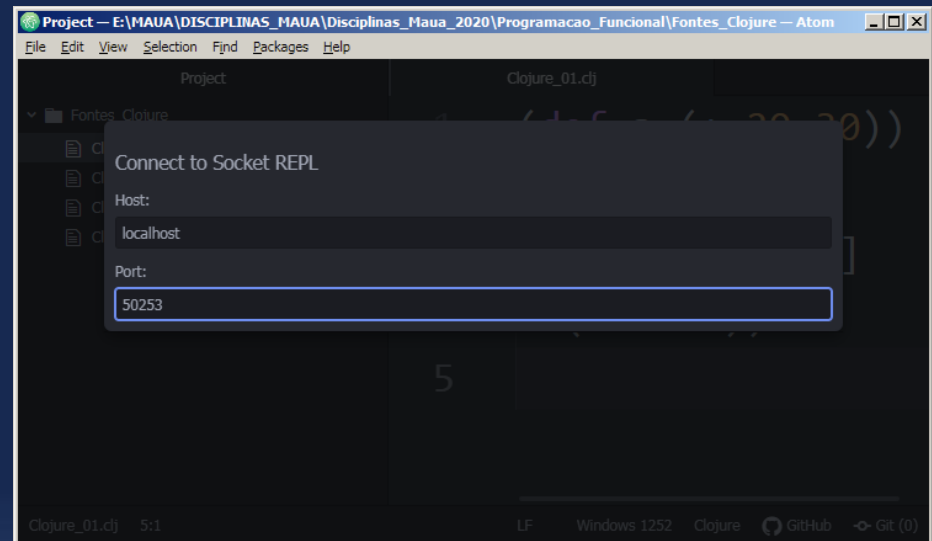
Ativar Atom

- ✓ Ativando **nREPL** (Leiningen)
- ✓ **Ctrl + Shift + p** > Connect to Socket REPL
 - > Host: **localhost**
 - > Port: aquela fornecida pela nREPL (Leiningen), por exemplo: **50253**

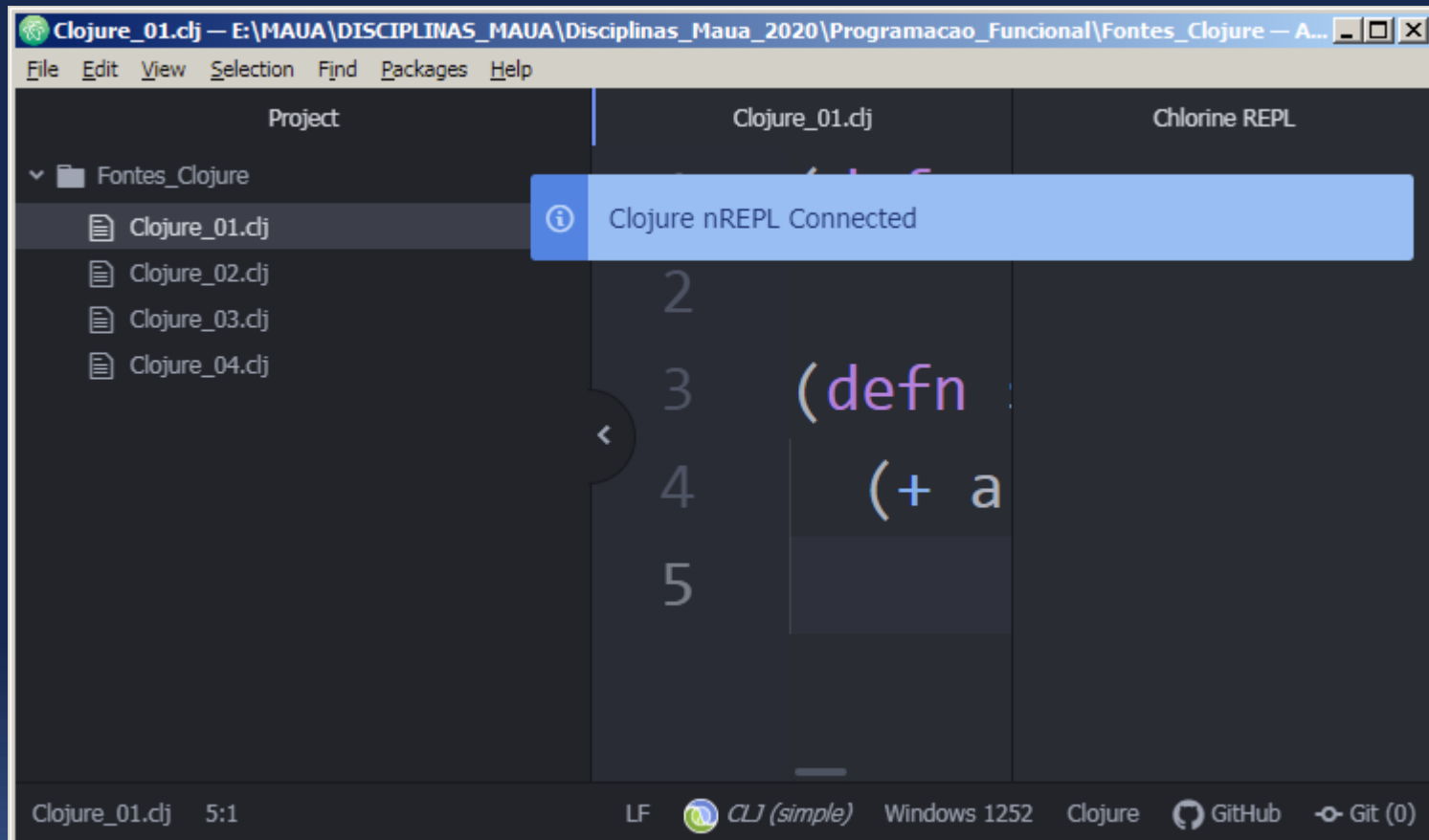


```
Leiningen
nREPL server started on port 50253 on host 127.0.0.1 - nrepl
REPL-y 0.4.4, nREPL 0.6.0
Clojure 1.10.0
Java HotSpot(TM) 64-Bit Server VM 1.8.0_241-b07
Docs: (doc function-name-here)
      (find-doc "part-of-name-here")
Source: (source function-name-here)
Javadoc: (javadoc java-object-or-class-here)
Exit: Control+D or (exit) or (quit)
Results: Stored in vars *1, *2, *3, an exception in *e

user=> 50253_
```

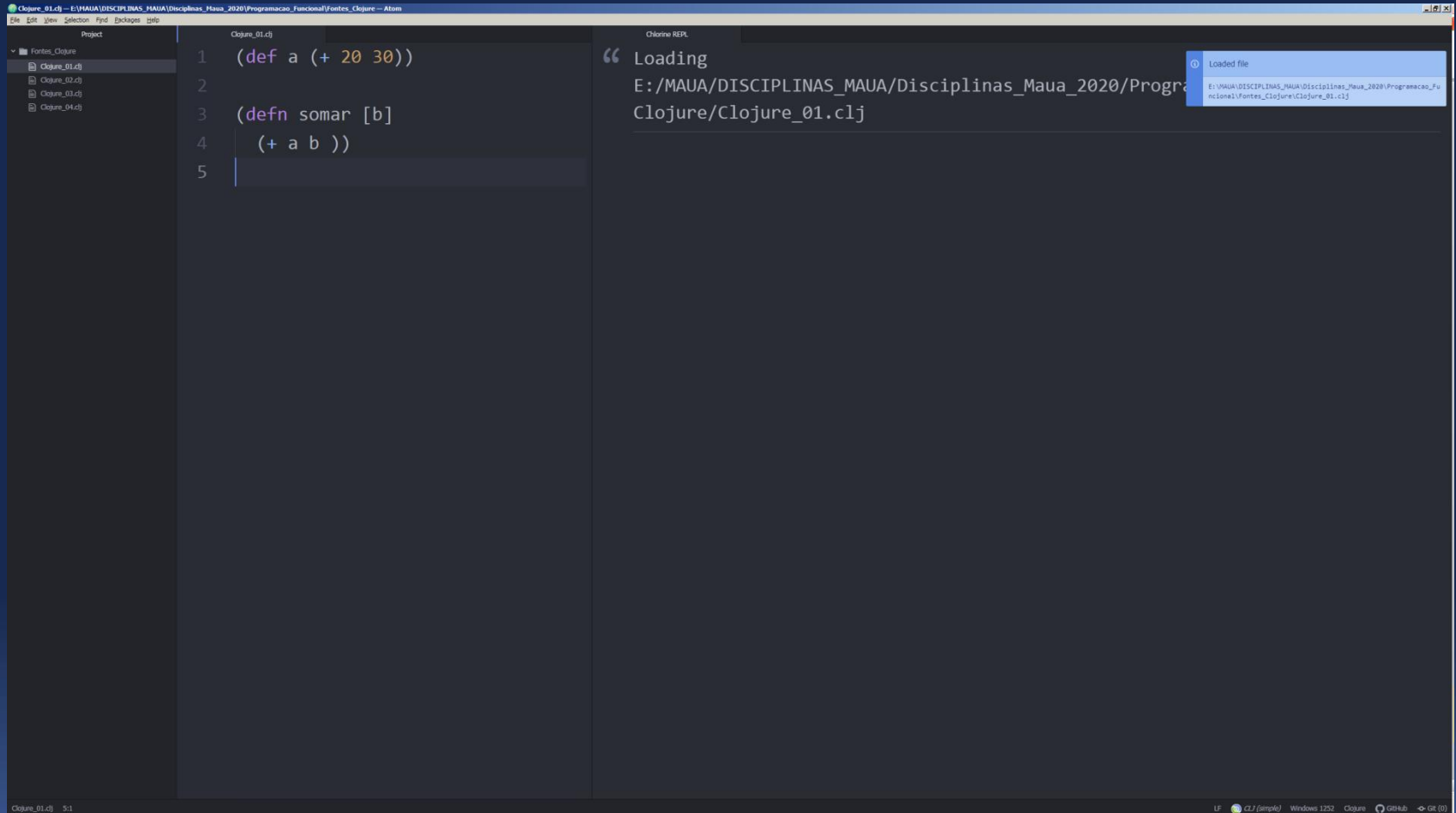


Clojure nREPL Connected



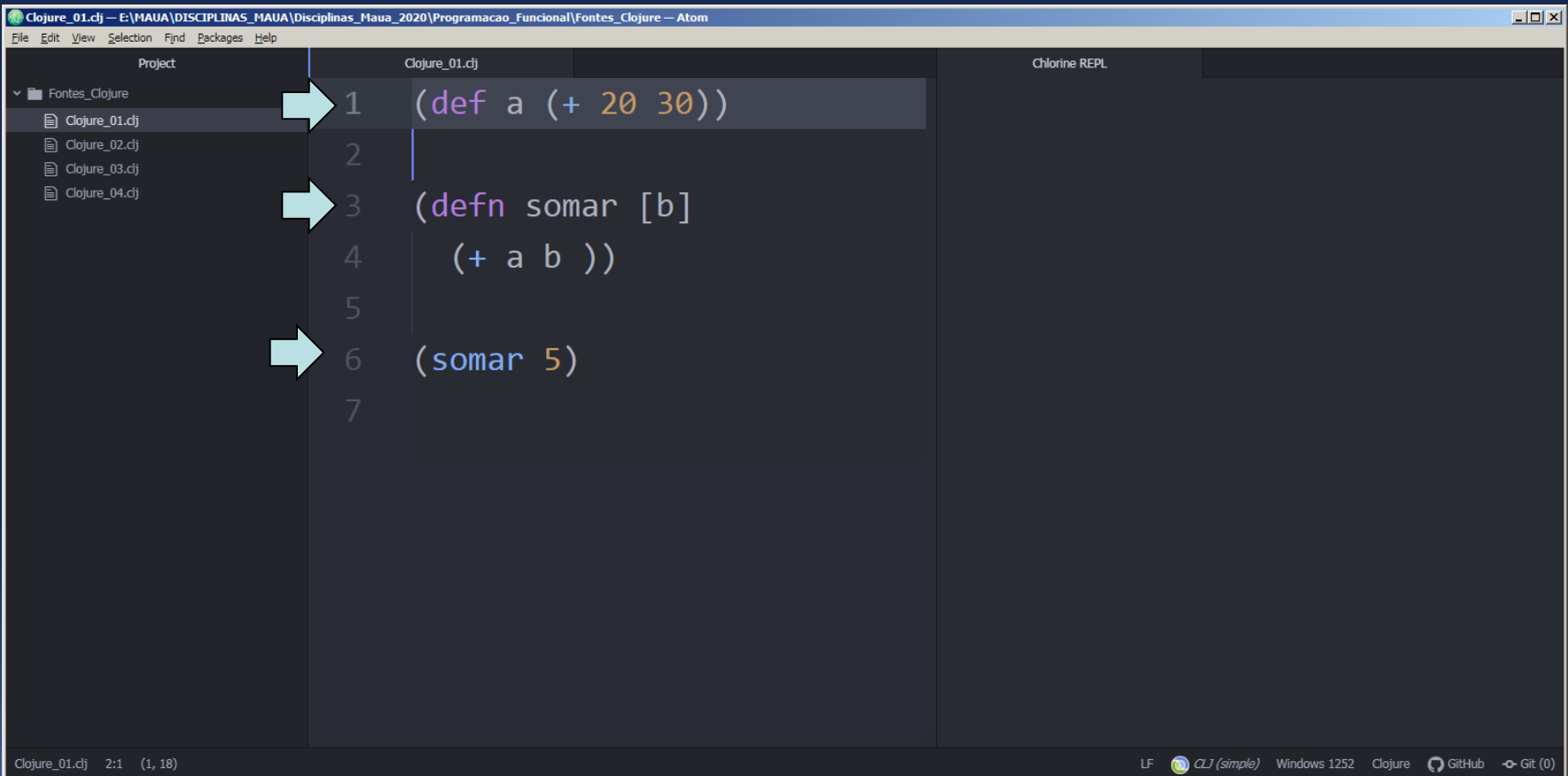
Carregando Código para nREPL

✓ Ctrl + Shift + p > Chroline: **Load File**



Evaluate "top-blocks"

- ✓ Hot Key: **Ctrl + Enter**
- ✓ Selecionar o top block desejado e teclar => **Ctrl + Enter**



The screenshot shows the Atom editor interface with a project named 'Fontes_Clojure'. The file 'Clojure_01.cj' is open, displaying the following Clojure code:

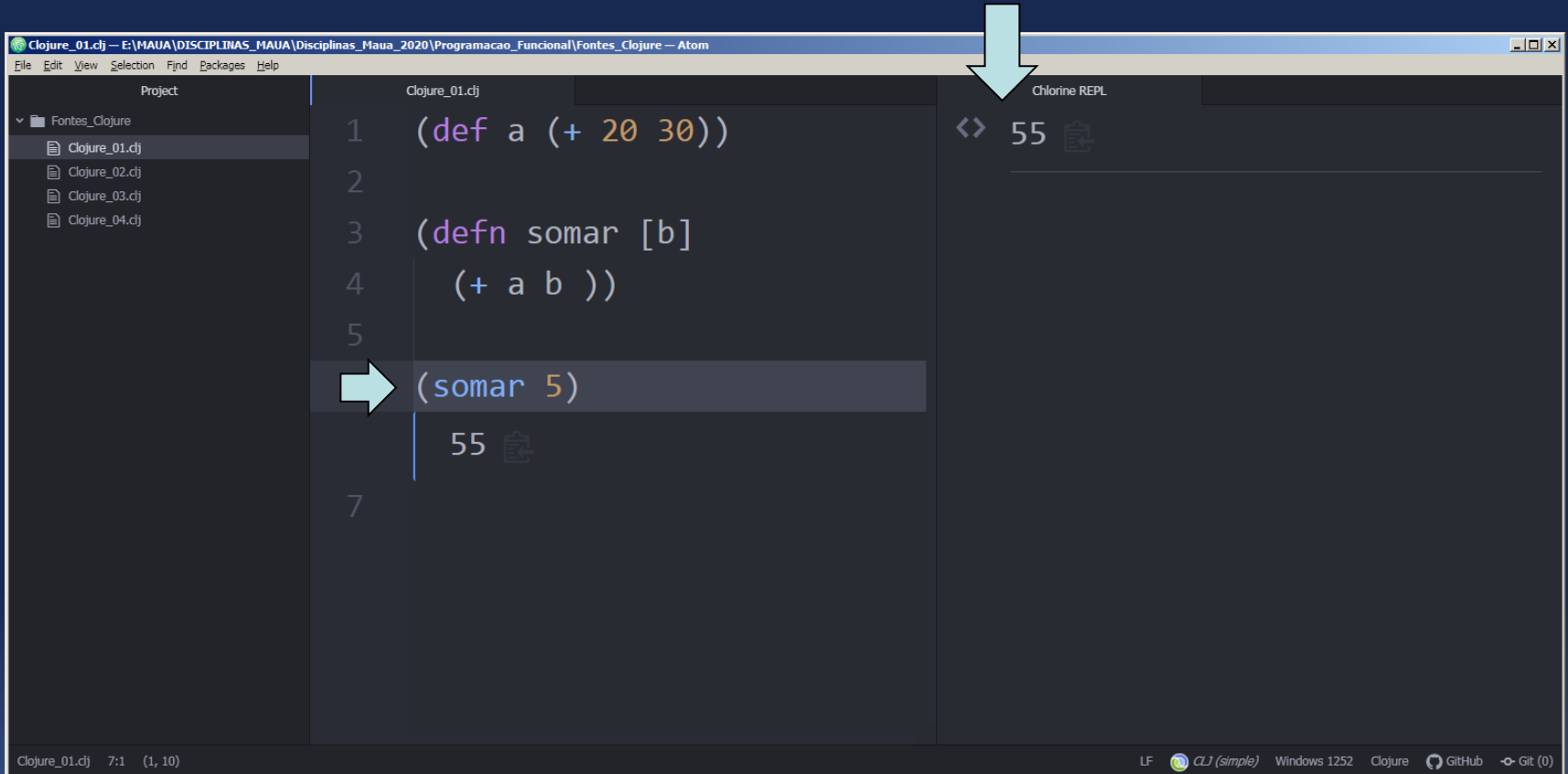
```
1 (def a (+ 20 30))  
2  
3 (defn somar [b]  
4   (+ a b ))  
5  
6 (somar 5)  
7
```

Three green arrows point to lines 1, 3, and 6, indicating the selection of top blocks. The right pane shows the 'Chlorine REPL'.

At the bottom status bar, the following information is visible: Clojure_01.cj 2:1 (1, 18) | LF | CLJ (simple) | Windows 1252 | Clojure | GitHub | Git (0)

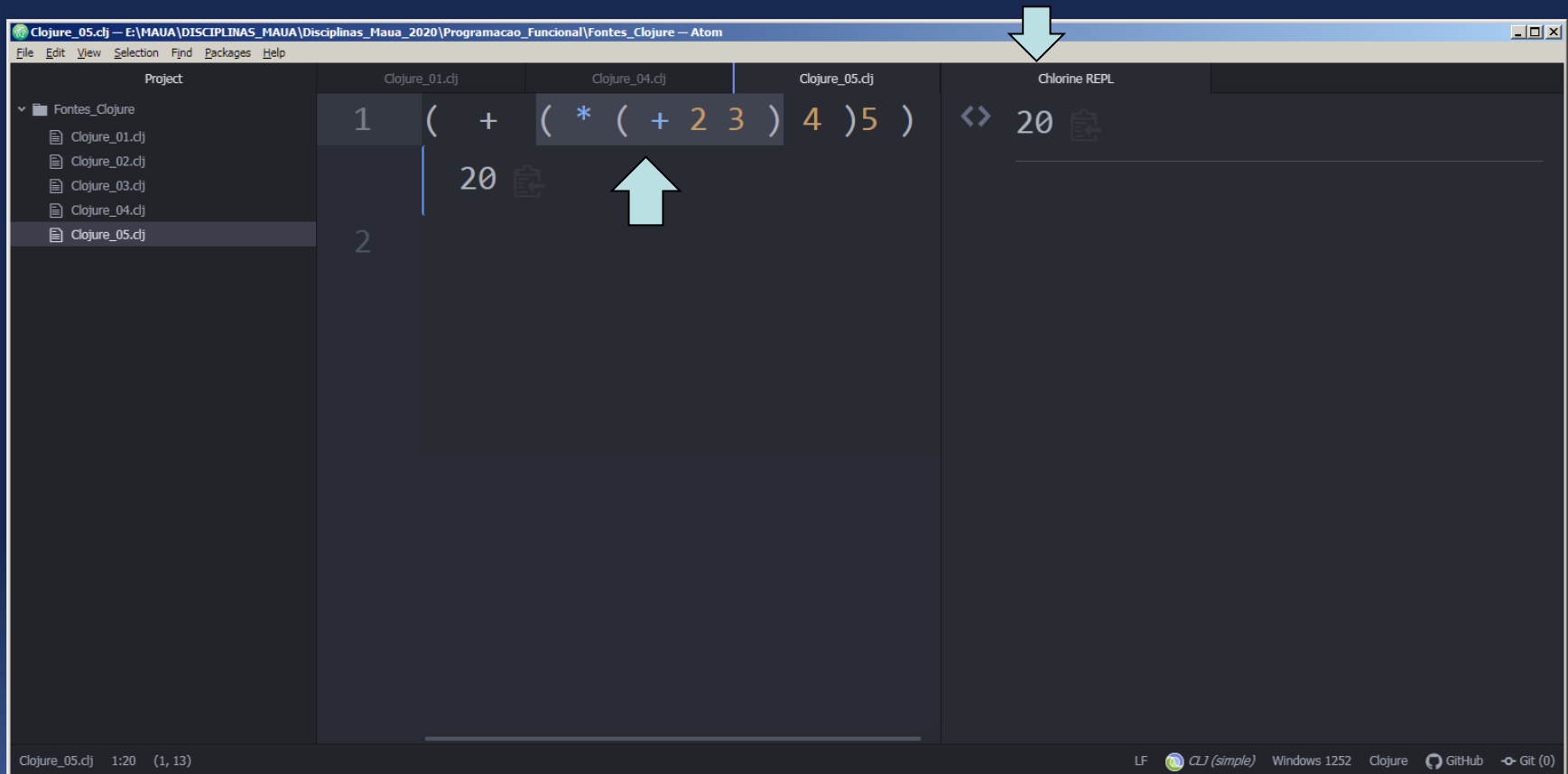
Evaluate "top-blocks"

- ✓ Por exemplo, marcar o top block (somar 5) e teclar => **Ctrl + Enter**



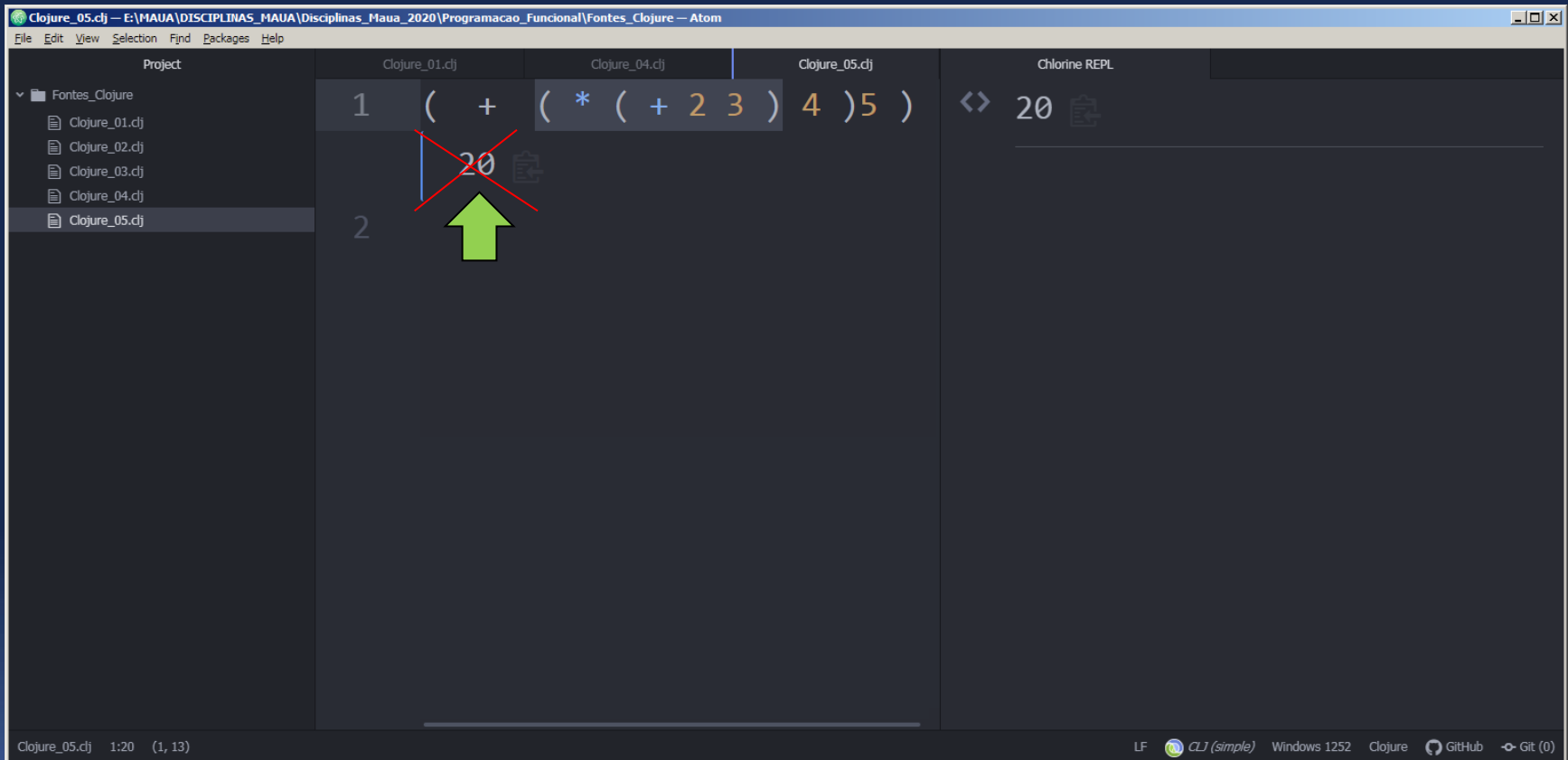
Evaluate “blocks”

- ✓ Hot Key: **Ctrl + Shift + Enter**
- ✓ Selecionar o block desejado e teclar => **Ctrl + Shift + Enter**



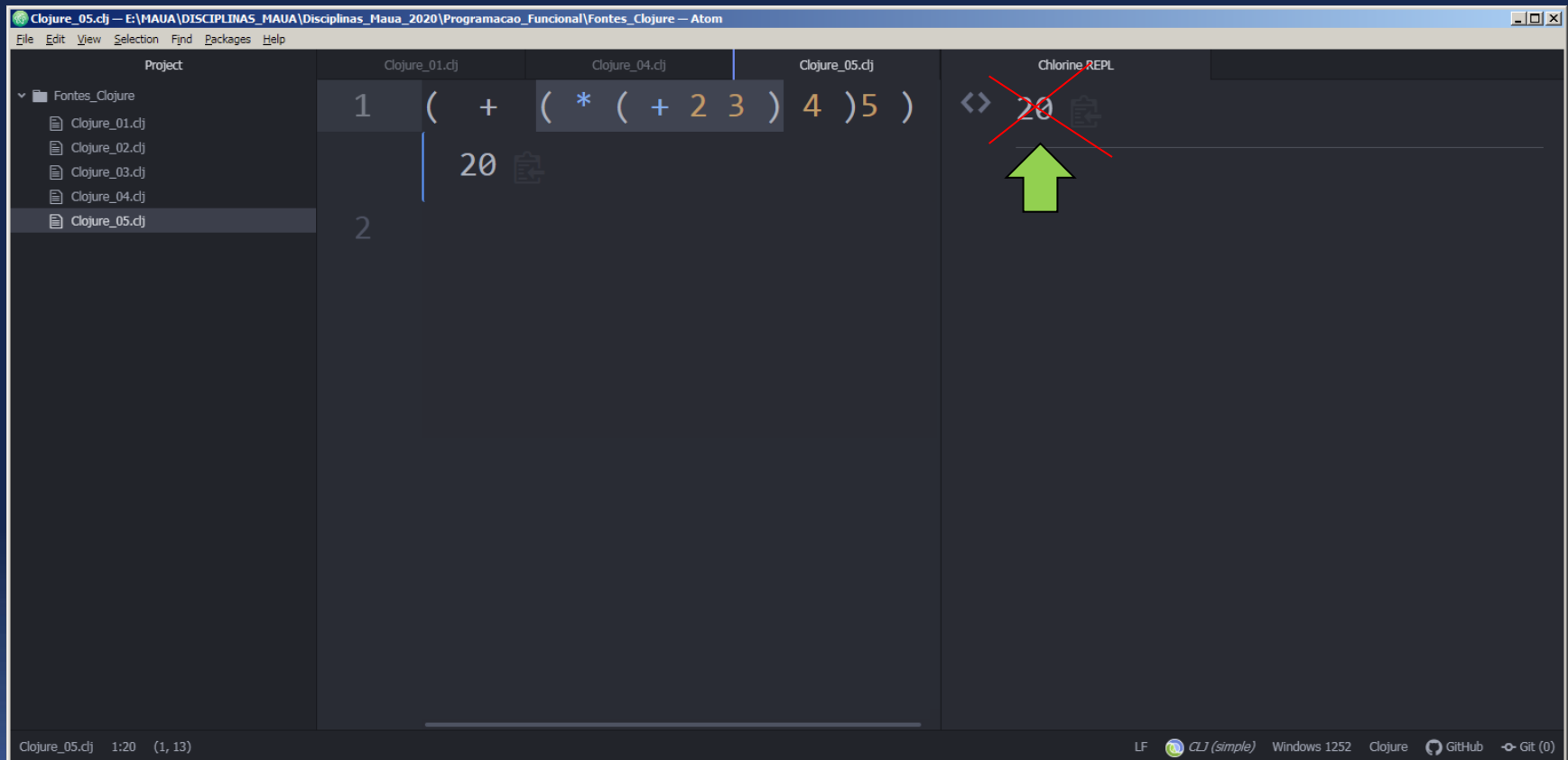
Clear Inline Results

✓ Pode também ser feito pela hot key: **Ctrl + Shift + L**



Clear Console

✓ Pode ser feito pela hot key: **Ctrl +k**



Clear console

✓ Ctrl + Shift + p > Chroline: **Clear Inline Results**

